Ministry Of Higher Education

Modern Academy For Computer

Science and Management Technology

In Maadi

LiverGuardian System

Submitted to

Modern Academy

Computer Science And Management Technology in Maadi
Computer Science Department

Team Members :

Omar Abdelfattah Mostafa(Team Leader)
Ahmed Hussien Sayed Ali Abo Omar
Ali Ahmed Ali Abdelfattah Khalifa
Mohamed Anwer Ibrahim Al Amir
Mahmoud Tolpa AbdElraouf AboSaleh

Supervised by

Dr: Mohamed Abdelfattah

Eng: Mohamed Hassan

Academic Year 2025 / 2026

Name	ID
Ahmed Hussien Sayed	12300421
Ali Ahmed Ali	12300001
Omar Abdelfattah ElSalhy	12300349
Mohamed Anwer Ibrahim	12300185
Mahmoud Tolpa	12300244

Acknowledgements

We would like to extend our sincere gratitude to everyone who contributed to the successful development and completion of the **LiverGuardian System**. This project would not have reached its current form without the guidance, support, and encouragement of many individuals who played a vital role throughout the journey.

First and foremost, we express our deepest appreciation to **Modern Academy** for providing the academic environment, resources, and learning opportunities that enabled us to pursue this project. The institution's continuous support for research, innovation, and scientific growth has been a cornerstone of our development.

We would like to convey our special gratitude to our project supervisor ,

Dr. Mohamed Abdelfattah whose guidance, expertise, and constructive feedback were essential at every stage of the **LiverGuardian System**. His deep knowledge in medical imaging, artificial intelligence, and advanced analytical techniques helped us refine our ideas, overcome technical challenges, and maintain a clear research vision.

We are equally grateful to our teaching assistant, **Eng. Mohamed Hassan**, for his continuous follow-up, technical support, and practical insights. His availability and problem-solving mindset played a major role in keeping the workflow organized and ensuring steady progress throughout the project.

Our appreciation also goes to the faculty members of the **Computer Science Department**, whose teachings and mentorship over the past years laid the foundation upon which this project was built. The concepts, skills, and scientific

approach gained through their courses formed the backbone of our technical work in medical imaging and machine learning.

We also acknowledge with gratitude the global open-source communities and research institutions whose tools, libraries, datasets, and scientific contributions supported the development of the **LiverGuardian System**. Their dedication to advancing knowledge has been instrumental in enabling us to work with modern AI-driven medical imaging techniques.

Finally, we would like to extend heartfelt thanks to our families and friends for their continuous encouragement, patience, and unwavering support. Their belief in our abilities and their understanding during the demanding periods of this project have been a constant source of motivation.

This work represents not only our technical accomplishment but also the collective support and influence of all the individuals and organizations who contributed to our journey. We are truly grateful for their impact and guidance.

Abstract

The **LiverGuardian System** is an AI-powered healthcare platform designed to support the early detection and diagnosis of liver cancer through the integration of medical imaging analysis and clinical data assessment. Liver cancer remains one of the leading causes of cancer-related mortality, and delayed diagnosis often reduces treatment effectiveness and patient survival rates. To address this challenge, LiverGuardian provides a comprehensive framework that combines artificial intelligence techniques with patient medical data to support early screening and clinical decision-making.

The proposed system utilizes three specialized AI pathways operating on different data modalities:

1. **Clinical Risk Assessment Model (Machine Learning)**
   A CatBoost-based classification model trained on structured clinical records, blood test results, and patient medical history to estimate liver cancer risk and provide rapid clinical assessment.
2. **3D CT Segmentation Model**
   A volumetric liver tumor segmentation model based on a 3D Residual U-Net architecture. The model processes CT scans to accurately identify liver structures and tumor regions.
3. **3D MRI Segmentation and Classification Pipeline**
   A multi-stage MRI analysis framework that performs tumor segmentation followed by lesion classification using deep learning techniques. The pipeline processes multi-sequence MRI volumes to support detailed tumor characterization and diagnostic analysis.

The clinical risk assessment model was trained using a dataset containing approximately 5,000 patient records, while the CT and MRI models were

developed and evaluated using specialized medical imaging datasets consisting of volumetric CT scans and multi-sequence MRI studies.

Experimental results demonstrated promising performance across all AI components. The Clinical Risk Assessment model achieved an overall accuracy of **98.0%** and an ROC-AUC score of **99.0%**. The CT Segmentation model achieved a Dice Coefficient of **0.9094**, while the MRI analysis pipeline demonstrated effective segmentation and classification capabilities for multiple liver lesion categories.

To enhance clinical interpretability, explainability techniques were incorporated into the imaging pipelines, allowing better understanding of model predictions and supporting medical decision-making processes.

Beyond the AI models, LiverGuardian is implemented as a complete healthcare platform built using **Next.js 15**, **TypeScript**, and **.NET REST APIs**. The system includes patient self-screening, medical record management, doctor review workflows, appointment scheduling, real-time notifications, and an interactive healthcare chatbot.

The LiverGuardian System demonstrates how artificial intelligence can be effectively integrated into modern healthcare solutions to support early liver cancer detection, improve diagnostic efficiency, and assist healthcare professionals in clinical decision-making.

Table of Contents

Abstract 5

Chapter 1 12

1.1 Introduction 13

1.2 Background 14

1.3 Motivation 14

1.4 Problem Definition 15

1.5 Why LiverGuardian? 16

1.6 Objectives 17

1.7 Expected Outcome.....18

Chapter 2 19

Related Work and System Methodology..... 20

2.1 Related Works and Architectural Foundations 20

2.1.1 U-Net for Biomedical Image Segmentation..... 20

2.1.2 3D Volumetric Segmentation Architectures 20

2.1.3 Multi-Stage and Hierarchical Frameworks 21

2.2 Proposed LiverGuardian Technical Pipeline 21

2.2.1 Clinical Risk Stratification Model 21

2.2.2 Volumetric 3D CT Segmentation Engine..... 22

2.2.3 Multi-Phase Cascaded 3D MRI Diagnostic Engine 22

2 Cascaded Volumetric Segmentation Architecture and Performance 23

3. Hierarchical Multi-Stage Classification Subsystem and Empirical Performance..... 24

2.3 Optimization Protocols and Training Regularization 25

2.4 System Performance Metrics and Evaluation Criteria 25

2.5 Data Privacy, Source Provenance, and System Validation 26

Chapter 3 27

3.1 Development Methodology (Agile – Scrum & anban Hybrid) 28

3.2 Functional Requirements.....30

3.3 Non-Functional Requirements 33

3.4 Risk Analysis 36

3.5 Use Cases 38

3.6 Sequence Diagrams 40

3.7 Process Models.....41

3.8 Feasibility Analysis.....42

3.9 Stakeholder Analysis.....43

Chapter 4 45

4.1Design.....46

4.1.1 Web 46

4.3 Artificial Intelligence.....78

4.3.1 Artificial Intelligence in Healthcare 78

4.3.2 Machine Learning in Clinical Prediction..... 78

4.3.3 Deep Learning for Medical Image Analysis..... 79

4.3.4 Computed Tomography (CT) 79

4.3.5 Magnetic Resonance Imaging (MRI)..... 79

4.3.6 Medical Image Segmentation..... 80

4.3.7 Medical Image Classification 80
4.3.8 Explainable Artificial Intelligence (XAI) 80
4.3.9 Web-Based Healthcare Systems..... 80
4.3.10 AI-Powered Chatbots in Healthcare 81
Chapter 5 82
5.1 Web Application 83
5.1.1 Front End 83
5.1.2 Back End 87
5.2 AI Models Overview.....90
5.2.1 Machine Learning Module Documentation 90
1. Data Loading.....90
2. Dataset Exploration.....91
3. Statistical Analysis.....91
4. BMI Outlier Handling.....92
5. AFP Outlier Treatment.....92
6. Liver Function Score Outlier Treatment 93
7. Feature Selection and Data Splitting.....93
8. Categorical Feature Definition.....94
9. CatBoost Model Construction.....94
10. Model Training 95
11. Model Evaluation.....96
12. Patient Risk Prediction System.....96
13. Model Saving.....97
5.2.2 MRI Module Documentation 98
1-Data loading 98
2. Dataset Preparation.....99
3. Data Preprocessing 101

4. Dataset Information 101
5. Segmentation Model 1 (DWI-Based) 103
6. Segmentation Model 2 (T2WI-Based) 104
6. Cascaded Segmentation Model 104
7. Binary Classification Model 105
8. Malignant Lesion Classification.....106
9. Benign Lesion Classification.....106
10. Training Strategy.....106
11. Loss Functions.....107
12. Test Evaluation 107
13. Inference Pipeline 107
13. MRI Analysis Workflow 108
5.2.3 CT Module Documentation..... 109
5.2.4 AI Models Integration 123
5.2 Testing Methodology 123
5.2.1 CT Segmentation Model Testing 124
5.2.2 MRI Segmentation Model Testing.....125
5.2.3 MRI Classification Model Testing.....126
6.2.4 AI Models Results Analysis 127
5.2.5 API Testing.....128
5.3 Test Case Design 130
5.4 Integration Testing 131
5.5 End-to-End (E2E) Manual Testing.....132
5.6 End-to-End (E2E) Automation Testing 133
5.7 Performance & Load Testing.....133
5.8 Security Testing 134
5.9 Accessibility & Usability Testing 135
5.10 Quality Assurance Measures.....135

Chapter 6 137

6.1 Conclusion 138

6.2 Future Work 138

References..... 140

Chapter 1

1.1 Introduction

This section provides a high-level overview of the LiverGuardian project. Liver cancer represents one of the most challenging oncological problems worldwide — and is Egypt's single deadliest cancer, responsible for 28.3% of all cancer deaths nationally. The primary clinical challenge lies in the extreme variability and heterogeneity of hepatic lesion presentations across different imaging modalities. Liver tumor appearances vary significantly across contrast-enhanced CT phases, multi-sequence MRI scans, and individual patient clinical demographics, making manual radiological interpretation highly demanding and prone to inter-observer variability.

The project's primary goal is to leverage state-of-the-art machine learning and deep learning architectures to provide a multi-layered diagnostic system. This is achieved through three specialized pipelines:

- A **CatBoost gradient-boosting classifier** to analyze structured clinical records and patient history for rapid risk scoring.
- An optimized **3D Residual U-Net architecture** for automated volumetric tumor segmentation from 3D CT scans.
- A multi-stage framework combining **3D Residual U-Net** and **3D ResNet** architectures to perform sequential cascaded segmentation and hierarchical classification on multi-sequence MRI volumes.

By providing detailed tumor localization and precise lesion typing, LiverGuardian aims to address the critical clinical gap of late-stage diagnosis. Early and accurate detection dramatically increases the 5-year survival rate from the current 17% (typically observed in late-stage presentations) toward the 70%+ survival rate achievable with timely, Stage 1 diagnostic intervention.

1.2 Background

Hepatocellular Carcinoma (HCC) accounts for the majority of primary liver cancers globally. In Egypt specifically, the historical prevalence of Hepatitis B and C, liver cirrhosis, and metabolic risk factors creates a uniquely high-risk patient population. Existing clinical workflows rely on a multi-step process: biochemical screening (such as AFP, ALT, and AST levels) followed by imaging confirmation using contrast-enhanced CT or gadolinium-enhanced MRI. However, this sequential approach requires specialized radiologists, expensive equipment, and weeks of waiting — barriers that prevent early-stage identification for most Egyptian patients, particularly in underserved areas.

Deep learning has demonstrated transformative potential in medical imaging and clinical decision support. The U-Net family of architectures, specifically 3D variants, has become the standard for volumetric medical segmentation due to its ability to capture spatial context across contiguous slices. Simultaneously, gradient-boosting classifiers (such as XGBoost, LightGBM, and CatBoost) have consistently outperformed traditional machine learning and standard statistical methods on structured clinical tabular datasets. LiverGuardian brings these algorithmic advances together into a comprehensive, multi-layered, and clinically deployable framework designed to assist healthcare professionals across each of these diagnostic stages

1.3 Motivation

The motivation behind the LiverGuardian System stems from both healthcare and technological perspectives.

- **Clinical-centric:** To create an advanced clinical decision-support framework that provides medical professionals with fast, transparent, and evidence-based AI predictions. By assisting in early detection and localized tumor analysis, the system aims to significantly reduce diagnostic delays,

enable earlier treatment interventions, and ultimately improve survival outcomes for liver cancer patients in Egypt.

- **Technical-centric:** To design and implement a comprehensive, multi-layered end-to-end AI pipeline that spans diverse medical data modalities. This includes executing precise data preprocessing, feature engineering, 3D deep learning model training, and performance evaluation on structured clinical tables, volumetric CT scans, and multi-sequence MRI volumes, culminating in a real-world, full-stack web deployment for a high-stakes medical domain

1.4 Problem Definition

The problem is formally defined through three independent, specialized tasks addressing clinical screening, volumetric tumor localization, and multi-sequence lesion differentiation. Each sub-system is characterized as follows:

- **Clinical Risk Assessment Model (CatBoost):**
  - **Input:** 14 structured clinical, biochemical, and demographic features (including age, gender, BMI, alcohol use, smoking, Hepatitis B/C, liver function indicators, AFP levels, history of cirrhosis, family cancer history, physical activity, and diabetes).
  - **Output:** A continuous risk probability score along with a three-tiered categorical risk classification (High Suspicion, Moderate Risk, or Low Risk).
- **Volumetric CT Segmentation Model (3D Residual U-Net):**
  - **Input:** 3D volumetric NIfTI CT scans preprocessed and normalized using Hounsfield Unit (HU) windowing.
  - **Output:** A precise 3D per-voxel binary tumor segmentation mask mapping the spatial volume and exact boundaries of the hepatic lesion.
- **Multi-Sequence 3D MRI Cascade & Hierarchical Pipeline:**

- **Input:** Volumetric multi-phase 3D MRI sequences (comprising 8 scan types: DWI, T2WI, C-pre, C+A, C+V, C+Delay, In Phase, and Out Phase).
- **Output:** Executed via a dual-stage pipeline:
  - *Cascaded Segmentation Stage:* Uses three sequential 3D Residual U-Net models combining DWI and T2WI modalities to generate a refined, consolidated 3D final tumor mask.
  - *Hierarchical Classification Stage:* Integrates the 8 MRI sequences with the generated final mask into a 3D ResNet-based tree architecture. It first executes a binary classification (Malignant vs. Benign). Depending on the outcome, it branches into granular sub-type classification: Malignant differentiation (HCC, ICC, or Metastasis) or Benign differentiation (Hemangioma, Abscess, Cyst, or FNH).

1.5 Why LiverGuardian?

While commercial radiology AI tools exist (e.g., Paige Liver, Perspectum LiverMultiScan), they operate as proprietary, high-cost "black boxes" accessible only to high-resource hospitals and international medical centers. LiverGuardian is specifically architected for the Egyptian and MENA healthcare context: it is open, fully documented, and strategically structures three distinct processing pipelines that closely mirror real clinical workflows.

Instead of relying on opaque numeric scores, LiverGuardian delivers high clinical interpretability. The system provides precise 3D volumetric segmentation masks that overlay directly onto imaging scans, giving radiologists clear, structural, and spatial evidence of tumor boundaries. The name *LiverGuardian* directly reflects our ambition to build a digital guardian standing between high-risk patients and late-stage cancer diagnosis through multi-layered screening and analysis.

1.6 Objectives

- **Clinical Tabular Pipeline:**
    - Collect, clean, and preprocess a structured clinical dataset containing 5,000 patient records encompassing 14 biochemical, historical, and demographic features.
    - Perform comprehensive Exploratory Data Analysis (EDA) to evaluate feature distributions, handle outliers (such as BMI and AFP metrics), and uncover critical clinical correlations.
    - Train and optimize a CatBoost Classifier to achieve an overall predictive Accuracy of 98% and an ROC AUC Score of 99% for multi-tiered patient risk assessment.
- **Volumetric CT Segmentation Pipeline:**
    - Implement and train an optimized 3D Residual U-Net architecture to automate localized tumor extraction from 3D CT scans.
    - Apply Hounsfield Unit (HU) windowing and Mixed-Precision training to achieve a high-fidelity volumetric target Dice Coefficient of 0.9094.
- **Multi-Sequence 3D MRI Pipeline:**
    - Design and train a multi-stage Cascaded Segmentation network using three sequential 3D Residual U-Nets to combine DWI and T2WI sequences, reaching a final mask Dice Score of 0.456.
    - Construct a Hierarchical Classification tree based on 3D ResNet architectures to ingest 8 MRI sequences alongside the generated mask, executing binary risk sorting (Malignant vs. Benign) followed by granular multi-class sub-type differentiation.
    - Evaluate and document the structural limitations imposed by dataset scale (498 patients) and profound class imbalances (e.g., HCC vs. FNH) on type classification accuracy.
- **System Integration & Deployment:**

- Deploy the unified diagnostic sub-systems into a production-ready, full-stack web application featuring a Next.js frontend, a .NET REST API, an interactive guidance chatbot, and real-time clinical notification workflows powered by SignalR.

• Deploy a full-stack web platform (Next.js + .NET) supporting patient, doctor, and admin workflows.

1.7 Expected Outcome

- **Clinical ML Model:** A trained CatBoost model (catboost_liver_model.cbm) achieving **98% accuracy** and **99% ROC-AUC** for instant risk screening.
- **3D CT Pipeline:** A 3D Residual U-Net model optimized for volumetric tumor segmentation, achieving a **0.9094 Dice Coefficient** and **99.45% Binary Accuracy**.
- **3D MRI Pipeline:** A cascaded and hierarchical framework achieving a **0.456 Final Dice Score** for segmentation, **65.3% binary accuracy** (Malignant vs. Benign), **70.6% accuracy** for malignant subtypes, and **59.4% accuracy** for benign subtypes.
- **Full-Stack Platform:** A production-ready web platform deployed on a live hosting URL, integrating all three independent diagnostic pipelines with doctor/patient workflows and a guidance chatbot.

Chapter 2

Related Work and System Methodology

This chapter delineates the foundational frameworks, architectural milestones, and procedural methodologies undergirding the LiverGuardian system. It establishes the mechanical bridge between clinical data structures and deep learning engines designed for computer vision and non-invasive pathology tracking.

2.1 Related Works and Architectural Foundations

The integration of Artificial Intelligence (AI) and advanced medical imaging for hepatic pathology diagnosis has evolved rapidly. Modern configurations transition from isolated theoretical networks to multi-modal engineering workflows combining volumetric segmentation, non-invasive clinical predictive modeling, and multi-stage hierarchical classification.

2.1.1 U-Net for Biomedical Image Segmentation

Ronneberger et al. (2015) introduced the U-Net architecture, establishing a cornerstone for modern medical image analysis. Its symmetrical contracting (encoder) and expanding (decoder) paths, combined with high-resolution skip connections, ensure precise pixel-level localization even when training on restricted annotated datasets. However, traditional 2D U-Nets isolate individual image slices, completely ignoring the cross-slice volumetric context inherent to standard clinical acquisitions.

2.1.2 3D Volumetric Segmentation Architectures

To retain multi-slice spatial dependencies, deep learning architectures evolved into three dimensions (Çiçek et al., 2016). By employing 3D convolutions and volumetric spatial pooling, models capture the spatial structural continuities of internal organs across contiguous slices. Modern clinical platforms heavily implement 3D residual structures—integrating residual skip connections within local convolutional blocks (He et al., 2016)—to mitigate vanishing gradient

artifacts during deep optimization and preserve fine boundaries of complex tumors like Hepatocellular Carcinoma (HCC).

2.1.3 Multi-Stage and Hierarchical Frameworks

While standard 3D networks excel at single-task processing, complex diagnostic domains involving highly heterogeneous tumors require multi-stage architectures. Recent advancements demonstrate that cascading multiple 3D U-Net structures sequentially significantly enhances volumetric segmentation in noisy environments by progressively refining region-of-interest masks. Furthermore, integrating these localized masks into hierarchical classification trees powered by 3D ResNet architectures allows systems to first sort global tissue states (Malignant vs. Benign) before executing granular, multi-class sub-type differentiation, mapping directly to real-world clinical staging workflows.

2.2 Proposed LiverGuardian Technical Pipeline

The LiverGuardian framework addresses the diagnostic process through three independent, specialized processing pipelines:

2.2.1 Clinical Risk Stratification Model

- **Model Engine:** CatBoost Classifier, optimized to process categorical and numerical clinical features robustly without extensive manual target encoding.
- **Core Clinical Indicators:** Processes **14 distinct features**, including Age, Gender, BMI, Alcohol Use, Smoking, Hepatitis B/C status, Liver Function Indicators, AFP Levels, Cirrhosis History, Family Cancer History, Physical Activity, and Diabetes.
- **Outputs:** A continuous probability score mapped to a **three-tiered categorical risk classification** (High Suspicion, Moderate Risk, or Low Risk).

2.2.2 Volumetric 3D CT Segmentation Engine

- **Preprocessing & Augmentation:** Standardizes raw voxel intensities using Hounsfield Unit (HU) windowing tailored for soft-tissue hepatic visualization. Data arrays are dynamically normalized and augmented via multi-axis spatial flipping, rotations, and intensity scaling.
- **Architecture:** A **3D Residual U-Net** optimized with Mixed Precision for localized voxel-level tumor segmentation.
- **Optimization:** Trained using a compound loss blending Binary Cross-Entropy (BCE) with Dice Similarity Coefficient (DSC) loss to stabilize boundary extraction against high background-to-tumor class imbalances.

2.2.3 Multi-Phase Cascaded 3D MRI Diagnostic Engine

- **Input Data:** Volumetric multi-phase 3D MRI data spanning **8 distinct clinical sequences** (DWI, T2WI, C-pre, C+A, C+V, C+Delay, In Phase, Out Phase).
- **Stage 1: Cascaded Segmentation:** Utilizes three sequential 3D Residual U-Net models combining DWI and T2WI modalities to generate a refined, consolidated 3D final tumor mask.
- **Stage 2: Hierarchical Classification:** Ingests all 8 MRI sequences alongside the generated mask into a 3D ResNet-based tree architecture. It executes a primary binary classification (Malignant vs. Benign), then branches into granular sub-type classification: Malignant (HCC, ICC, Metastasis) or Benign (Hemangioma, Abscess, Cyst, FNH).

1. Multi-Phase Input Configuration

The system captures and processes eight distinct clinical acquisition sequences simultaneously for each patient to generate complete diagnostic vectors:

Sequence Type	**Clinical / Functional Role**

C-pre	Unenhanced baseline T1-weighted sequence
C+A	Arterial contrast enhancement phase for vascularized lesions
C+V	Portal venous contrast enhancement phase for hepatic venous washouts
C+Delay	Delayed equilibrium contrast phase to map interstitial space retention
T2WI	T2-Weighted Imaging for fluid-structural composition and edema tracking
DWI	Diffusion-Weighted Imaging for identifying cellular hyper-density restriction
In Phase	Chemical shift in-phase sequence for tissue fat accumulation tracking
Out Phase	Chemical shift opposed-phase sequence for multi-spectral evaluation

2 Cascaded Volumetric Segmentation Architecture and Performance

The structural localization utilizes a Cascaded Multi-Model Segmentation Network built on **3D Residual U-Net** configurations to extract fine-grained masks from multi-sequence inputs:

- **Model 1 (DWI Specialist):** Focuses heavily on cellular restriction anomalies, achieving a standalone validation Dice Score of **0.453**.

- **Model 2 (T2WI Specialist):** Isolates fluid structures, hyperintensities, and morphological boundaries, demonstrating a validation Dice Score of **0.429**.
- **Model 3 (Cascade Fusion Engine):** Synthesizes cross-sequence spatial features from both streams to mitigate phase-isolated artifacts. By executing contextual consolidation, the final Cascade Model established the peak volumetric boundary performance with a **Final Dice Score of 0.456**.

3. Hierarchical Multi-Stage Classification Subsystem and Empirical Performance

Rather than performing a flat single-step multi-class sorting, the classification infrastructure is divided into a hierarchical tree optimized via specialized PyTorch modules to mimic human differential diagnosis:

- **Stage 1 (Binary Malignancy Filter):** Processes multi-phase features inside the localized lesion mask to classify findings into Benign or Malignant categories. This foundational gatekeeper attained an accuracy of **62.5%** on the Validation Cohort and **65.3%** on the Evaluation Test Set.
- **Stage 2-A (Malignant Sub-class Classifier):** Routes malignant volumes to differentiate between **3 clinical classes** (HCC, ICC, and Metastasis). This expert subsystem achieved a strong **70.6% accuracy** on the Validation Cohort and **40.8%** on the diversified Test Set, highlightening generalization gaps typical in distinct multi-center data.
- **Stage 2-B (Benign Sub-class Classifier):** Routes benign volumes to classify etiologies across **4 distinct classes** (Hemangioma, Abscess, Cyst, and FNH). This sub-classifier reached an accuracy of **59.4%** on the Validation Cohort and **50.0%** on the Evaluation Test Set.

2.3 Optimization Protocols and Training Regularization

To manage high-dimensional volumetric tensors, stabilize parameters, and enhance generalization, the following deep learning mechanics are implemented directly within the core training loops:

- **Mixed Precision Computing:** Utilizes native mixed-precision casting (torch.cuda.amp) using FP16 representations. This significantly reduces the VRAM footprint, allowing for an efficient batch size of 2 on consumer laptop hardware (specifically targeted to accommodate the NVIDIA GeForce RTX 4050 Laptop GPU with 6.44 GB VRAM), and accelerates training via dynamic gradient scaling.
- **Optimization and Schedulers:** Weight updates are driven by advanced optimizers coupled with a CosineAnnealingWarmRestarts learning rate schedule, allowing the model to escape local minima during intense optimization cycles. Overfitting is strictly controlled via adaptive Early Stopping with a patience of 15 epochs based on validation loss tracking.

2.4 System Performance Metrics and Evaluation Criteria

2.4.1 Tabular Risk and Multi-Stage Classification Metrics

Performance is benchmarked using Accuracy, Sensitivity (Recall), Specificity, and the Area Under the Receiver Operating Characteristic curve (ROC-AUC), formulated via True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN):

- **Overall Diagnostic Accuracy:** Measures the ratio of correctly predicted clinical cases out of the total patient cohort.
- **Sensitivity (Recall):** Evaluates the system's ability to correctly identify malignant tumors or high-risk patients without missing critical pathology.
- **Specificity:** Evaluates the engine's capability to correctly identify benign conditions, preventing unnecessary invasive biopsies.

2.4.2 Volumetric 3D Segmentation Accuracy

To measure the exact geometric accuracy of the 3D localized masks generated by the **3D Residual U-Net** engines against the expert-annotated ground truths, the framework tracks spatial overlap metrics:

- **Dice Similarity Coefficient (DSC):** Measures spatial overlap accuracy on a scale from 0 to 1, where 1 represents a perfect voxel match between the predicted tumor volume and manual clinical segmentation.
- **Intersection over Union (IoU / Jaccard Index):** Quantifies the total bounding accuracy of the segmented hepatic lesions.

2.5 Data Privacy, Source Provenance, and System Validation

The system establishes training integrity by utilizing diverse, fully de-identified multi-center open repositories:

1. **CT Dataset:** Sourced from the **Liver Tumor Segmentation (LiTS) Challenge** collections, providing diverse clinical scans containing complex standard hepatic tumors.
2. **Clinical Records:** Derived from the **UCI Machine Learning Repository** historical records, validating non-invasive risk predictive models.
3. **MRI Datasets:** Sourced from multi-phase public **Kaggle repositories and The Cancer Imaging Archive (TCIA)** annotations, totaling 498 complete patients (3,984 individual phase scans) with verified histological classifications mapped directly to the seven primary diagnostic outcomes.

All operations maintain strict compliance with global patient data privacy regulations through full anonymization at the source repositories.

Chapter 3

3.1 Development Methodology (Agile – Scrum & anban Hybrid)

The development of Liver Guardian adopted a hybrid Agile methodology that combines the structured iterative practices of Scrum with the continuous workflow management capabilities of Kanban. This approach was selected due to the dynamic nature of AI-driven medical systems, where requirements, model performance, and integration workflows evolve continuously throughout the development lifecycle.

Traditional development methodologies such as Waterfall were considered less suitable because the project required flexibility, continuous experimentation, and iterative refinement of both software components and AI models. The hybrid Agile model enabled the team to rapidly adapt to new technical challenges, optimize workflows, and continuously improve system quality.

Key Agile Principles Applied

• **Iterative Development:**
The project was divided into multiple short development cycles (sprints), allowing the team to incrementally implement and evaluate features such as AI prediction modules, medical record management, and real-time communication systems.

• **Incremental Delivery:**
Features were developed and deployed progressively. Each sprint focused on delivering functional system increments such as authentication, appointment management, AI integration, and session management.

• **Cross-Functional Collaboration:**
The development team collaborated across backend development, frontend implementation, AI model development, database design, and infrastructure management.

• **Continuous Feedback and Improvement:**
Frequent internal reviews and testing sessions enabled the team to identify issues early and refine both functional and non-functional aspects of the system.

• **Adaptability to Change:**
The Agile approach enabled rapid adaptation to evolving AI model requirements, medical workflows, and system security enhancements.

Scrum Practices Implemented

• Product Backlog management using ClickUp.
• Sprint planning and task prioritization.
• Daily stand-up meetings for synchronization and progress tracking.
• Sprint reviews to evaluate completed functionality.
• Sprint retrospectives to improve development workflows.
• GitHub-based version control and collaborative development.

Kanban Workflow Integration

Kanban principles were integrated to manage tasks that required continuous monitoring or were not sprint-bound, including:

• AI model optimization and testing.
• Security monitoring and bug fixing.
• Infrastructure and deployment tasks.
• Background job monitoring and maintenance.

The hybrid Scrum-Kanban model provided both structured planning and flexible workflow management, which was highly suitable for the research-oriented and healthcare-focused nature of the Liver Guardian platform.

[INSERT FIGURE 3.1: Agile Development Methodology Implementation]

Table 3.1: Agile Development Sprints and Deliverables

Sprint	Duration	Key Deliverables
Sprint 1	2 Weeks	Project Planning, Requirement Gathering, Initial Architecture
Sprint 2	2 Weeks	Authentication System, Role-Based Authorization
Sprint 3	3 Weeks	Medical History Management, Medical Records Upload
Sprint 4	2 Weeks	AI Integration for CT Scan Analysis
Sprint 5	2 Weeks	Appointment and Doctor Verification Modules
Sprint 6	2 Weeks	Real-Time Notifications and Chat Sessions
Sprint 7	2 Weeks	Security Enhancements, Logging, Rate Limiting
Sprint 8	1 Week	Testing, Deployment, Documentation

3.2 Functional Requirements

Functional requirements define the core services and capabilities provided by the Liver Guardian platform.

FR-01: User Authentication and Authorization

• The system shall allow users to register and log in securely.
• The system shall support role-based authorization for Patients, Doctors, and Administrators.
• The system shall support JWT-based authentication.
• The system shall provide email verification and password reset functionality.
• The system shall enforce secure session management.

FR-02: Doctor Verification System

• Doctors shall upload required verification documents and certificates.
• The system shall allow administrators to approve or reject doctor accounts.
• Rejected doctor accounts shall be restricted from accessing platform medical services.
• The system shall notify doctors regarding verification status updates.

FR-03: Medical History Management

• Patients shall be able to create and manage their medical history records.
• The system shall store medical indicators such as diabetes status, smoking habits, and related medical conditions.
• Medical history data shall be accessible to authorized doctors.

FR-04: Medical Record Upload

• Patients shall be able to upload CT scans and MRI scans individually.
• The system shall validate uploaded file types and file sizes.
• Uploaded records shall be securely stored and linked to the patient profile.

FR-05: AI-Based Disease Prediction

• The system shall analyze uploaded CT and MRI scans using AI models.
• The system shall analyze patient medical history using predictive AI models.
• The AI module shall return prediction confidence scores and disease severity indications.
• The system shall display AI-generated visualization heatmaps for CT scan analysis.

FR-06: AI Results Management

• Patients shall be able to view their AI prediction results.
• Doctors shall be able to review patient AI results and medical records.
• The system shall store historical AI analysis results for future reference.

FR-07: Appointment Management

• Patients shall be able to request appointments with doctors.
• Doctors shall be able to approve or reject appointment requests.
• Approved appointments shall automatically enable session creation.

FR-08: Real-Time Chat Sessions

• Doctors shall initiate chat sessions at the scheduled appointment time.
• Patients and doctors shall exchange real-time text messages.
• Chat sessions shall automatically expire after 30 minutes if not manually ended.
• Background jobs shall monitor and manage active sessions.

FR-09: Notification System

• The system shall provide real-time notifications using SignalR.
• Notifications shall include appointment updates, AI results, verification status, and session events.

• Notifications shall support extensible metadata storage using dynamic JSON payloads.

FR-10: Dashboard Management

• Patients shall access personal dashboards containing AI results, medical history, sessions, and notifications.
• Doctors shall access dashboards for appointments, sessions, and patient AI reviews.
• Administrators shall access system-wide monitoring dashboards.

FR-11: Logging and Monitoring

• The system shall log application events using Serilog.
• Logs shall support monitoring, searching, and debugging activities.

3.3 Non-Functional Requirements

Non-functional requirements define the quality attributes and operational constraints of the Liver Guardian platform.

NFR-01: Performance

• API requests shall maintain low response latency under normal load conditions.
• AI analysis operations shall process uploaded medical scans efficiently.
• Chat session communication shall support real-time responsiveness.
• Background jobs shall execute automatically without affecting system performance.

NFR-02: Security

• The system shall implement JWT-based stateless authentication.
• Sensitive configuration values shall be stored securely using environment

variables.

• Uploaded files shall undergo validation before processing.

• The system shall implement protection against SQL Injection, XSS attacks, and brute-force attacks.

• IP-based filtering and temporary banning mechanisms shall be applied for malicious requests.

• Login requests shall be protected using rate limiting policies.

• HTTPS communication shall be enforced.

NFR-03: Scalability

• The system architecture shall support future horizontal scalability.

• The application shall maintain modularity using Onion Architecture principles.

• The notification and AI services shall support future distributed deployment.

NFR-04: Maintainability

• The system shall follow Clean Architecture principles.

• Business logic shall remain decoupled from infrastructure and presentation layers.

• Services shall be implemented using abstraction and dependency injection to reduce tight coupling.

NFR-05: Reliability

• Background jobs shall continuously monitor session expiration and system operations.

• Logging and monitoring systems shall support fault diagnosis and troubleshooting.

• The platform shall maintain data consistency and integrity.

NFR-06: Usability

• The web application shall provide responsive user interfaces.
• The system shall support localization and multilingual capabilities.
• User workflows shall remain simple and accessible to non-technical users.

NFR-07: Compatibility

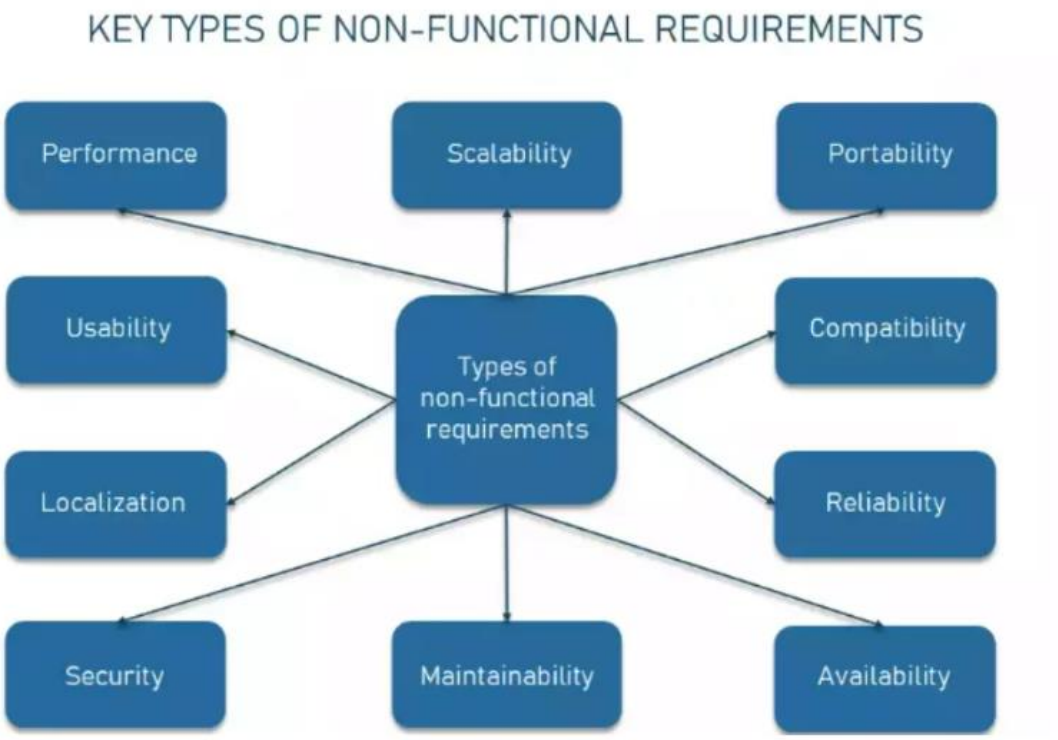
• The platform shall support modern web browsers.
• APIs shall support versioning to maintain backward compatibility.

NFR-08: Availability

• Real-time communication services shall remain available during active sessions.
• Notifications and AI services shall operate continuously under normal operating conditions.

NFR-09: Database Optimization

• Database indexing shall be implemented to improve query performance.
• Efficient relational database design shall be maintained using SQL Server and Entity Framework Core.



3.4 Risk Analysis

Risk analysis was conducted to identify technical, operational, and security-related risks associated with the Liver Guardian platform.

R01: AI Prediction Inaccuracy

• Description: AI models may produce false positives or false negatives during liver cancer detection.
• Likelihood: Medium
• Impact: High
• Mitigation: Continuous model testing, dataset refinement, and doctor review integration.

R02: Medical File Upload Failures

• Description: Large medical files may fail during upload or processing.
• Likelihood: Medium
• Impact: Medium
• Mitigation: File validation, upload restrictions, and optimized storage handling.

R03: AI Service Downtime

• Description: External or internal AI services may become temporarily unavailable.
• Likelihood: Medium
• Impact: High
• Mitigation: Error handling, retry mechanisms, and service monitoring.

R04: Security Vulnerabilities

• Description: Unauthorized access attempts or malicious payloads may target the platform.
• Likelihood: Medium
• Impact: High
• Mitigation: Rate limiting, IP filtering, request validation, and secure authentication mechanisms.

R05: Real-Time Session Failures

• Description: Real-time chat sessions may disconnect or terminate unexpectedly.
• Likelihood: Low
• Impact: Medium
• Mitigation: SignalR monitoring, reconnection handling, and background session monitoring.

R06: Scalability Challenges

• Description: Increasing numbers of users and AI requests may affect system performance.
• Likelihood: Medium
• Impact: Medium
• Mitigation: Optimized architecture design and scalable service separation.

R07: Doctor Verification Errors

• Description: Invalid or fraudulent doctor credentials may bypass verification processes.
• Likelihood: Low
• Impact: High
• Mitigation: Manual administrative verification and document validation.

[INSERT FIGURE 3.4: Risk Assessment Matrix]

3.5 Use Cases

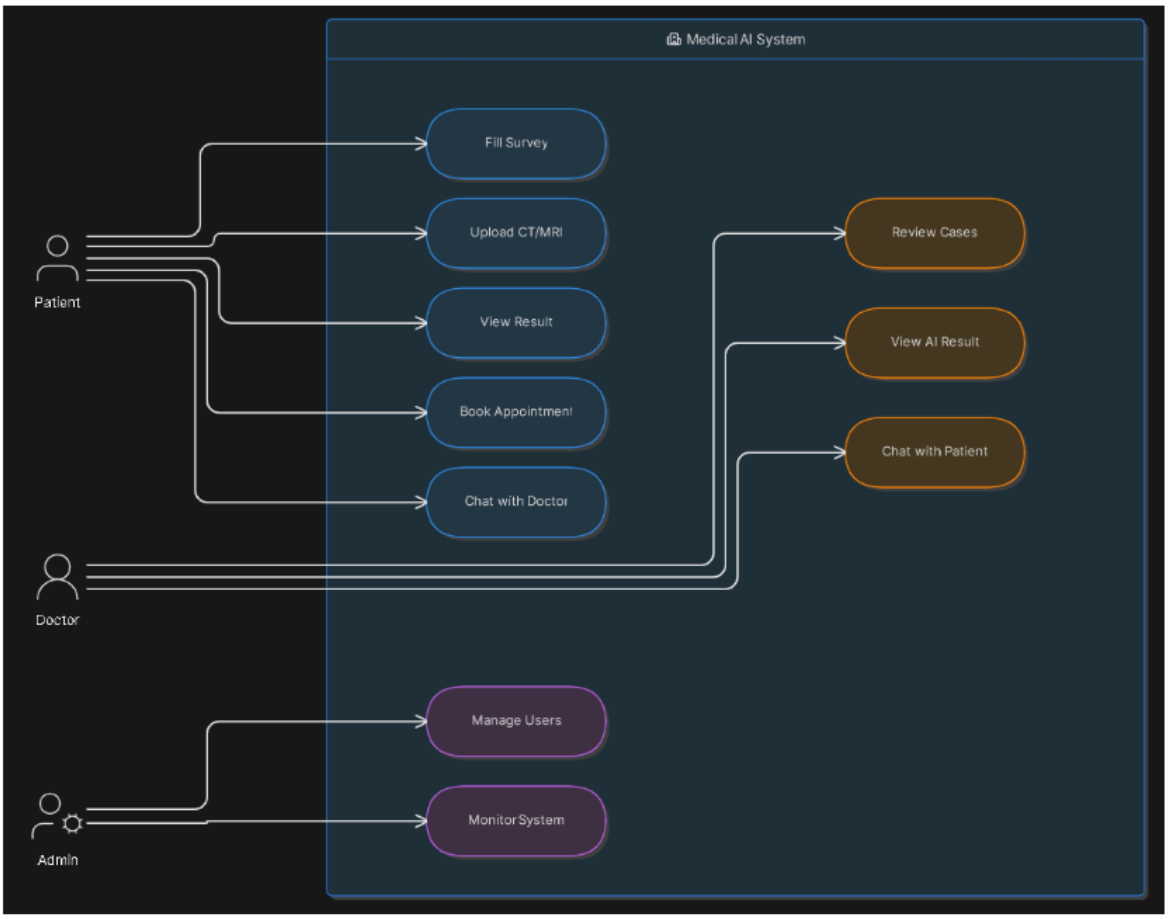
Use cases describe how different actors interact with the Liver Guardian platform to achieve specific goals.

Actors

• Patient
• Doctor
• Administrator
• AI Service

Main Use Cases

- Fill Survey
- Upload Medical Records
- Submit CT/MRI Scans
- Perform AI Analysis
- View AI Results
- Book Appointments
- Approve or Reject Appointments
- Start Chat Sessions
- Send Messages
- Manage Users and Reports



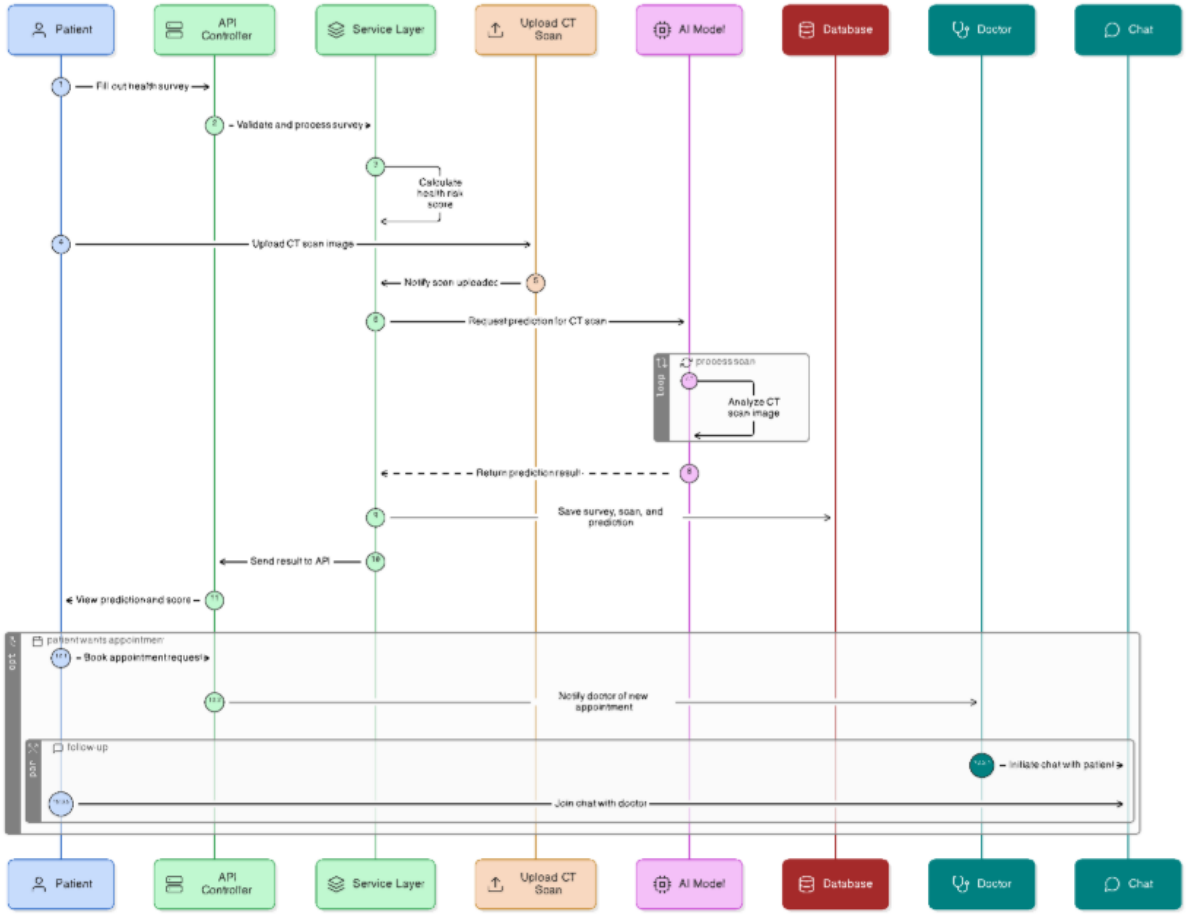
3.6 Sequence Diagrams

Sequence Diagrams were designed to illustrate the interaction sequence between users, system components, AI services, and databases over time. These diagrams provide a detailed view of how requests are processed and how different system entities communicate to complete specific operations.

The Sequence Diagrams illustrate:

- User registration, authentication, and authorization workflows.
- Medical record submission and retrieval processes.
- CT/MRI scan upload and AI analysis request sequences.
- AI prediction generation and result delivery workflows.
- Appointment booking, management, and doctor consultation processes.
- Real-time notification delivery and communication sequences.
- Session management and secure data access interactions.

The diagrams demonstrate the chronological exchange of messages between actors, services, controllers, databases, and external AI components, ensuring a clear understanding of the system's dynamic behavior and operational flow.



3.7 Process Models

Process models were developed to visualize critical operational workflows inside the Liver Guardian platform.

Key Process Workflows

1. Doctor Verification Process

The doctor submits registration information and uploads required verification documents. The administrator reviews the submitted credentials and either approves or rejects the application. Approved doctors gain access to the platform's medical services, while rejected accounts remain restricted.

2. AI Analysis Workflow

The patient uploads CT or MRI scans alongside medical history information. The backend system forwards the request to the AI analysis service, which processes the medical data and returns prediction results. The results are then stored and displayed to both patients and authorized doctors.

3. Appointment and Session Lifecycle

Patients request appointments with doctors through the platform. Doctors review appointment requests and approve suitable schedules. At the appointment time, doctors initiate real-time chat sessions. Hangfire background jobs continuously monitor active sessions and automatically terminate expired sessions when necessary.

4. Notification Workflow

The system generates real-time notifications for major events such as appointment approvals, AI result generation, session expiration, and doctor verification updates. Notifications utilize extensible JSON payloads (DataJson) to support flexible contextual metadata across system modules without requiring schema modifications.

3.8 Feasibility Analysis

Technical Feasibility

The technologies used in the Liver Guardian platform are modern, stable, and widely adopted within enterprise software development. The use of .NET 8, React.js, SQL Server, SignalR, Hangfire, and AI services ensures strong technical feasibility and long-term maintainability.

Economic Feasibility

The project was developed using mostly free and open-source technologies, minimizing operational and development costs. Existing infrastructure and cloud hosting solutions provided affordable deployment capabilities suitable for academic and prototype environments.

Operational Feasibility

The system provides intuitive user workflows and responsive interfaces that support ease of use for patients, doctors, and administrators. The platform's medical consultation and AI-assisted diagnosis workflows align with real-world healthcare operations.

Schedule Feasibility

The Agile development methodology enabled the team to divide the project into manageable sprints and deliver functionality incrementally. Continuous collaboration and iterative development ensured that project milestones were completed within the planned schedule.

3.9 Stakeholder Analysis

Patients

Patients are the primary end users of the platform. They utilize the system to upload medical records, receive AI analysis results, schedule appointments, and communicate with doctors.

Doctors

Doctors provide medical consultation services and review AI-generated predictions. Verified doctors can access patient medical data, manage appointments, and conduct consultation sessions.

Administrators

Administrators manage system operations, verify doctor credentials, monitor platform activities, and maintain overall system integrity and security.

Development Team

The development team is responsible for implementing, maintaining, testing, and improving the platform infrastructure, AI integrations, and software architecture.

Healthcare Organizations

Healthcare institutions may benefit from integrating AI-assisted diagnosis and remote consultation workflows into their operational environments.

Academic Supervisors

Supervisors oversee project progress, evaluate system quality, and provide technical and academic guidance throughout the project lifecycle.

Chapter 4

4.1Design

This chapter introduces the design of the Liver Guardian frontend application. The approaches and technologies used to construct this framework are described below.

4.1.1 Web

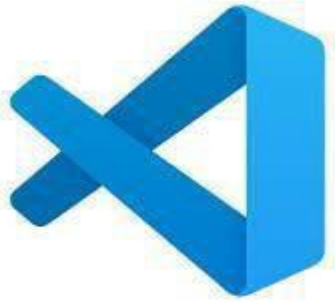
4.1.1.1 Front-end Design

The user interface is designed to work at the presentation layer. It is the user's primary means of interacting with the Liver Guardian platform. The most crucial aspect of healthcare system development is the external interface, as it directly impacts patient care and clinical workflows.

The client zone refers to the external interface where patients, doctors, and administrators access the system. The goal of frontend development is for users to read and process medical information quickly and accurately when they visit the platform. Web front development, also known as client-level development, is the process of creating web pages using technologies like:

- **Visual Studio Code**

Visual Studio Code is a dual-licensed source-code editor developed by Microsoft for Windows, Linux, and macOS. In the Stack Overflow 2019 Developer Survey, Visual Studio Code was ranked the most popular developer environment tool, with 50.7% of 87,317 respondents reporting that they use it. Visual Studio Code is a source-code editor that can be used with a variety of programming languages, including TypeScript, JavaScript, Go, Node.js, Python, and C++. It is based on the Electron framework, which is used to develop Node.js web applications that run on the Blink layout engine. Visual Studio Code employs the same editor component (codenamed "Monaco") used in Azure DevOps.



- **Next.js**

TypeScript version 5.x is a statically typed superset of JavaScript used throughout the Liver Guardian frontend. In a healthcare application, data integrity is paramount. TypeScript enforces compile-time type checking across all API types, form payloads, state shapes, and component props, preventing entire categories of runtime bugs. The application defines 30+ domain interfaces for request payloads and response shapes, 7 type aliases for enums (Gender, DoctorVerificationStatus, AlcoholConsumption, Smoking Status, PhysicalActivityLevel, MedicalRecordType, HealthTier, AppointmentStatus, UserRole), and 4 base response wrappers (ApiResponse<T>, ApiErrorDto, PaginatedResponse<T>, ApiWrapper<T>). The ApiError class provides structured error information including HTTP status, application-specific error code, and per-field validation errors.



- *TypeScript*

TypeScript version 5.x is a statically typed superset of JavaScript used throughout the Liver Guardian frontend. In a healthcare application, data integrity is paramount. TypeScript enforces compile-time type checking across all API types, form payloads, state

shapes, and component props, preventing entire categories of runtime bugs. The application defines 30+ domain interfaces for request payloads and response shapes, 7 type aliases for enums (Gender, DoctorVerificationStatus, AlcoholConsumption, Smoking Status, PhysicalActivityLevel, MedicalRecordType, HealthTier, AppointmentStatus, UserRole), and 4 base response wrappers (ApiResponse<T>, ApiErrorDto, PaginatedResponse<T>, ApiWrapper<T>). The ApiError class provides structured error information including HTTP status, application-specific error code, and per-field validation errors.



• Tailwind

- **Tailwind CSS** is an open-source CSS framework. Unlike other frameworks, like Bootstrap, it does not provide a series of predefined classes for elements such as buttons or tables. Instead, it creates a list of "utility" CSS classes that can be used to style each element by mixing and matching.
- For example, in other traditional systems, there would be a class messagewarning that would apply a yellow background color and bold text. To achieve this result in Tailwind, one would have to apply a set of classes created by the library: bgyellow-300 and font-bold.



• React

React is a JavaScript-based UI development library that is used to build large web applications which can update and render efficiently in response to data changes. It was developed by Facebook and is now maintained by Facebook and a community of individual developers and companies. React allows you to design simple views for each state in your application, and it will efficiently update and render the right components when your data changes. It introduces the concept of a virtual DOM that acts as a layer between the developer's description of how things should look and the work of updating the actual DOM to match. React has several key features:

- JSX: JSX is a syntax extension for JavaScript, which makes it possible to write HTML structures in the same file that contains JavaScript code.
- Components: React code is made of entities called components that you can reuse and combine to make complex UIs. Components are JavaScript classes or functions that optionally accept inputs, i.e., properties(props), and return a React element that describes how a section of the UI should appear.
- Unidirectional data flow and Flux: React implements one-way data flow which makes it easy to reason about your app. Flux is a pattern that helps to keep your data unidirectional. With Flux, actions are dispatched to stores, which then update the state and notify components that they need to re-render.
- Virtual DOM: React creates an in-memory data structure cache, computes the resulting differences, and then updates the browser's displayed DOM efficiently.

These features make React a powerful library for building web applications that can handle changing data and maintain high performance.



- **Lucide React**

Lucide React is an open-source icon library providing consistent SVG icons as React components. Lucide is a fork of the Feather icons project with over 1,000 uniformly designed icons (24x24 grid, 2px stroke). In Liver Guardian, Lucide React icons are used for navigation (Home, User, Bell), actions (Edit, Trash, Plus), status indicators (CheckCircle, XCircle, Clock), and medical icons (Activity, Heart, Brain, Scan). The library implements tree-shaken imports, ensuring only imported icons are bundled. Icons are imported as named imports (import { User, Settings } from 'lucide-react') and used as JSX components with configurable props, providing full TypeScript type safety.

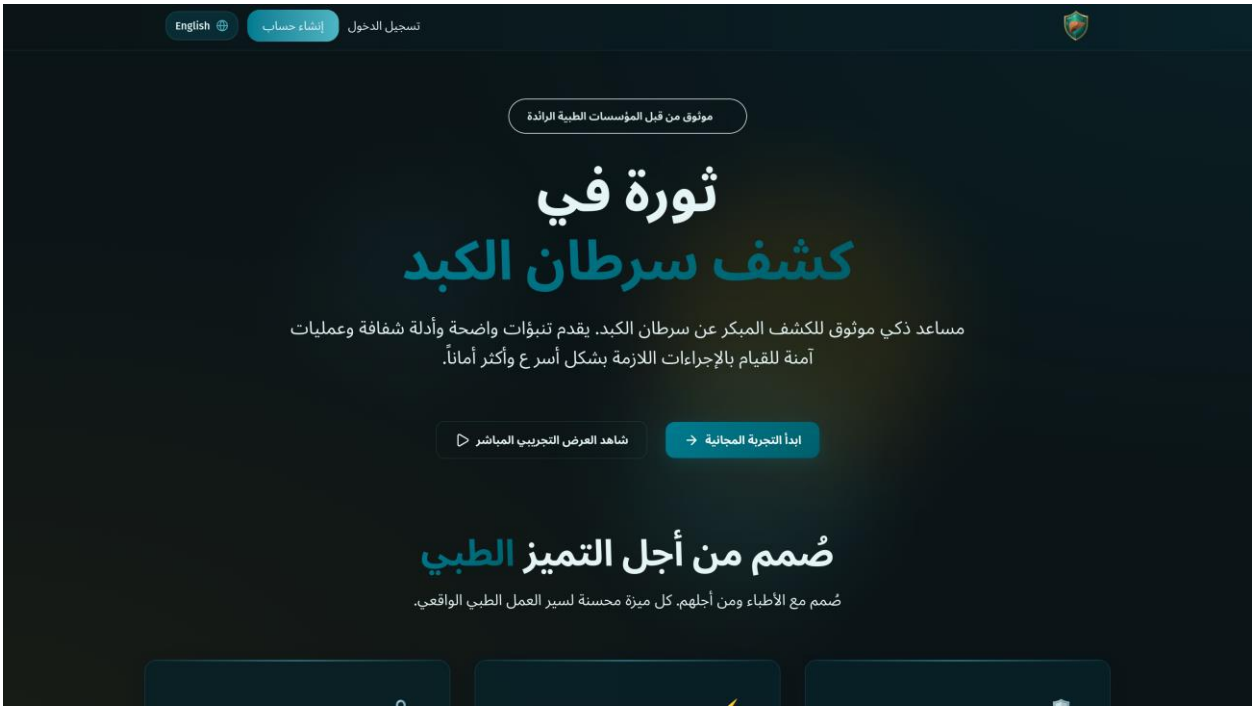


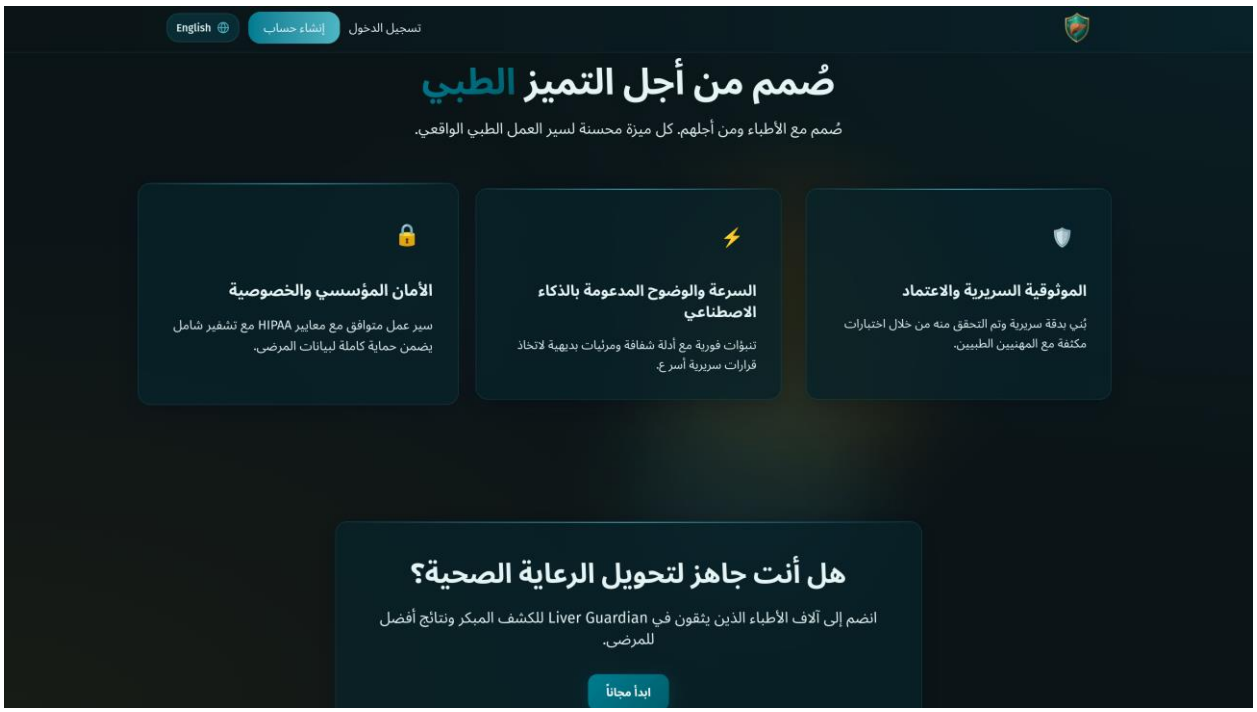
Our Work

Developers build components and functions that users access directly via a program or system interface. It is a way for users and servers to communicate. Customer development is what interface design is all about. The construction of an external interface is a crucial part of many aspects of software development. We designed a comprehensive external interface that allows patients, doctors, and administrators to interact with the AI-powered liver cancer early detection system easily. The frontend follows a three-layer client-side architecture with clear separation of concerns: presentation layer (pages and components), state layer (Zustand stores), and data access layer (Axios API client with interceptors, endpoint modules, and SignalR hub). The end-to-end data flow for every user action follows a consistent pattern from component call to store action to API request to response and UI re-render.

1. Landing Page

The landing page is the entry point of the MedFlow platform. It introduces the product to prospective patients and doctors, providing clear calls-to-action to sign up or sign in.


English
إنشاء حساب
تسجيل الدخول
موثوق من قبل المؤسسات الطبية الرائدة
ثورة في
كشف سرطان الكبد
مساعد ذكي موثوق للكشف المبكر عن سرطان الكبد. يقدم تنبؤات واضحة وأدلة شفافة وعمليات آمنة للقيام بالإجراءات اللازمة بشكل أسرع وأكثر أماناً.
ابدأ التجربة المجانية
شاهد العرض التجريبي المباشر
صُمم من أجل التميز الطبي
صُمم مع الأطباء ومن أجلهم. كل ميزة محسنة لسير العمل الطبي الواقعي.


English
إنشاء حساب
تسجيل الدخول
صُمم من أجل التميز الطبي
صُمم مع الأطباء ومن أجلهم. كل ميزة محسنة لسير العمل الطبي الواقعي.
الموثوقية السريرية والاعتماد
بُني بدقة سريرية وتم التحقق منه من خلال اختبارات مكثفة مع المهنيين الطبيين.
السرعة والوضوح المدعومة بالذكاء الاصطناعي
تنبؤات فورية مع أدلة شفافة ومرئيات بديهية لاتخاذ قرارات سريرية أسرع.
الأمان المؤسسي والخصوصية
سير عمل متوافق مع معايير HIPAA مع تشفير شامل يضمن حماية كاملة لبيانات المرضى.
هل أنت جاهز لتحويل الرعاية الصحية؟
انضم إلى آلاف الأطباء الذين يثقون في Liver Guardian للكشف المبكر ونتائج أفضل للمرضى.
ابدأ مجاناً

2. Patient User Flow

The patient journey covers the complete lifecycle from registration through to real-time consultation with a doctor. The diagram below summarises the end-to-end flow before each step is detailed with the corresponding screen.

1	Sign Up — Create a new patient account
2	Verify OTP — Confirm mobile/email via one-time password
3	Sign In — Authenticate with credentials
4	Complete Survey — Answer health-related onboarding questions
5	Medical History — Provide past diagnoses, medications, and allergies
6	Patient Dashboard — Central hub for all patient activities
7	Add Medical Records — Upload and manage health documents
8	Run AI Scan — Analyse medical records using AI diagnostics
9	Book an Appointment — Schedule a session with an available doctor
10	Chat with Doctor — Real-time consultation when session begins

2.1 Sign Up

New patients register by providing their personal information. The sign-up form captures name, email, mobile number, and password. Input validation ensures data quality before the account is created.

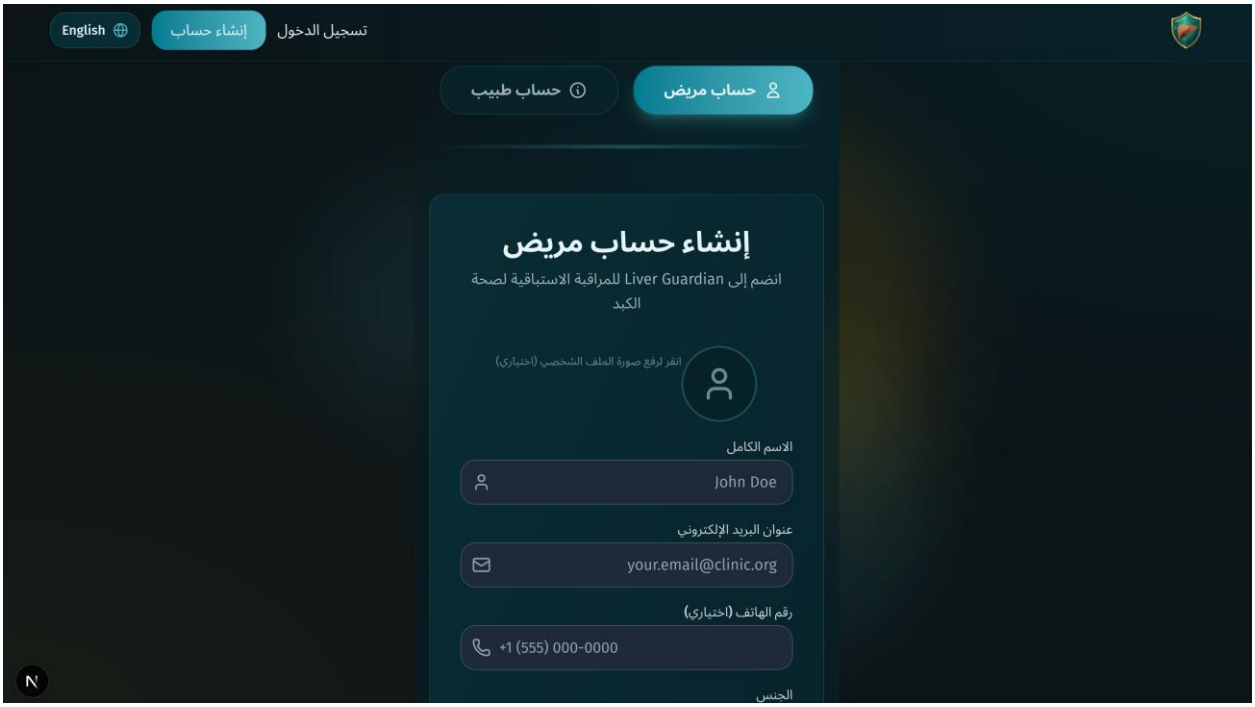


Figure 2 – Patient Sign Up (Step 1)

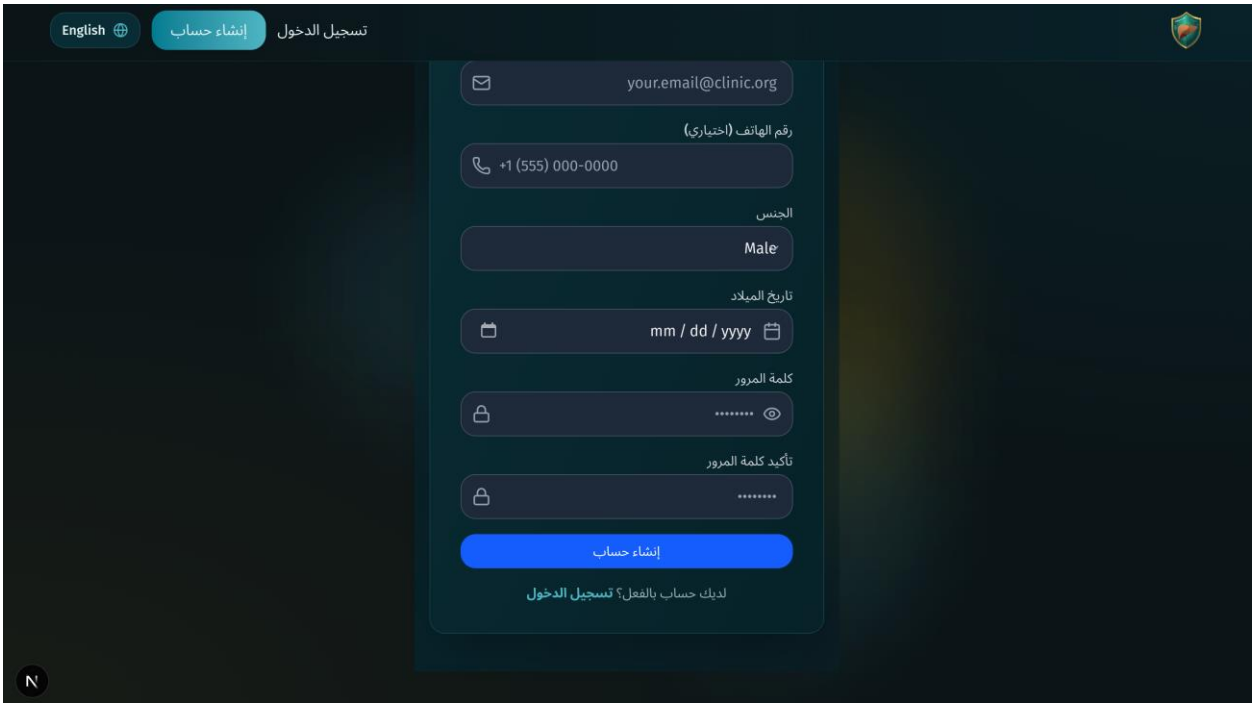


Figure 3 – Patient Sign Up (Step 2)

2.2 OTP Verification

After registration, a one-time password is sent to the patient's registered email or phone number. The patient must enter the OTP within the validity window to activate their account.

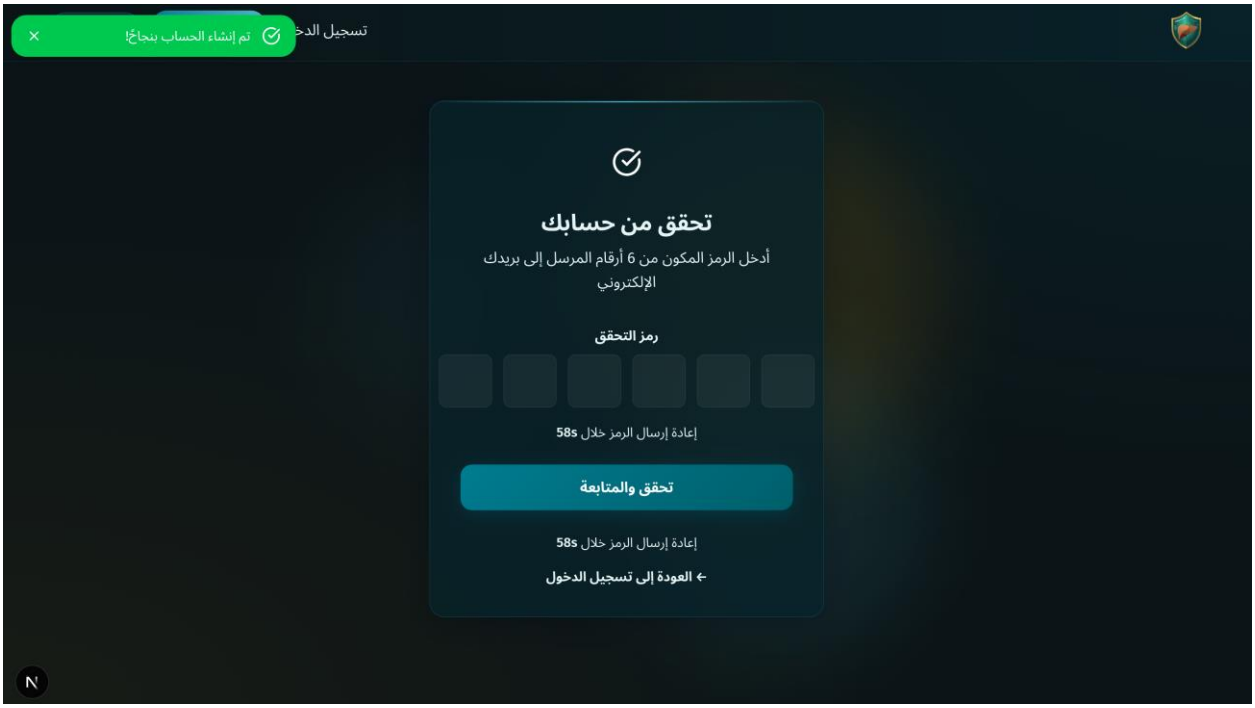


Figure 4 – OTP Verification Screen

2.3 Sign In

Returning patients sign in using their registered credentials. The login screen provides a secure authentication flow with options for password recovery.

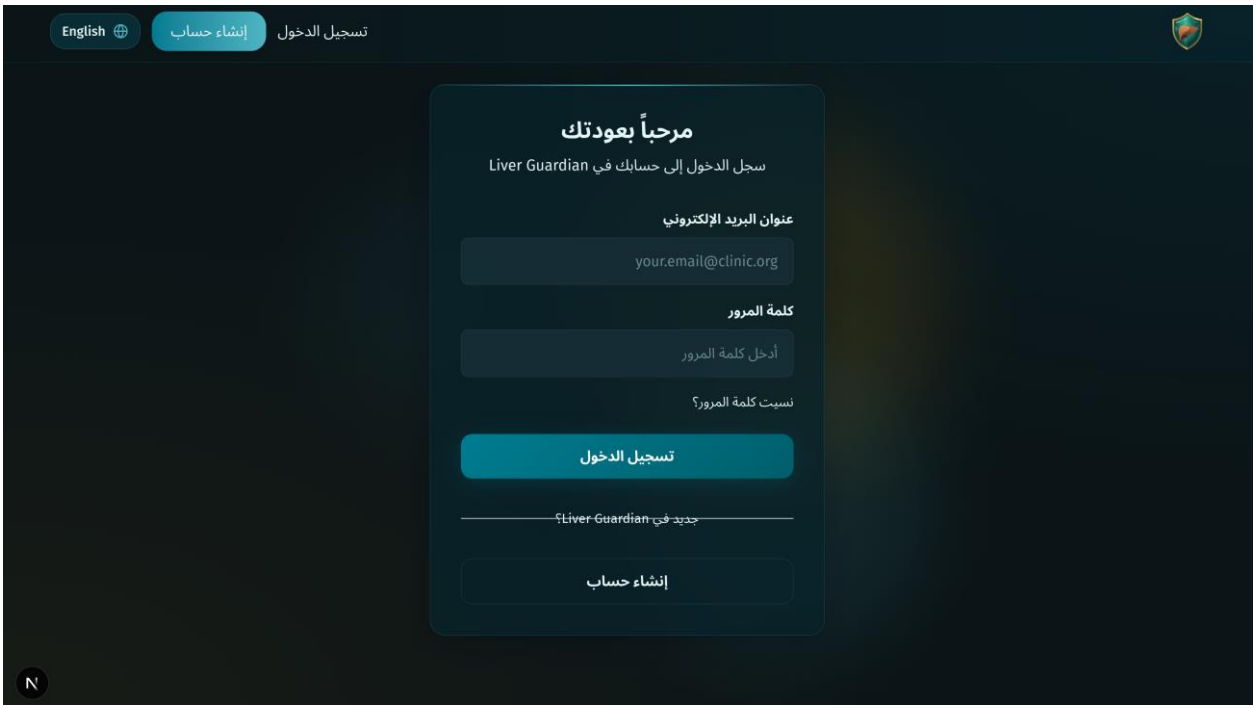


Figure 5 – Patient Sign In Screen

2.4 Patient Survey

Upon first login, patients are guided through a structured health survey. The survey collects lifestyle information, current symptoms, and general health goals to personalise the experience.

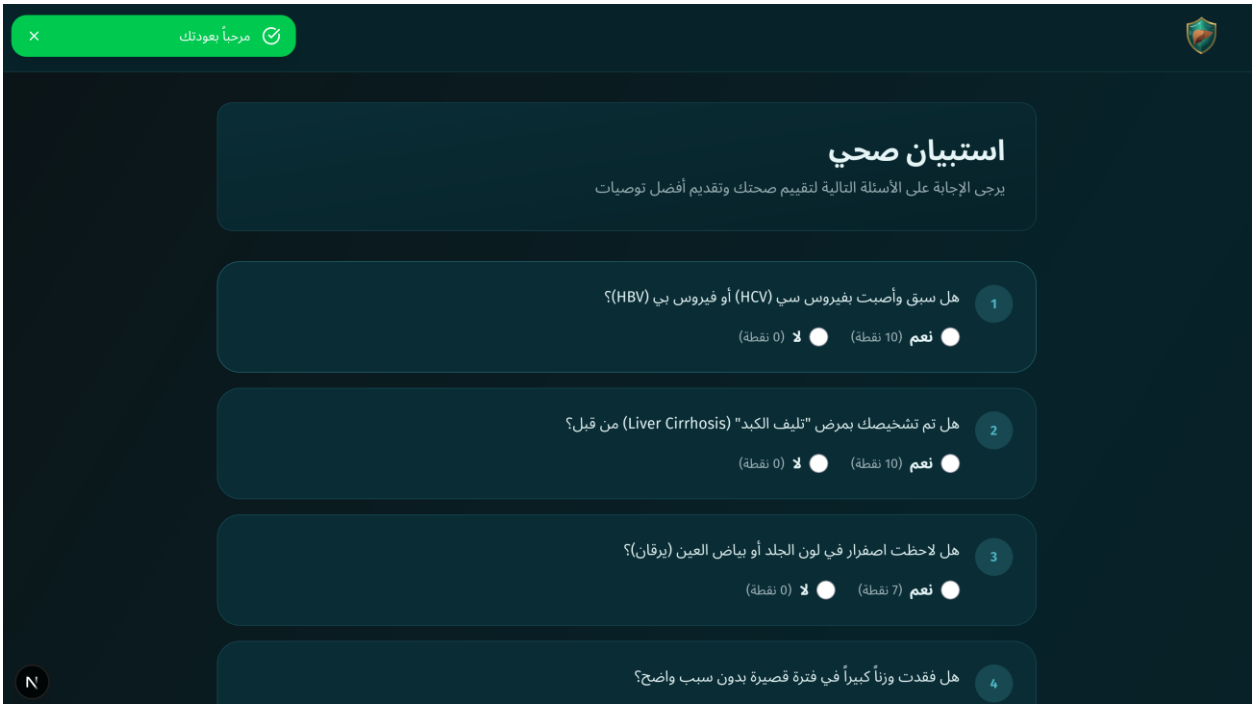
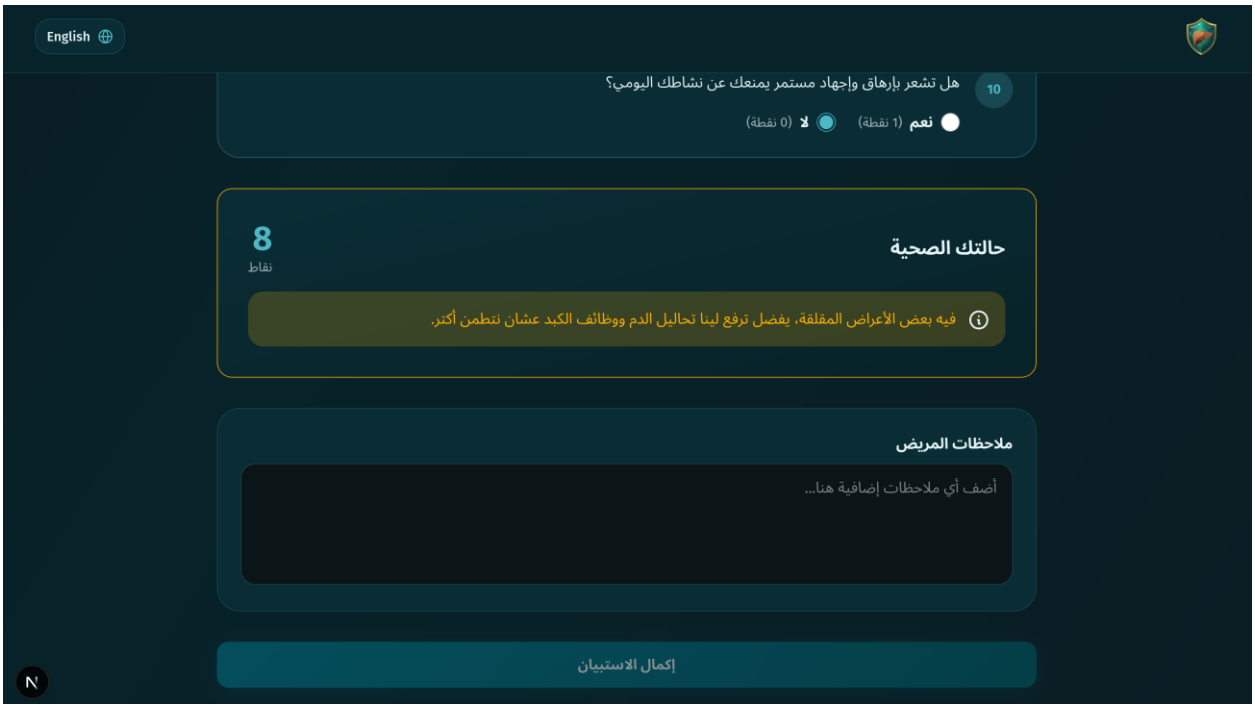


Figure 6 – Patient Survey (Page 1)

Figure 7 – Patient Survey (Page 2)

2.5 Medical History

Patients complete their medical history by documenting past conditions, ongoing medications, allergies, surgical history, and family medical background. This information is used to provide context for AI analysis and doctor consultations.



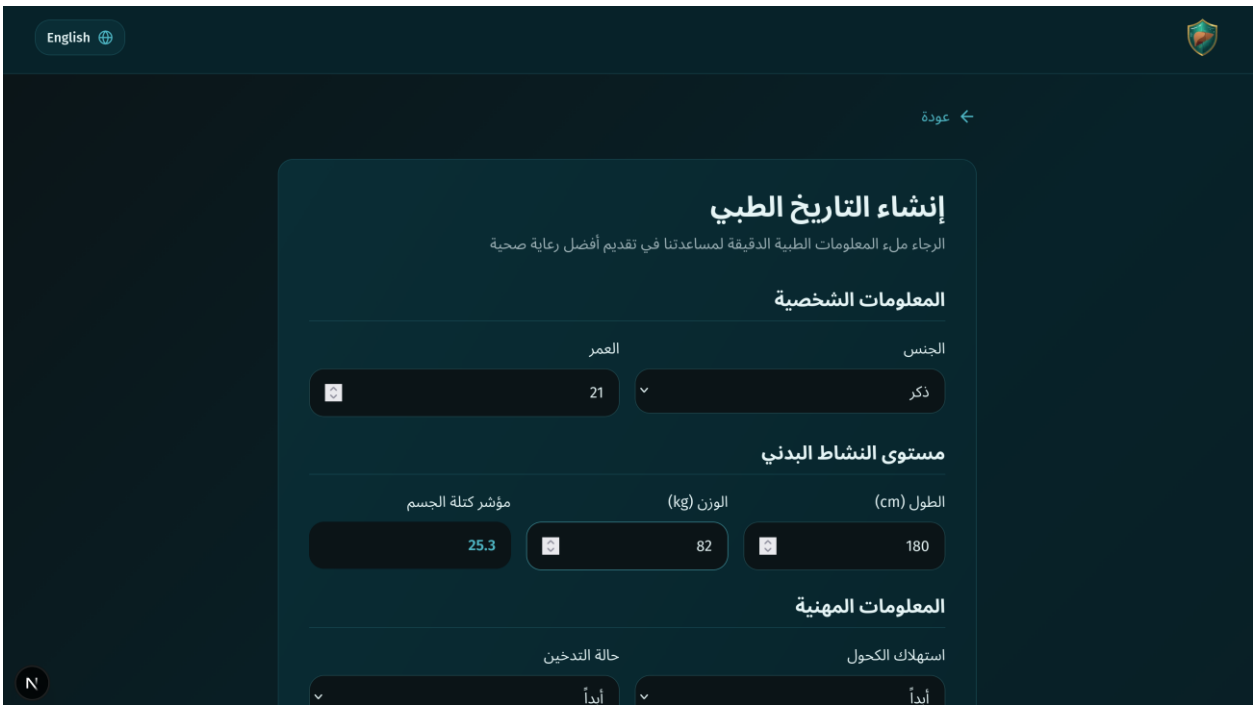
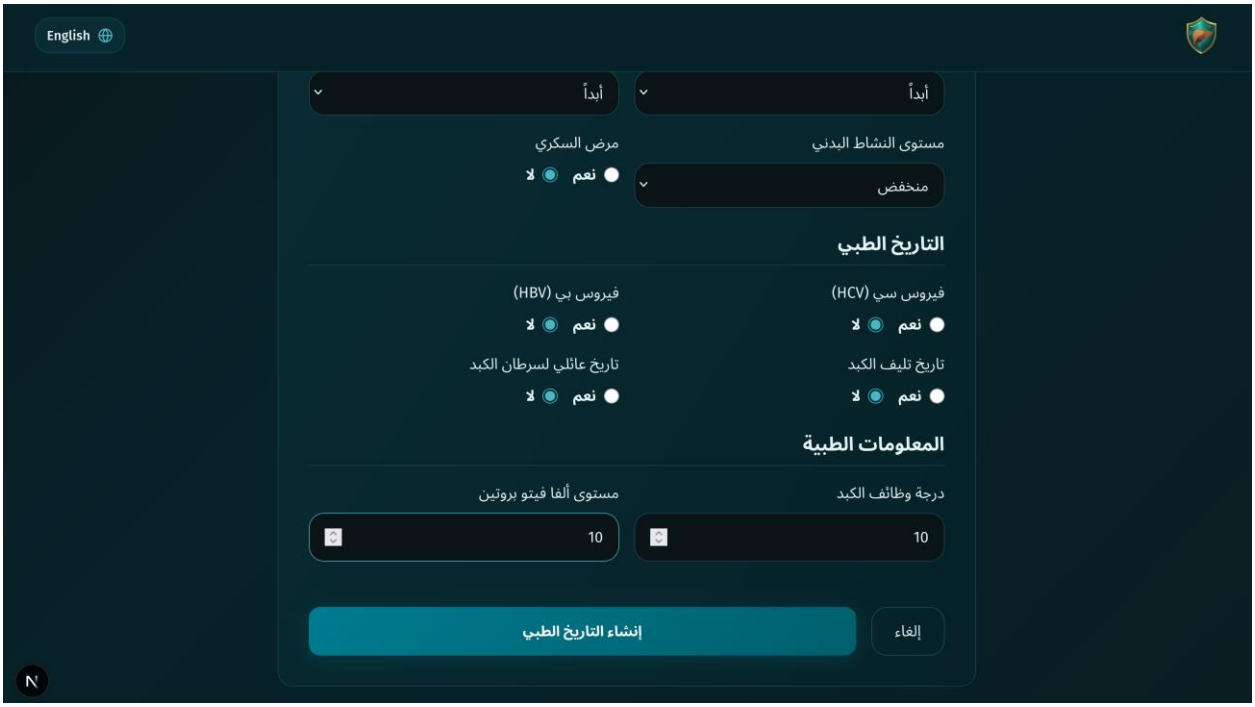


Figure 8 – Medical History (Page 1)

Figure 9 – Medical History (Page 2)



2.6 Patient Dashboard

The patient dashboard is the central hub of the platform. It provides an overview of upcoming appointments, recent AI scan results, quick access to medical records, and notifications from doctors.

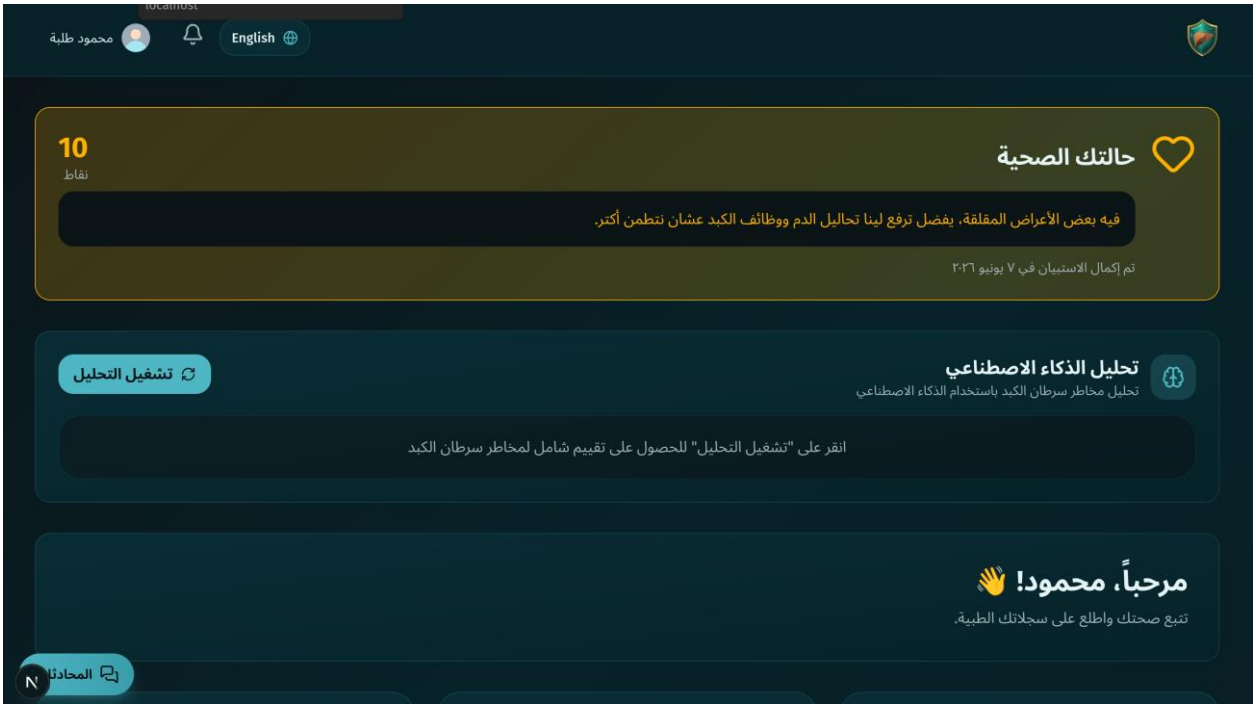
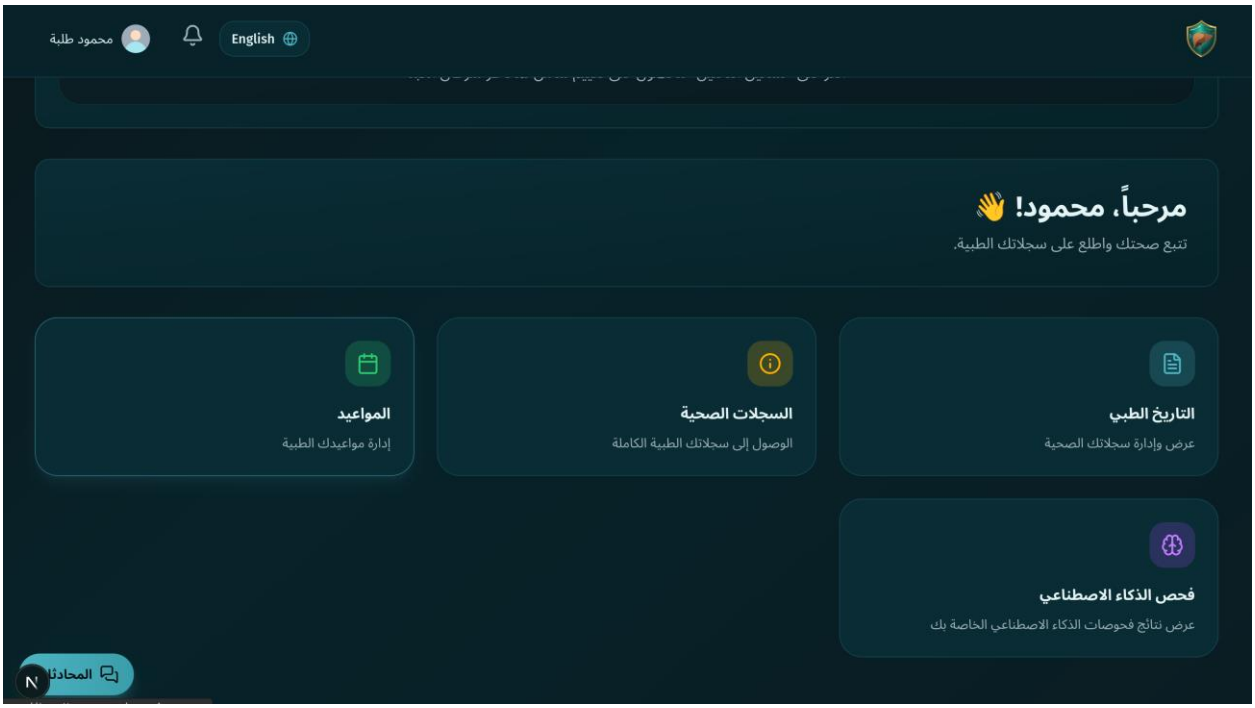


Figure 10 – Patient Dashboard (Overview)

Figure 11 – Patient Dashboard (Extended View)



2.7 Medical Records

Patients can upload, organise, and manage all their medical documents from this section. Supported file types include lab reports, imaging results, prescriptions, and discharge summaries. Documents are securely stored and available for AI analysis.

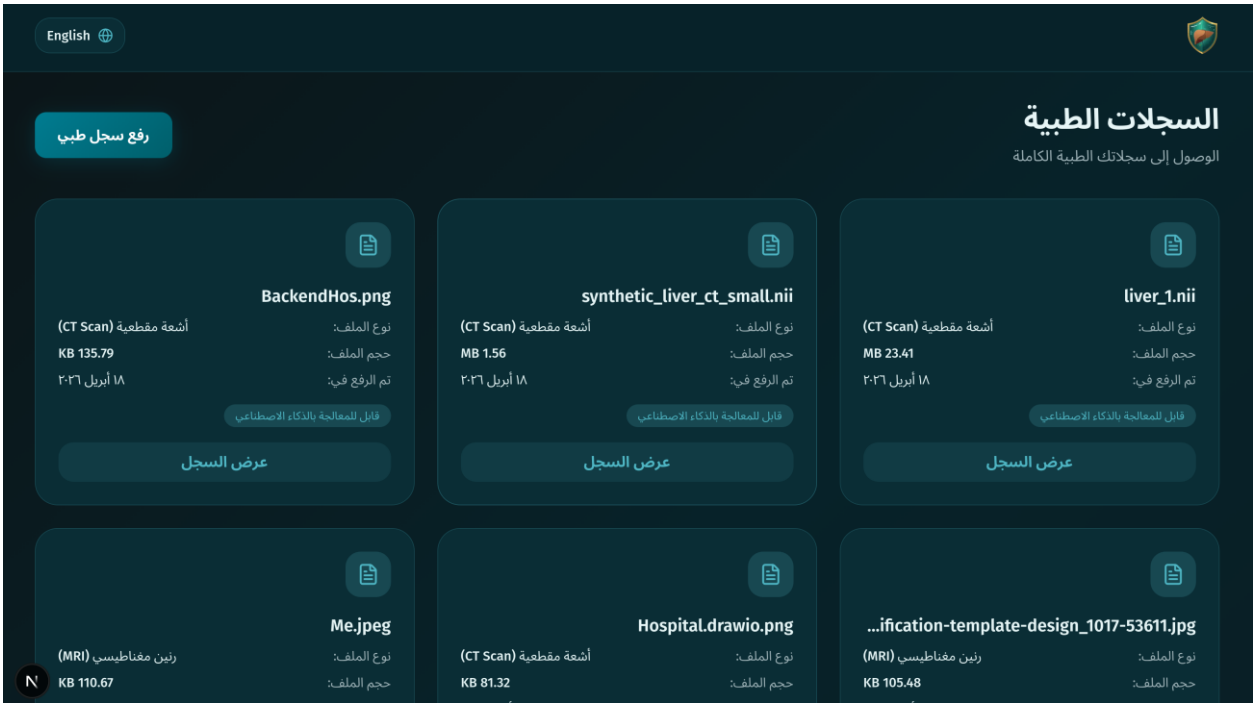


Figure 12 – Medical Records Screen

2.8 AI Scan

The AI Scan feature analyses uploaded medical records and generates intelligent diagnostic insights. Patients can initiate a new scan and review previously generated scan reports. The AI highlights anomalies, trends, and recommendations based on the records.

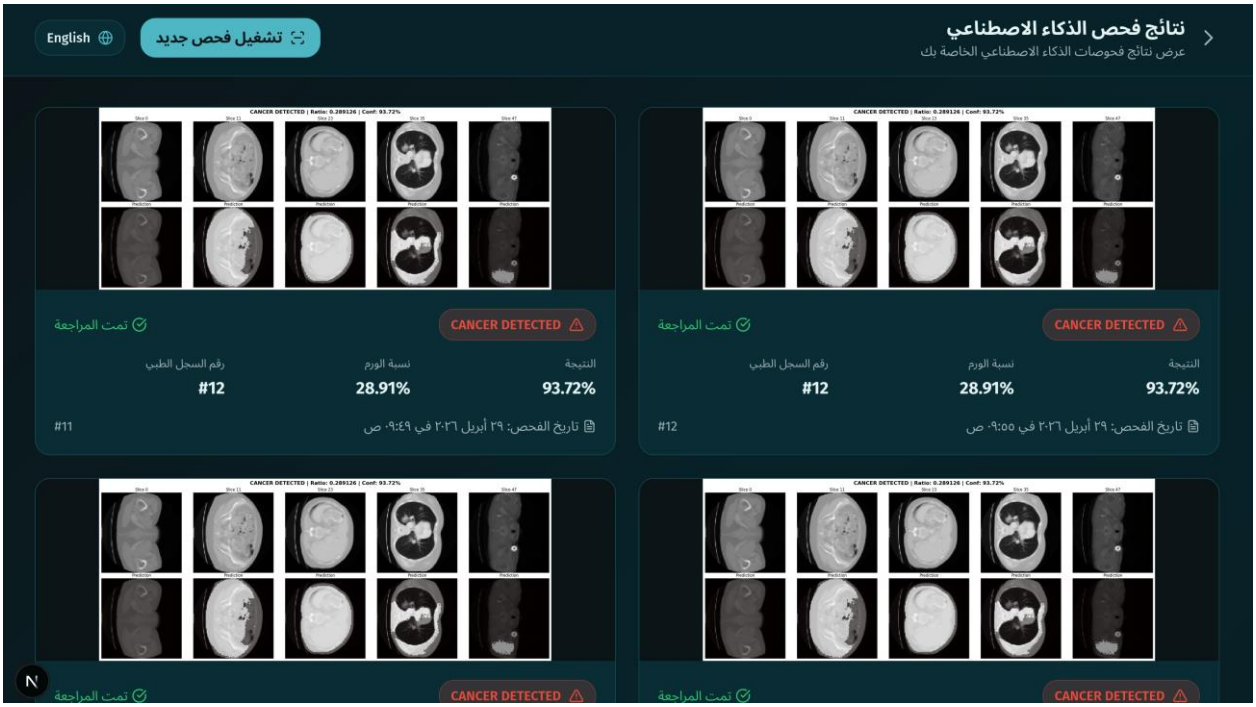


Figure 13 – My AI Scans (Scan History)

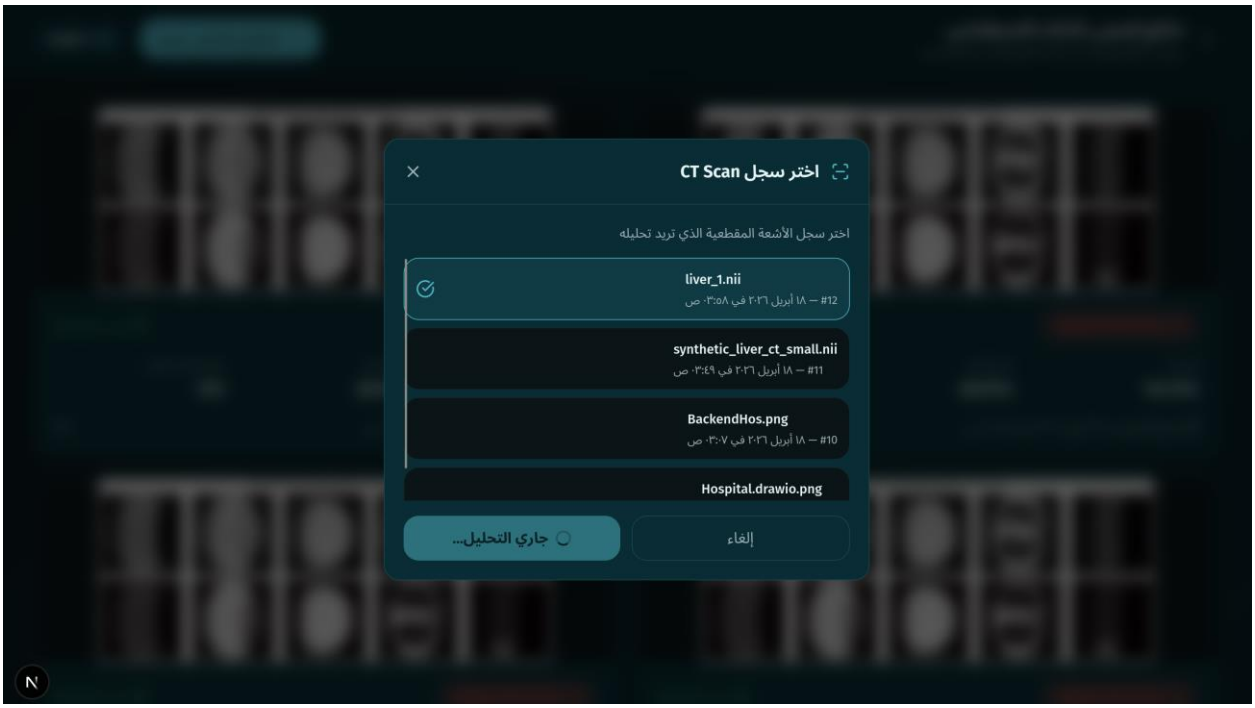


Figure 14 – Run New AI Scan

2.9 Book an Appointment

Patients can browse available doctors and request an appointment by selecting a preferred date and time. The request is sent to the chosen doctor for acceptance. Patients can monitor the status of all their appointment requests from the appointments screen.

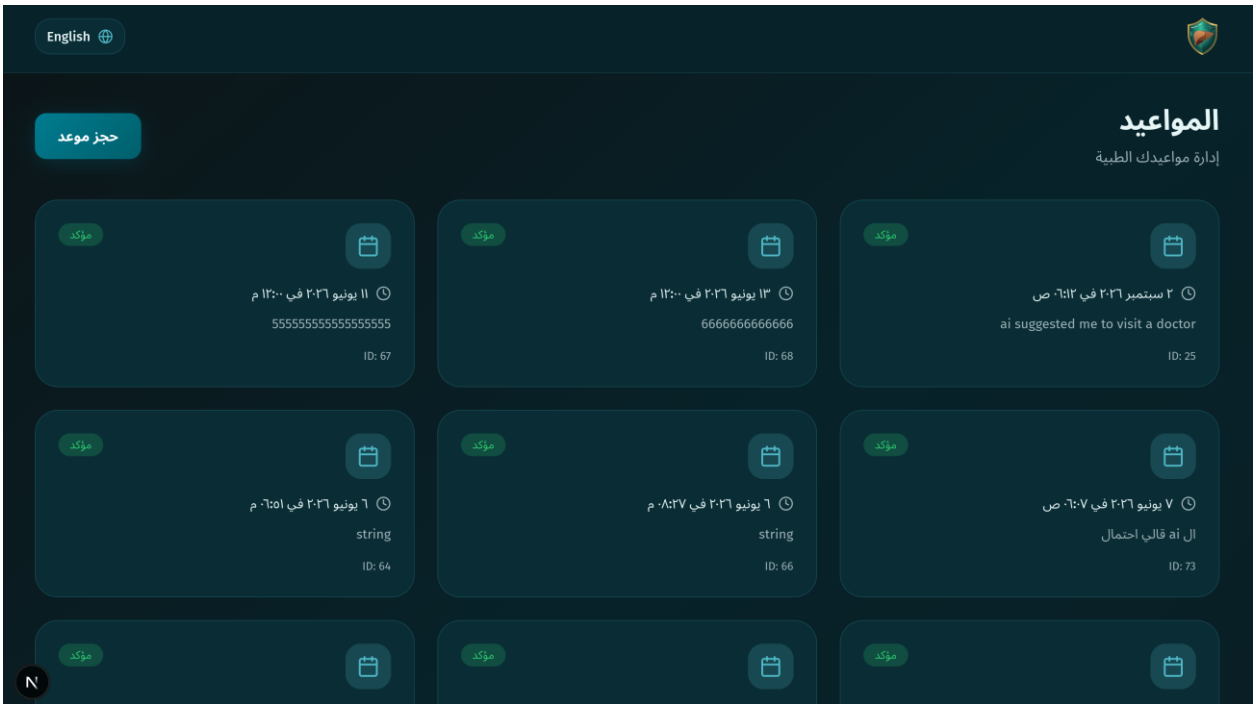


Figure 15 – My Appointments

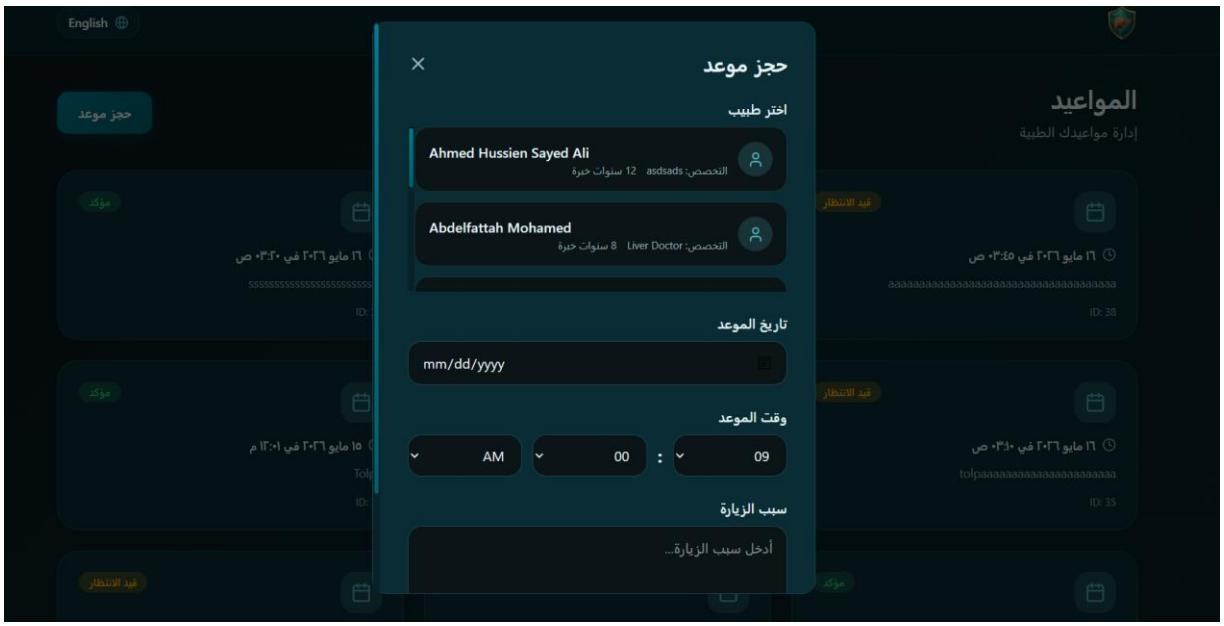


Figure 16 – Create New Appointment

2.10 Chat with Doctor

Once the doctor accepts the appointment and the session time arrives, the patient can enter the live chat room. The chat interface enables real-time text-based consultation between patient and doctor.

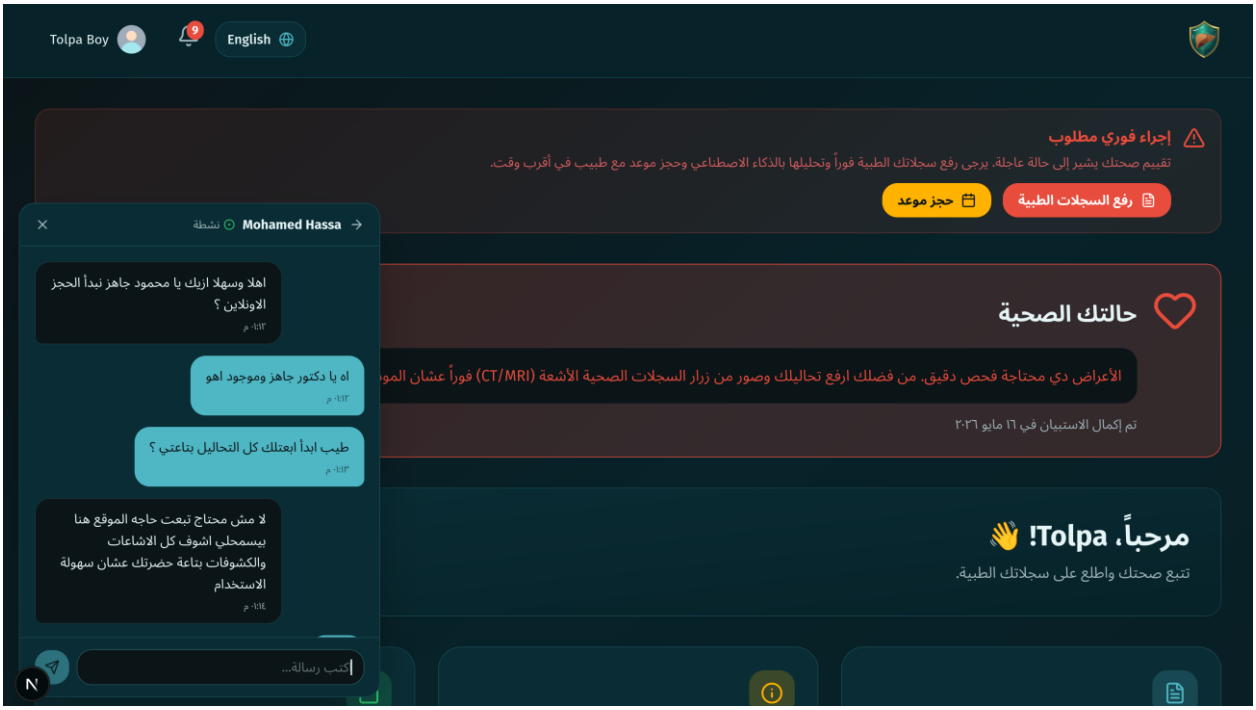


Figure 17 – Live Chat Session with Doctor

3. Doctor User Flow

The doctor journey begins with registration and identity verification by the platform administrator. Once approved, doctors can manage appointments, review AI-generated patient scans, and conduct live consultations.

1	Sign Up — Register as a licensed medical professional
2	Verify OTP — Confirm identity via one-time password
3	Await Admin Verification — Account reviewed and approved by the platform administrator
4	Manage Appointments — View, accept, or decline patient appointment requests
5	Review Patient AI Scans — Examine AI-generated diagnostic reports before the session
6	Start Chat Session — Initiate real-time consultation with the patient

3.1 Doctor Sign Up

Doctors register by providing professional credentials including their medical licence number, specialisation, and contact information. The multi-step form ensures all required details are captured for the admin verification process.

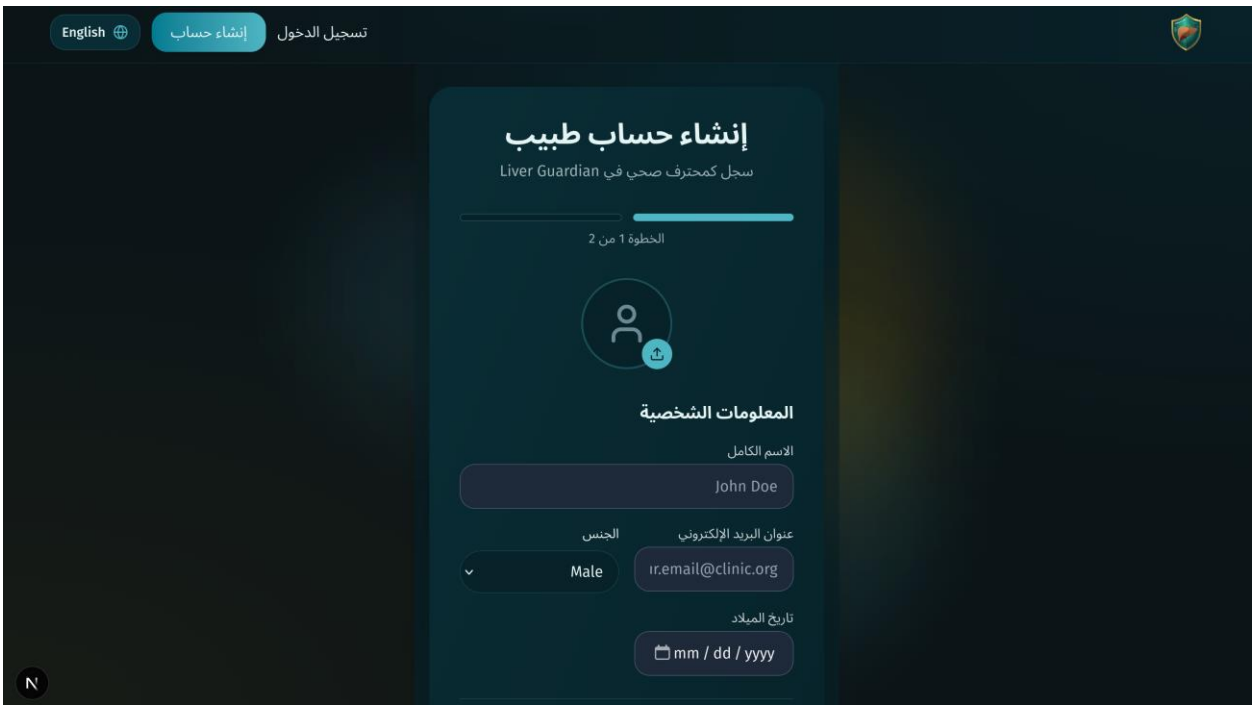


Figure 18 – Doctor Sign Up (Step 1)

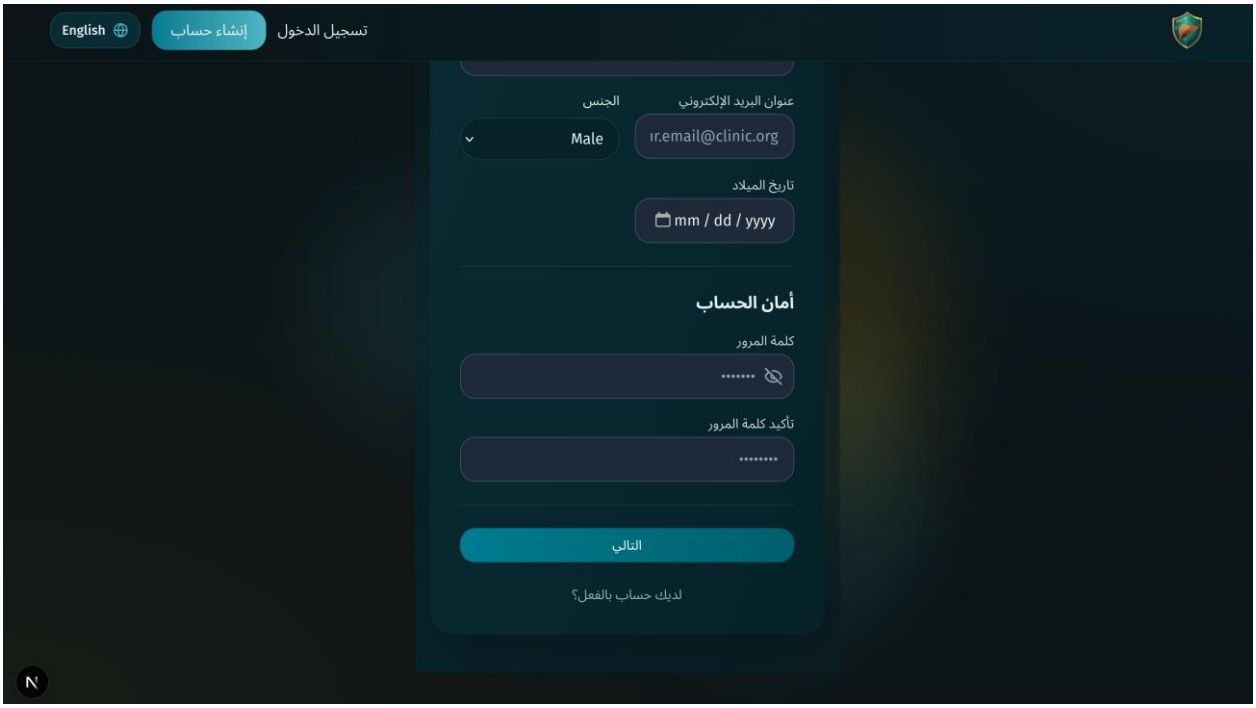


Figure 19 – Doctor Sign Up (Step 2)

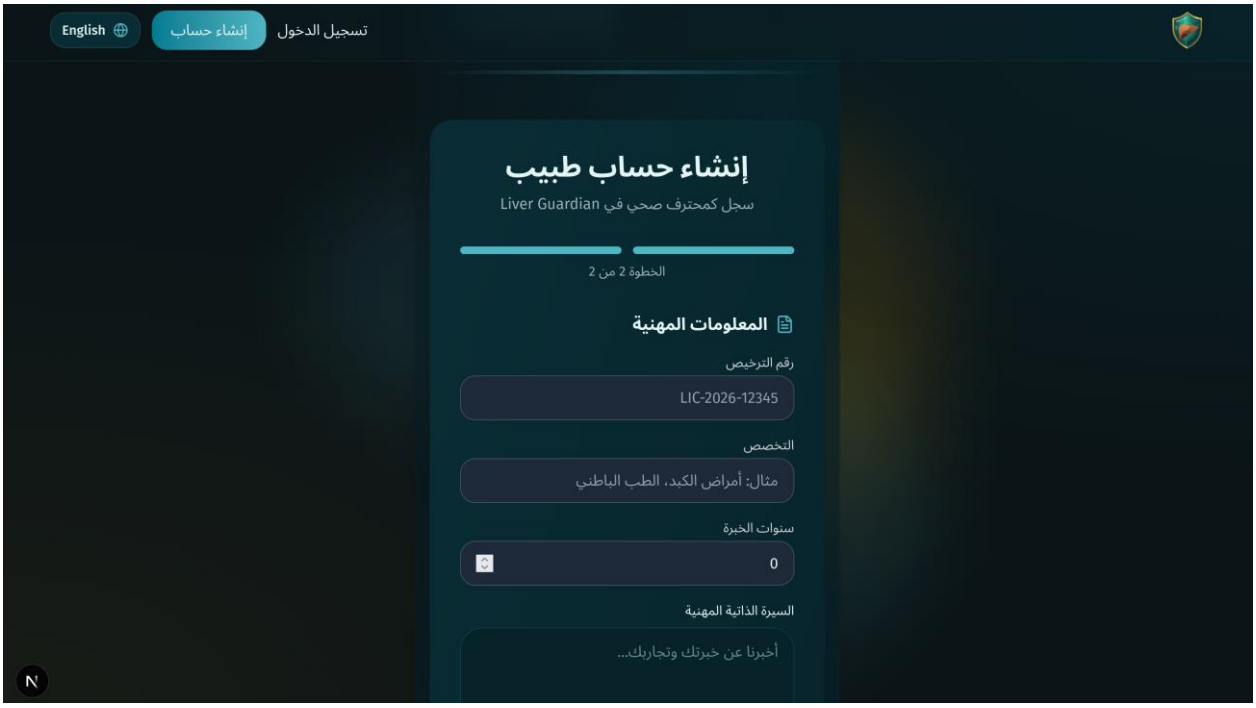


Figure 20 – Doctor Sign Up (Step 3)

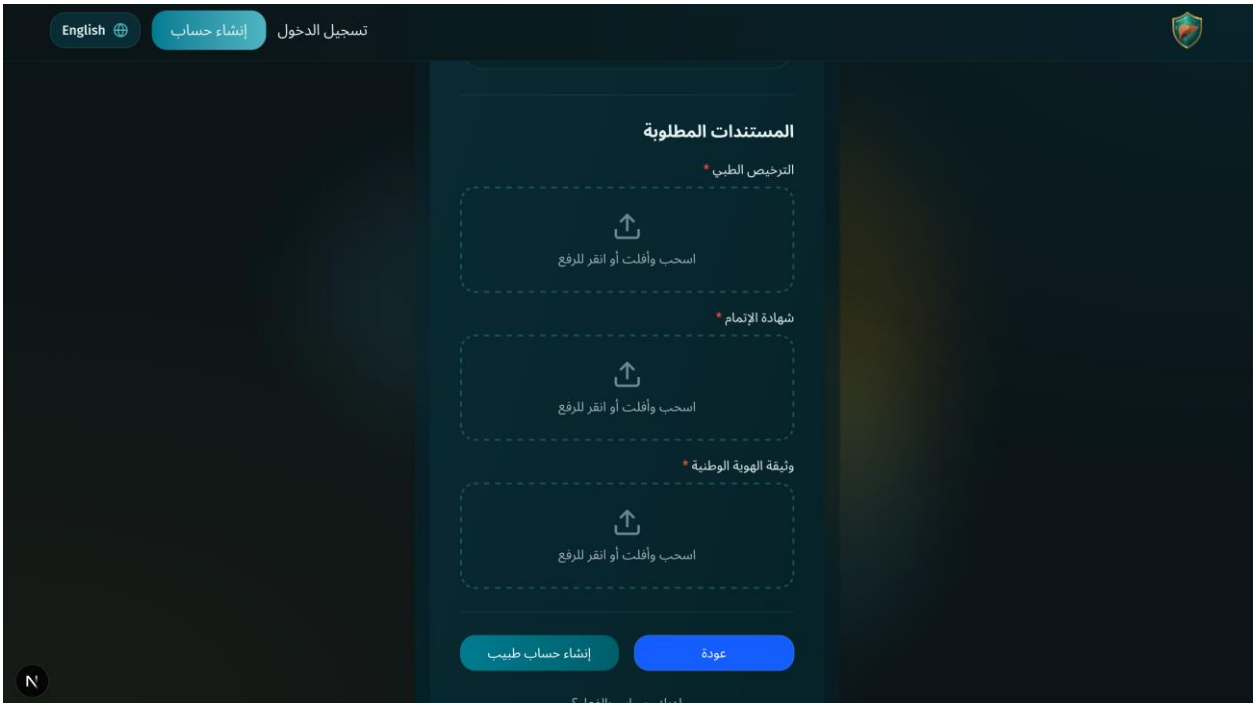


Figure 21 – Doctor Sign Up (Step 4)

3.2 OTP Verification & Admin Approval

After registration, the doctor verifies their contact details through an OTP. Following successful verification, the account enters a pending state until a platform administrator reviews and approves the doctor's credentials. The doctor is notified upon approval.

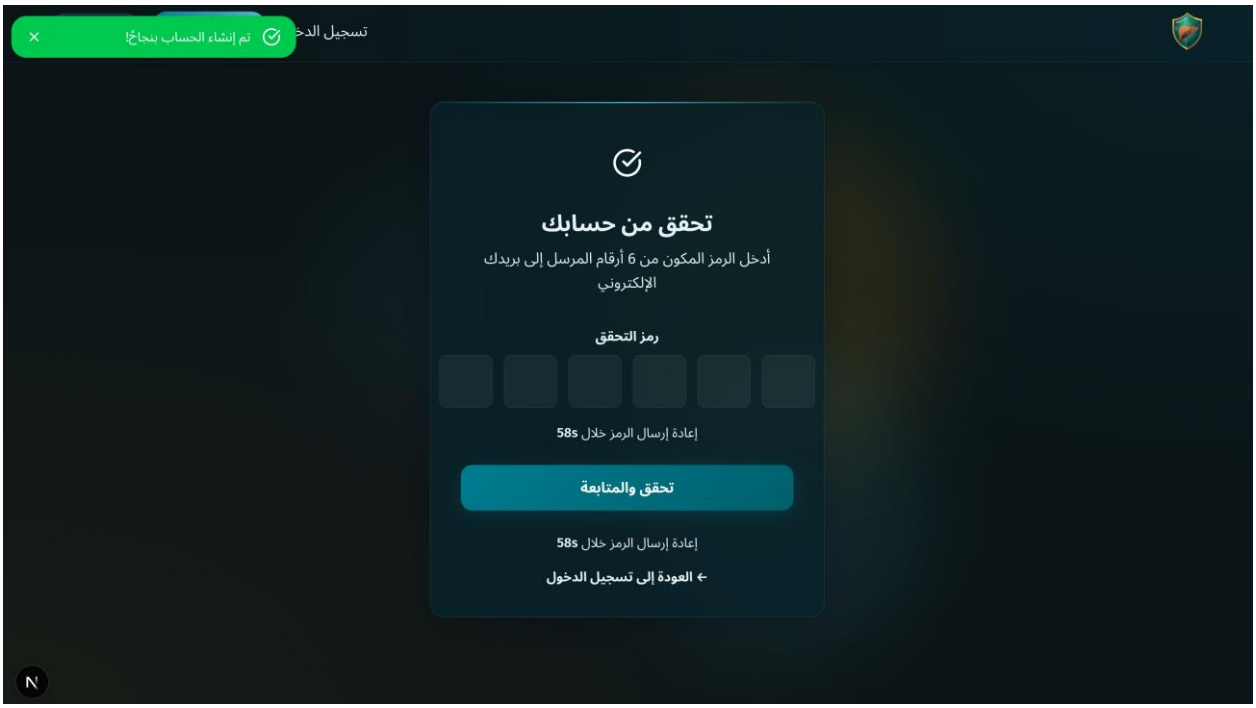


Figure 22 – OTP Verification (Doctor)

Admin verification steps:

1. Doctor submits registration and verifies OTP.
2. Platform admin receives a notification in the admin panel.
3. Admin reviews credentials and approves or rejects the application.
4. Doctor receives approval notification and gains access to the doctor dashboard.

3.3 Manage Appointments

Once approved, doctors can view all incoming appointment requests from patients. They can accept or decline requests based on their availability. Accepted appointments are added to their schedule, and the patient is notified of the confirmation.

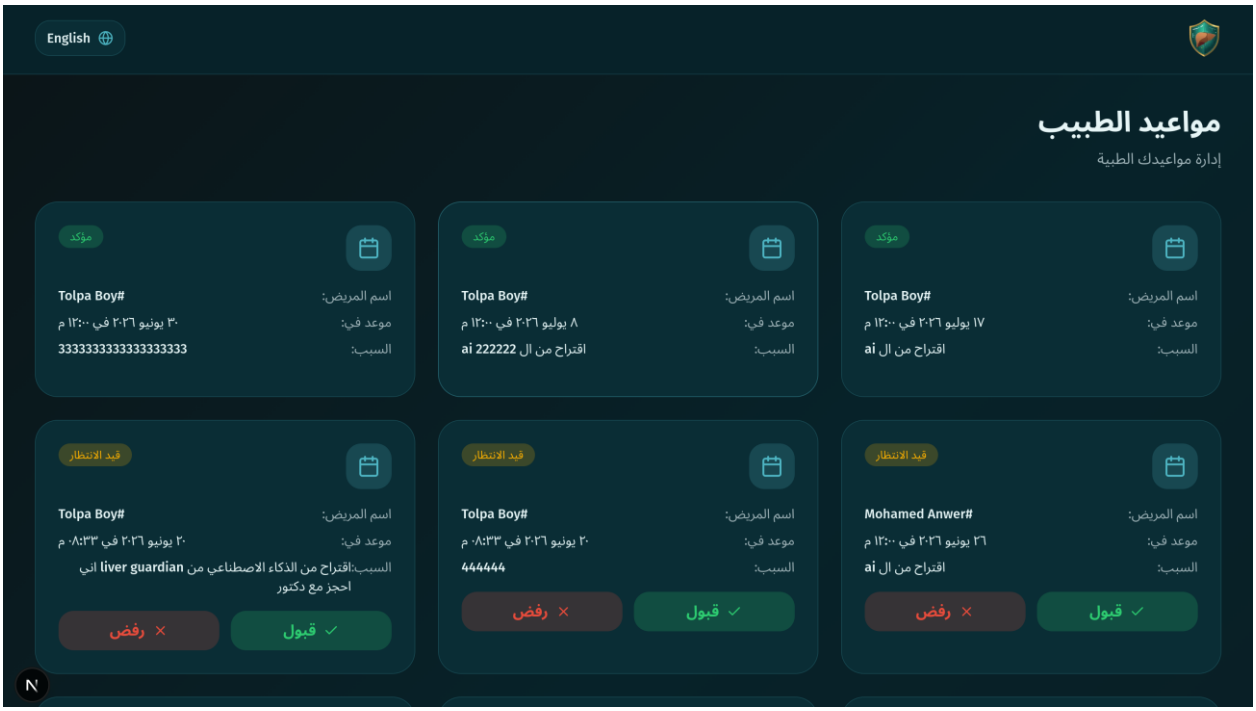


Figure 23 – Doctor Appointments (Request List)

3.4 Review Patient AI Scans

After accepting an appointment, doctors can access the AI-generated scan reports for the patient. This allows the doctor to review diagnostic insights and prepare for the consultation before the session begins.

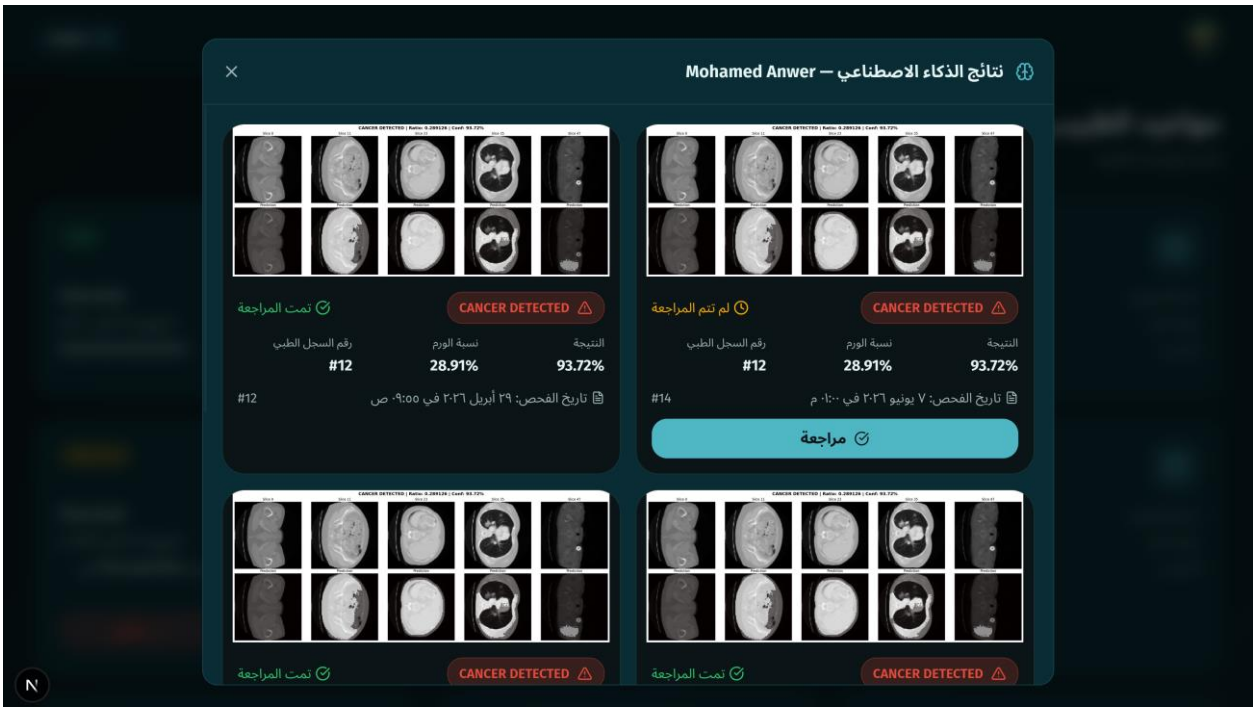


Figure 25 – Patient AI Scan Results (Doctor View)

3.5 Chat Session with Patient

At the scheduled time, the doctor can initiate the live chat session with the patient. The real-time communication interface allows the doctor to discuss findings, answer questions, provide prescriptions, and close the session upon completion.

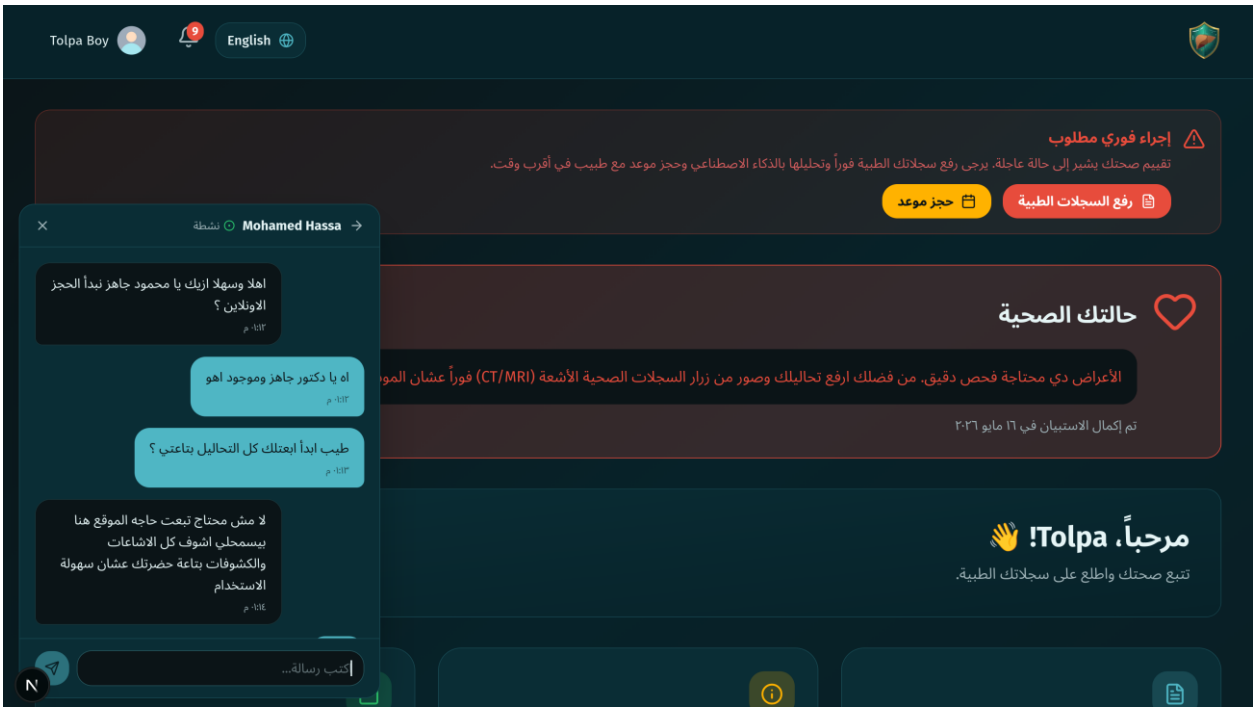


Figure 26 – Live Chat Session with Patient (Doctor View)

4.1.1.2 Back-End

The backend of the LiverGuardian System contains the core business logic and serves as the central component responsible for processing requests, managing data, handling authentication and authorization, communicating with AI services, and providing secure access to system resources.

The backend acts as a bridge between the frontend application, the database, and the AI diagnostic modules. All user interactions performed through the web application are processed by the backend before being stored, validated, or returned to the client.

Any failure within the backend services may directly affect the operation of the platform, making backend reliability and security critical requirements for the LiverGuardian System.

The backend is responsible for:

• User Authentication and Authorization.

• Patient and Doctor Management.

• Medical History Management.

• Medical Record Upload and Processing.

• AI Result Management and Review.

• Appointment Scheduling.

• Real-Time Notifications.

• Chat Session Management.

• Role and Permission Management.

• Secure Communication with Frontend Services.

Architecture

The LiverGuardian backend follows a layered architecture that separates responsibilities into different logical layers to improve maintainability, scalability, and code organization.

The architecture consists of:

• Presentation Layer (ASP.NET Core Controllers)

• Business Logic Layer (Services)

• Data Access Layer (Repositories)

• Database Layer (SQL Server)

• Real-Time Communication Layer (SignalR)

• Authentication & Security Layer (JWT Authentication & Authorization)

This architecture ensures clear separation of concerns and simplifies future maintenance and feature expansion.

ASP.NET Core Web API



ASP.NET Core Web API was used as the primary backend framework for the LiverGuardian System. It is a modern, high-performance, cross-platform framework developed by Microsoft for building secure and scalable web applications and RESTful APIs.

ASP.NET Core provides powerful features that make it suitable for healthcare applications, including dependency injection, middleware architecture, authentication mechanisms, routing, logging, and performance optimization.

The backend exposes a collection of RESTful API endpoints that allow communication between the frontend application and the server.

Key Features of ASP.NET Core

1. High Performance

ASP.NET Core is optimized for speed and scalability, allowing the system to handle multiple concurrent requests efficiently while maintaining stable performance.

2. Dependency Injection

The framework includes built-in dependency injection support, which improves code maintainability, modularity, and testability by reducing coupling between application components.

3. Middleware Pipeline

ASP.NET Core uses a middleware pipeline architecture where each incoming request passes through a sequence of processing components before reaching the application logic.

Examples include:

• Authentication Middleware

• Authorization Middleware

• Exception Handling Middleware

• Logging Middleware

• CORS Middleware

This approach improves security and allows centralized request processing.

4. RESTful API Development

The backend provides RESTful APIs that enable secure communication between the frontend and backend systems.

These APIs handle operations such as:

• User Registration and Login

• Patient Management

• Doctor Verification

• Medical History Management

• Medical Records Upload

• AI Result Retrieval

• Appointment Management

• Notifications

• Chat Sessions

SignalR Integration

SignalR was integrated into the backend to support real-time communication across the platform.

It enables instant delivery of notifications related to:

• Appointment Requests.

• Appointment Acceptance or Rejection.

• AI Analysis Completion.

• Doctor Reviews.

• System Notifications.

SignalR improves user experience by allowing updates to appear instantly without requiring page refreshes.

JWT Authentication & Authorization

The LiverGuardian System uses JSON Web Tokens (JWT) to secure access to protected resources.

The authentication process follows these steps:

1. User submits login credentials.
2. The backend validates the credentials.
3. A JWT Access Token and Refresh Token are generated.
4. The frontend stores the tokens securely.
5. All subsequent API requests include the access token.
6. The backend validates the token before granting access.

Role-based authorization is implemented to control access for:

• Patients

• Doctors

• Administrators

This ensures that each user can only access the functionalities assigned to their role.

SQL Server Database



SQL Server was used as the primary database management system for the LiverGuardian System.

The database stores:

• User Accounts.

• Patient Profiles.

• Doctor Profiles.

• Medical Histories.

• Medical Records.

• AI Analysis Results.

• Appointments.

• Notifications.

• Chat Sessions.

The database design was optimized to ensure data integrity, efficient querying, and secure storage of healthcare information.

Our Work

The LiverGuardian backend was developed using ASP.NET Core Web API following a layered architecture and RESTful design principles. The system provides secure and scalable services that support patient workflows, doctor review processes, AI result management, appointment scheduling, real-time notifications, and chat communication.

The backend currently exposes more than 70 API endpoints covering authentication, patient management, medical records, AI analysis, appointment workflows, notifications, role management, and administrative operations. These services form the foundation of the LiverGuardian platform and enable seamless integration between the frontend application, database, and AI diagnostic modules.

4.3 Artificial Intelligence

4.3.1 Artificial Intelligence in Healthcare

Artificial Intelligence (AI) has become one of the most influential technologies in modern healthcare. By analyzing large volumes of medical data, AI systems can assist healthcare professionals in disease detection, diagnosis, treatment planning, and patient monitoring. Recent advancements in machine learning and deep learning have significantly improved the ability of computer systems to extract meaningful patterns from clinical records and medical images, enabling faster and more accurate decision-making.

AI-driven healthcare solutions have demonstrated promising results in various medical fields, including radiology, oncology, cardiology, and pathology. These systems can process large datasets efficiently, identify hidden patterns that may be difficult for humans to detect, and support clinicians in making evidence-based decisions. As healthcare data continues to grow in volume and complexity, AI is expected to play an increasingly important role in improving patient outcomes and healthcare accessibility.

4.3.2 Machine Learning in Clinical Prediction

Machine Learning is a branch of artificial intelligence that enables systems to learn patterns from historical data and make predictions without being explicitly programmed. In healthcare applications, machine learning models are widely used to analyze patient demographics, laboratory tests, medical history, and lifestyle factors in order to estimate disease risk and support clinical decision-making.

Among the various machine learning techniques, gradient boosting algorithms have gained significant attention due to their high predictive performance and ability to handle heterogeneous clinical datasets. These models can identify complex relationships between patient features and disease outcomes, making them suitable for medical risk assessment tasks. By leveraging clinical information, machine learning systems can assist healthcare professionals in detecting potential health risks at an early stage and prioritizing patients who may require further medical evaluation.

4.3.3 Deep Learning for Medical Image Analysis

Deep Learning is a specialized subset of machine learning that utilizes artificial neural networks with multiple layers to automatically learn hierarchical representations from data. In medical imaging, deep learning has demonstrated remarkable success in tasks such as image classification, object detection, segmentation, and disease diagnosis.

Unlike traditional image-processing techniques, deep learning models can automatically learn relevant features directly from raw medical images, reducing the need for manual feature engineering and improving diagnostic accuracy. Recent advancements in computational power and large-scale medical datasets have accelerated the adoption of deep learning technologies in healthcare, enabling the development of intelligent systems capable of assisting radiologists and clinicians in image interpretation.

4.3.4 Computed Tomography (CT)

Computed Tomography (CT) is one of the most commonly used medical imaging modalities for evaluating internal organs and detecting abnormalities. CT scans provide cross-sectional images of the body, allowing physicians to examine anatomical structures with high spatial resolution.

In liver disease diagnosis, CT imaging plays an important role in identifying tumor location, size, shape, and surrounding tissue characteristics. Contrast-enhanced CT scans are particularly valuable for assessing liver lesions and detecting abnormalities that may not be visible through conventional imaging methods. Automated image analysis techniques can further assist clinicians by accurately delineating tumor boundaries and highlighting suspicious regions within the liver, improving diagnostic efficiency and consistency.

4.3.5 Magnetic Resonance Imaging (MRI)

Magnetic Resonance Imaging (MRI) is a non-invasive imaging technique that provides detailed visualization of soft tissues using magnetic fields and radiofrequency signals. Compared to CT imaging, MRI offers superior soft-tissue contrast and is particularly valuable for characterizing liver lesions and assessing tissue composition.

Different MRI sequences provide complementary diagnostic information, enabling physicians to distinguish between benign and malignant lesions and evaluate disease progression more accurately. Advanced MRI analysis techniques combined with artificial intelligence have shown significant potential in improving lesion detection, segmentation, and classification, making MRI an essential component of modern liver disease assessment.

4.3.6 Medical Image Segmentation

Medical image segmentation is the process of partitioning an image into meaningful regions to isolate anatomical structures or pathological findings. In liver imaging applications, segmentation is commonly used to identify liver boundaries and detect tumor regions.

Accurate segmentation is a critical step in computer-aided diagnosis systems because it allows subsequent analysis to focus on clinically relevant regions while reducing the influence of surrounding tissues. Deep learning-based segmentation models have achieved remarkable performance in medical imaging by learning complex spatial patterns and generating precise masks that support diagnosis, treatment planning, and disease monitoring.

4.3.7 Medical Image Classification

Medical image classification refers to the task of assigning predefined diagnostic labels to medical images. Classification models are commonly used to distinguish between healthy and diseased tissues or to identify specific disease categories.

In liver imaging, classification systems can support radiologists by providing automated assessments of detected lesions and assisting in differential diagnosis. By learning patterns from large collections of annotated medical images, deep learning models can achieve high classification accuracy and contribute to faster and more reliable diagnostic workflows.

4.3.8 Explainable Artificial Intelligence (XAI)

Although deep learning models often achieve high predictive performance, they are frequently criticized for operating as "black-box" systems. Explainable Artificial Intelligence (XAI) aims to improve transparency by providing insights into how models generate predictions.

In healthcare applications, explainability is particularly important because clinicians need to understand and trust AI-generated recommendations before incorporating them into medical decision-making processes. Visualization techniques and explainability methods help identify the image regions or clinical features that influence model predictions, thereby increasing confidence in the diagnostic process and supporting clinical validation.

4.3.9 Web-Based Healthcare Systems

Modern healthcare platforms increasingly rely on web technologies to improve accessibility and streamline communication between patients and healthcare providers. Web-based healthcare systems enable remote access to medical

services, appointment scheduling, data management, and diagnostic support tools.

These platforms facilitate information exchange, improve healthcare efficiency, and provide users with convenient access to medical resources regardless of geographical constraints. By integrating artificial intelligence models into a web-based environment, healthcare services become more accessible, scalable, and user-friendly while supporting both patients and healthcare professionals.

4.3.10 AI-Powered Chatbots in Healthcare

Artificial Intelligence-powered chatbots have become increasingly important in modern healthcare systems due to their ability to provide instant assistance and improve user engagement. These conversational systems utilize Natural Language Processing (NLP) and machine learning techniques to understand user queries and generate relevant responses in real time.

In healthcare applications, chatbots are commonly used to answer frequently asked questions, provide general health information, assist users in navigating digital healthcare platforms, and improve access to medical resources. By offering continuous support, chatbots help reduce the time required to obtain basic information and enhance the overall user experience.

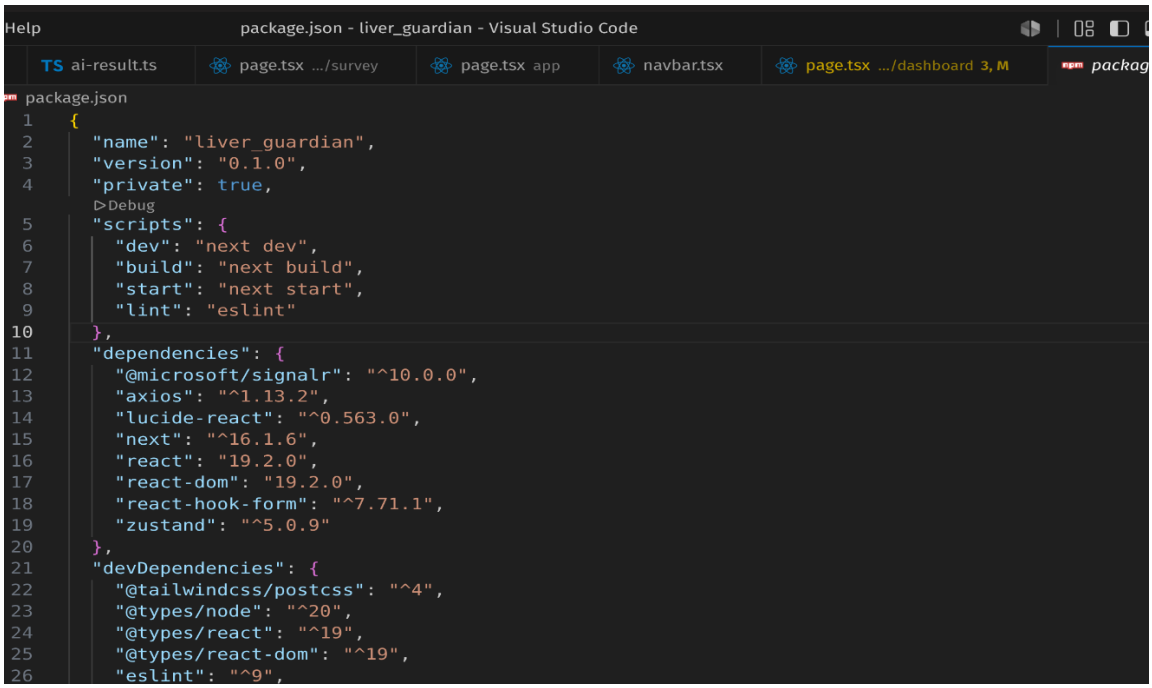
Within the LiverGuardian System, an intelligent chatbot is integrated to serve as a virtual assistant for users interacting with the platform. The chatbot helps users understand the system's features, guides them through different functionalities, and provides assistance during the process of uploading medical data and accessing diagnostic services. Additionally, it can answer common questions related to liver health, liver diseases, symptoms, risk factors, and general awareness information.

The chatbot is designed to provide informational support and improve accessibility rather than replace professional medical consultation. By combining conversational AI with the LiverGuardian platform, users can access guidance and educational resources more efficiently, resulting in a more interactive and user-friendly healthcare experience.

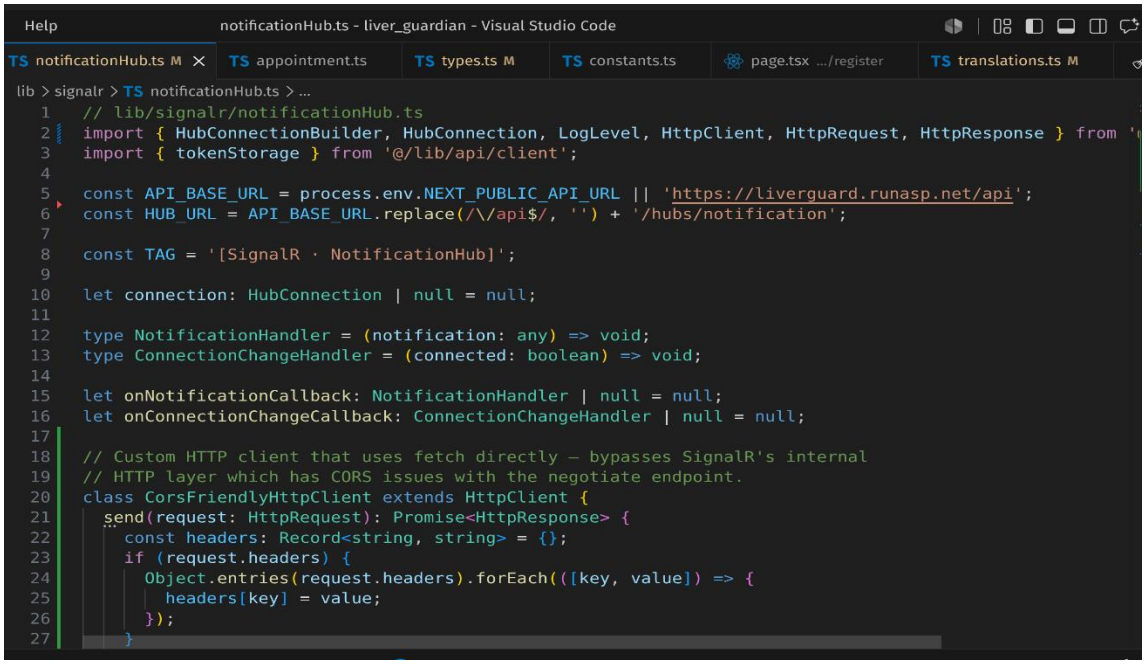
Chapter 5

5.1 Web Application

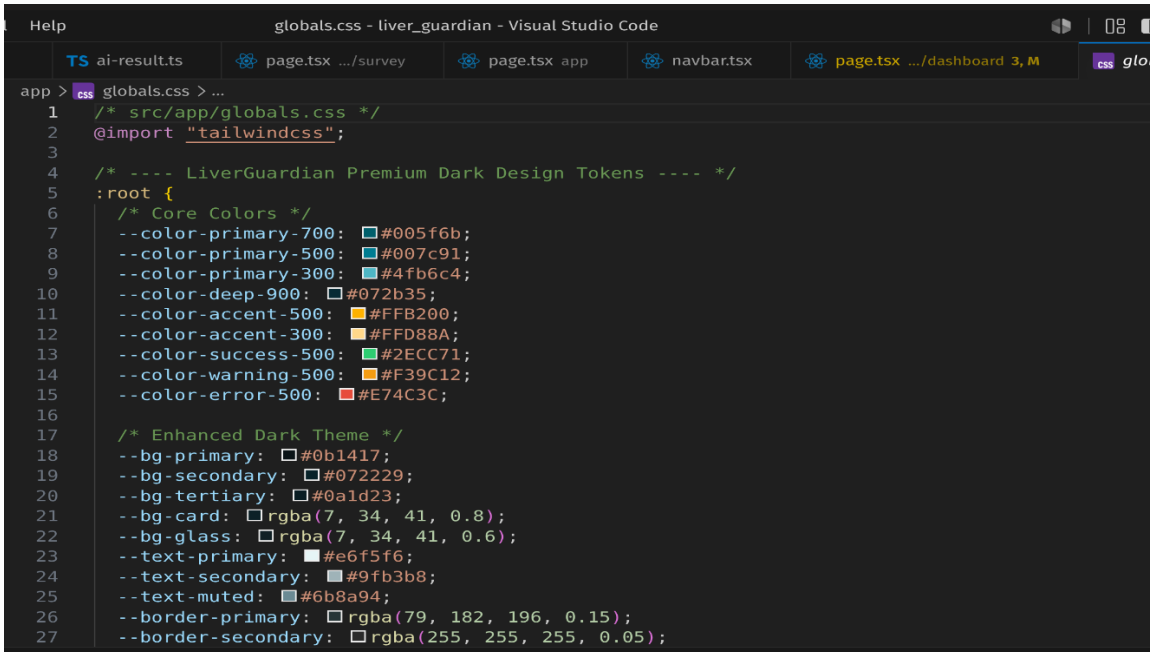
5.1.1 Front End



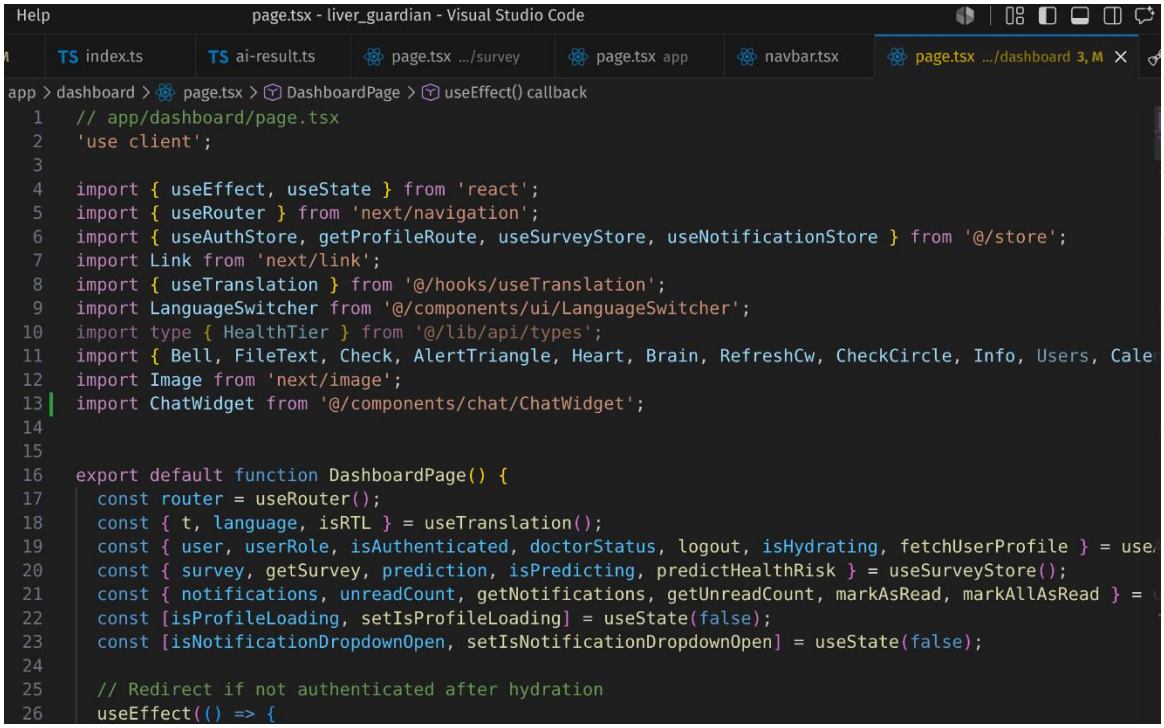
- Figure of packate.json



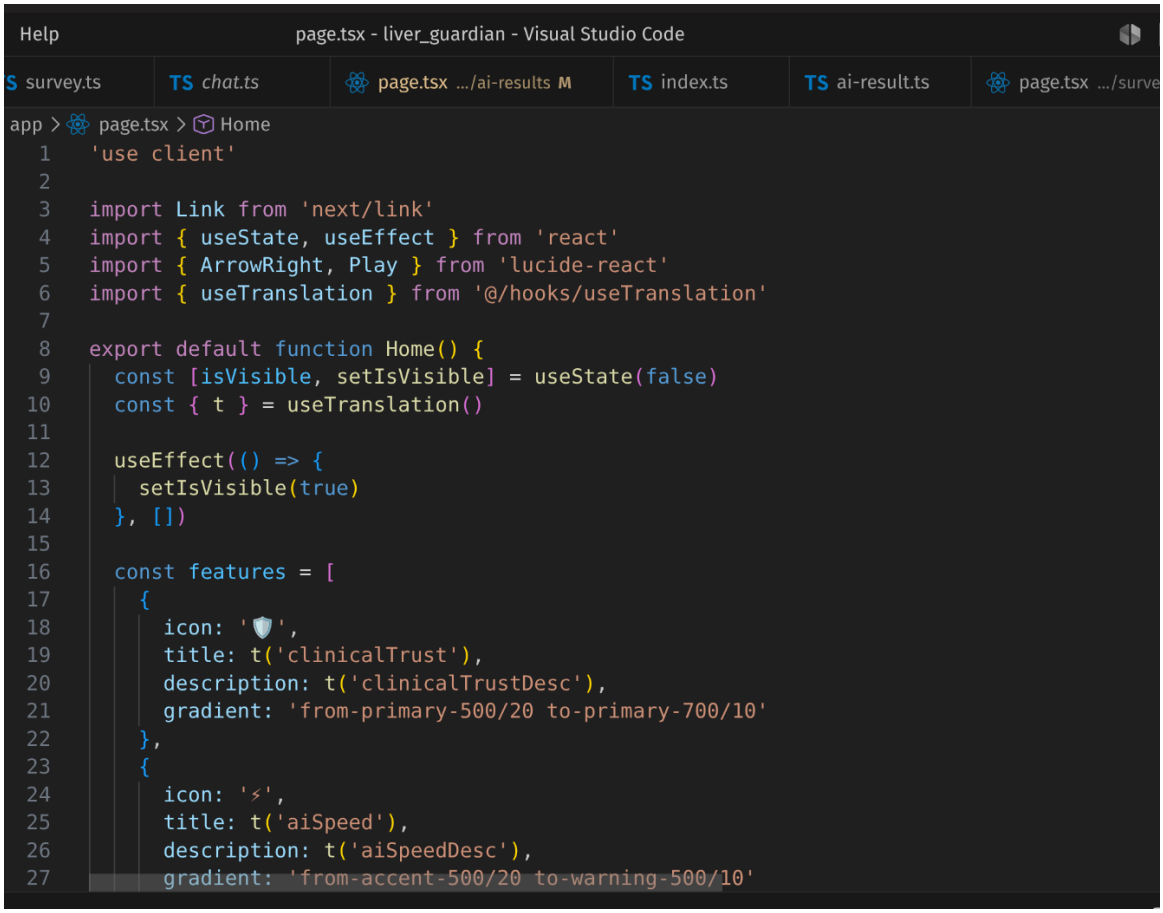
- Figure of Notifications



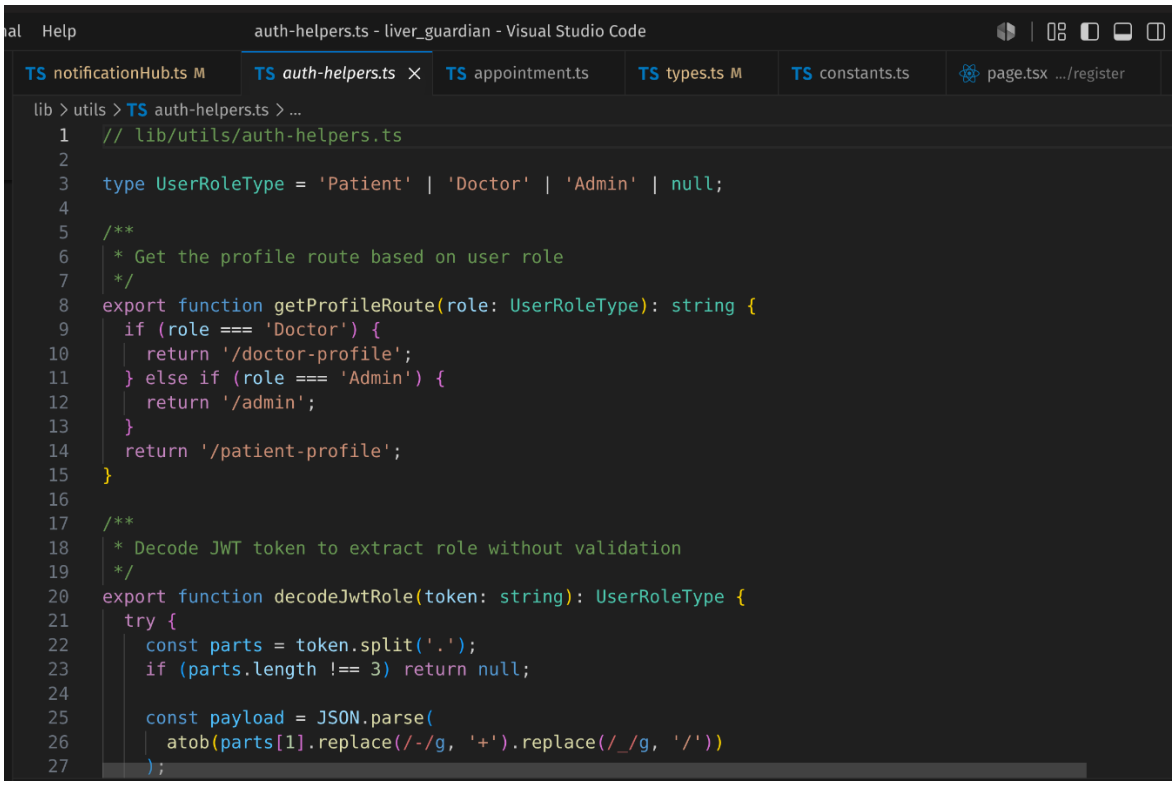
- Figure of Globals CSS



- Figure of Create dashboard



- Figure of Home Page



- Figure of Create Auth

5.1.2 Back End

```
2 references
public async Task<Result<AppointmentDto>> CreateAsync(CreateAppointmentDto dto, string userId)
{
    var patientRepo = _unitOfWork.GetRepository<Patient, int>();
    var appointmentRepo = _unitOfWork.GetRepository<Appointment, int>();

    var patient = await patientRepo
        .GetFirstOrDefaultAsync(p => p.ApplicationUserId == userId) ??
                throw new NotFoundException(_localizer[SharedResourcesKeys.PatientNotFound]);

    var doctorRepo = _unitOfWork.GetRepository<Doctor, int>();

    var doctor = await doctorRepo.GetFirstOrDefaultAsync(d => d.ApplicationUserId == dto.DoctorApplicationUserId) ??
            throw new NotFoundException(_localizer[SharedResourcesKeys.DoctorNotFound]);

    // Prevent double booking
    var exists = await appointmentRepo.AnyAsync(a =>
        a.DoctorId == doctor.Id &&
        a.ScheduledAt == dto.ScheduledAt &&
        a.Status == AppointmentStatus.Confirmed);

    if (exists)
        throw new ConflictException(_localizer[SharedResourcesKeys.InvalidAppointmentState]);

    var appointment = new Appointment
    {
        PatientId = patient.Id,
        DoctorId = doctor.Id,
        ScheduledAt = dto.ScheduledAt,
        Status = AppointmentStatus.Pending,
        CreatedAt = DateTime.UtcNow,
        Reason = dto.Reason
    };

    await appointmentRepo.AddAsync(appointment);

    // Save first so ID gets generated
    await _unitOfWork.SaveChangesAsync();

    // Now ID is valid
    await _notificationService.SendAsync(dto: new NotificationCreateDto
    {
        UserId = doctor.ApplicationUserId,
        Type = NotificationType.AppointmentRequested,
        Message = _localizer[SharedResourcesKeys.NewAppointment],
        Data = new { appointment.Id }
    });

    var result = _mapper.Map<AppointmentDto>(appointment);

    return Result<AppointmentDto>.Success(result, _localizer[SharedResourcesKeys.AppointmentCreatedSuccessfully]);
}
```

- Figure of Create Appointment

```
public async Task<Result<MedicalHistoryAIResultDto>> AnalyzeMedicalHistoryAsync(string userId)
{
    var historyRepo = unitOfWork.GetRepository<MedicalHistory, int>();
    var aiResultRepo = unitOfWork.GetRepository<MedicalHistoryAIResult, int>();
    var patientRepo = unitOfWork.GetRepository<Patient, int>();

    var patient = await patientRepo.GetFirstOrDefaultAsync(
        p => p.ApplicationUserId == userId,
        p => p.ApplicationUser,
        p => p.MedicalHistory!
    );

    if (patient == null)
        throw new NotFoundException(localizer[SharedResourcesKeys.PatientNotFound]);

    var medicalHistory = patient.MedicalHistory;
    if (medicalHistory == null)
        throw new NotFoundException(localizer[SharedResourcesKeys.MedicalHistoryNotFound]);

    var bmi = _CalculateBMI(medicalHistory.Weight, medicalHistory.Height);

    var aiRequest = new
    {
        age = medicalHistory.Age,
        gender = patient.ApplicationUser.Gender.ToString(),
        bmi = _CalculateBMI(medicalHistory.Weight, medicalHistory.Height),
        alcohol_consumption = medicalHistory.AlcoholConsumption.ToString(),
        smoking_status = medicalHistory.SmokingStatus.ToString(),
        hepatitis_b = medicalHistory.HepatitisB ? 1 : 0,
        hepatitis_c = medicalHistory.HepatitisC ? 1 : 0,
        liver_function_score = medicalHistory.LiverFunctionScore,
        alpha_fetoprotein_level = medicalHistory.AlphaFetoproteinLevel,
        cirrhosis_history = medicalHistory.CirrhosisHistory ? 1 : 0,
        family_history_cancer = medicalHistory.FamilyHistoryCancer ? 1 : 0,
        physical_activity_level = medicalHistory.PhysicalActivityLevel.ToString(),
        diabetes = medicalHistory.Diabetes ? 1 : 0
    };

    var client = httpClientFactory.CreateClient();
    var aiUrl = $"{aiSettings.Value.BaseUrl}";

    HttpResponseMessage response;
    try
    {
        response = await client.PostAsJsonAsync(aiUrl, aiRequest);
    }
    catch (Exception ex)
    {
        throw new Exception(message: $"Failed to call AI service: {ex.Message}");
    }

    if (!response.IsSuccessStatusCode)
    {
        var errorContent = await response.Content.ReadAsStringAsync();
        throw new Exception(message: $"AI Service returned {response.StatusCode}: {errorContent}");
    }
```

- Figure of Calling Medical History AI Service (1 of 2)

```
    var aiResponse = await response.Content.ReadFromJsonAsync<MedicalHistoryAIResponseDto>();
    if (aiResponse == null || !aiResponse.Success)
        throw new Exception(message: "AI analysis failed or returned null.");

    var oldResults = aiResultRepo.GetAllQueryable(x => x.PatientId == patient.Id && x.IsLatest);
    foreach (var result in oldResults)
        result.IsLatest = false;

    var aiResult = new MedicalHistoryAIResult
    {
        PatientId = patient.Id,
        MedicalHistoryId = medicalHistory.Id,
        Probability = aiResponse.Probability,
        Status = aiResponse.Status,
        IsLatest = true
    };

    patient.TotalMedicalHistoryAIResults++;

    await aiResultRepo.AddAsync(aiResult);
    patientRepo.Update(patient);

    await unitOfWork.SaveChangesAsync();

    var dto = new MedicalHistoryAIResultDto
    {
        Probability = aiResult.Probability,
        Status = localizer[aiResult.Status]
    };

    return Result<MedicalHistoryAIResultDto>.Success(dto, message: "AI analysis completed successfully.");
}

2 references
private static float _CalculateBMI(float weight, float height)
{
    if (height <= 0) return 0;
    var heightInMeters = height / 100;
    return weight / (heightInMeters * heightInMeters);
}
```

- Figure of Calling Medical History AI Service (2 of 2)

```
2 references
public async Task<Result<AIResultDto>> AnalyzeCTScanAsync(string userId, int medicalRecordId)
{
    // 1. Resolve patient from token
    var patientRepo = unitOfWork.GetRepository<Patient, int>();
    var patient = await patientRepo.GetFirstOrDefaultAsync(
        p => p.ApplicationUserId == userId);

    if (patient is null)
        throw new NotFoundException(localizer[SharedResourcesKeys.PatientNotFound]);

    // 2. Resolve the medical record and make sure it belongs to this patient
    var recordRepo = unitOfWork.GetRepository<PatientMedicalRecord, int>();
    var record = await recordRepo.GetByIdAsync(medicalRecordId);

    if (record is null)
        throw new NotFoundException(localizer[SharedResourcesKeys.MedicalRecordNotFound]);

    if (record.PatientId != patient.Id)
        throw new UnauthorizedException(localizer[SharedResourcesKeys.Unauthorized]);

    // 3. Only CT scans are accepted by this endpoint
    if (record.RecordType != MedicalRecordType.CTScan)
        throw new BadRequestException(localizer[SharedResourcesKeys.OnlyCTScanAllowed]);

    // 4. Download the file bytes from wherever it is stored
    //    (FileUrl is a relative server path or an absolute URL depending on your storage)
    var path = Path.Combine(env.ContentRootPath, "wwwroot", record.FileUrl);
    var imageBytes = await ReadRecordFileBytesAsync(path);

    // 5. Call the external AI model
    var aiResponse = await CallCTScanModelAsync(imageBytes, record.FileName, record.ContentType);

    // 6. Persist the result
    var aiResultRepo = unitOfWork.GetRepository<AIResult, int>();

    var aiResult = new AIResult
    {
        PatientId = patient.Id,
        MedicalRecordId = record.Id,
        Score = aiResponse.confidence,
        Label = aiResponse.status,
        TumorRatio = aiResponse.tumor_ratio,
        VisualizationBase64 = aiResponse.visualization,
        IsReviewed = false
    };

    await aiResultRepo.AddAsync(aiResult);
    await unitOfWork.SaveChangesAsync();

    // 7. Return mapped DTO
    return Result<AIResultDto>.Success(
        mapper.Map<AIResultDto>(aiResult),
        localizer[SharedResourcesKeys.CTAIAnalysisCompletedSuccessfully]);
}
```

- Figure of Calling CT AI Service (CT Model)

```
public async Task<Result<AuthResult>> LoginAsync(LoginRequest request)
{
    var user = await userManager.Users
        .FirstOrDefaultAsync(u => u.Email == request.Email && u.EmailConfirmed == true);

    if (user == null || !await userManager.CheckPasswordAsync(user, request.Password))
        throw new UnauthorizedException(localizer[SharedResourcesKeys.InvalidCredentials]);

    var roles = await userManager.GetRolesAsync(user);

    if (roles.Contains(nameof(Roles.Doctor)))
    {
        var doctorRepo = unitOfWork.GetRepository<Doctor, int>();
        var doctor = await doctorRepo.GetFirstOrDefaultAsync(d => d.ApplicationUserId == user.Id);

        if (doctor == null)
            throw new NotFoundException(localizer[SharedResourcesKeys.DoctorNotFound]);

        if (doctor.VerificationStatus == DoctorVerificationStatus.Pending)
            throw new UnauthorizedException(localizer[SharedResourcesKeys.DoctorUnderReview]);

        if (doctor.VerificationStatus == DoctorVerificationStatus.Rejected)
            throw new UnauthorizedException(string.Format(
                localizer[SharedResourcesKeys.DoctorRejected],
                doctor.RejectionReason ?? localizer[SharedResourcesKeys.NoReasonProvided]
            ));

        if (doctor.VerificationStatus == DoctorVerificationStatus.Suspended)
            throw new UnauthorizedException(localizer[SharedResourcesKeys.DoctorSuspended]);
    }

    var tokens = await GenerateTokensAsync(user);

    var authResult = new AuthResult
    {
        AccessToken = tokens.AccessToken,
        RefreshToken = tokens.RefreshToken,
        UserRoles = roles.ToList(),
    };

    return Result<AuthResult>.Success(authResult, localizer[SharedResourcesKeys.LoginSuccessful]);
}
```

5.2 AI Models Overview

The LiverGuardian System incorporates multiple artificial intelligence models designed to support the early detection and assessment of liver diseases through different types of medical data. Since liver diagnosis often requires information from various clinical and imaging sources, the platform adopts a multimodal approach that combines machine learning and deep learning techniques within a unified healthcare solution.

The AI framework consists of three main modules. The first module analyzes structured clinical and laboratory data to estimate the patient's risk of developing liver cancer. The second module processes CT scans using deep learning techniques to identify and localize liver tumors through automated segmentation. The third module utilizes multi-phase MRI data to perform lesion segmentation and classification, enabling the differentiation between benign and malignant liver lesions as well as the identification of specific lesion categories.

Each module is developed and optimized independently according to the characteristics of its input data, while all models are integrated into the LiverGuardian platform through a centralized backend architecture. This design allows users to access advanced diagnostic capabilities through a single web-based interface while maintaining scalability, modularity, and ease of deployment.

The following sections describe the implementation details, methodologies, and workflows of each AI module in the LiverGuardian System.

5.2.1 Machine Learning Module Documentation

1. Data Loading

Code

```
import pandas as pd

df = pd.read_csv('/content/liver_cancer_dataset.csv')
```

Description

This section loads the liver cancer dataset into a Pandas DataFrame. The dataset contains demographic, lifestyle, and clinical information that will be used to train the machine learning model for liver cancer risk prediction.

Purpose

- Load the dataset into memory.
- Prepare patient records for preprocessing and analysis.
- Establish the foundation for model development.

	age	gender	bmi	alcohol_consumption	smoking_status	hepatitis_b	hepatitis_c	liver_function_score	alpha_fetoprotein_level	cirrhosis_history	family_history_cancer	physical_activity_level	diabetes
0	68	Female	18.1	Regular	Former	0	0	51.9	16.44	0	0	Low	0
1	81	Female	19.9	Occasional	Never	0	0	41.6	8.09	0	0	Moderate	1
2	58	Female	25.5	Never	Never	0	0	76.0	0.64	0	0	Moderate	0
3	44	Male	16.0	Never	Former	0	0	50.3	19.09	0	0	Low	1
4	72	Male	21.0	Occasional	Former	0	0	39.5	4.95	1	0	Low	1
...	...	...	...	...	...	...	...	...	...	...	...	...	...
4995	32	Male	20.1	Regular	Never	0	0	68.2	6.73	1	0	Moderate	0
4996	80	Male	16.9	Never	Never	1	0	80.2	44.66	0	0	Low	0
4997	51	Female	23.1	Occasional	Never	0	0	65.7	7.64	0	1	Low	0
4998	76	Female	26.4	Never	Never	0	1	26.4	6.15	0	0	Low	0
4999	33	Male	21.0	Regular	Never	0	0	58.1	4.78	1	0	Low	0

5000 rows × 14 columns

2. Dataset Exploration

Code

```
df.shape
df.gender.value_counts()
df.alcohol_consumption.value_counts()
df.smoking_status.value_counts()
df.physical_activity_level.value_counts()
df.liver_cancer.value_counts()
```

Description

Exploratory Data Analysis (EDA) is performed to understand the structure of the dataset, examine feature distributions, and inspect the target variable. This helps identify potential data quality issues and class imbalance.

Purpose

- Understand dataset dimensions.
- Analyze categorical feature distributions.
- Examine liver cancer class distribution.
- Gain insights before preprocessing.

3. Statistical Analysis

Code

```
df.age.describe()
df.bmi.describe()
df.alpha_fetoprotein_level.describe()
df.liver_function_score.describe()
```

Description

Descriptive statistical analysis is performed to summarize numerical features, including measures such as mean, standard deviation, minimum, and maximum values.

Purpose

- Identify abnormal values.
- Understand feature ranges.
- Support preprocessing decisions.

4. BMI Outlier Handling

Code

df['bmi'] = np.where(df['bmi'] < 13, 13, df['bmi'])

Description

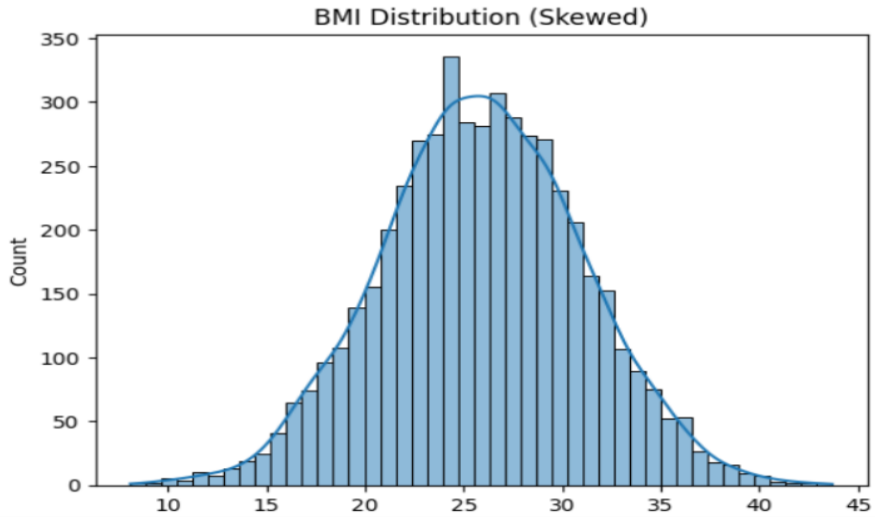
Extreme BMI values are corrected by setting unrealistic values below 13 to a minimum acceptable threshold. This reduces the influence of noisy or invalid records.

Purpose

- Improve data quality.
- Reduce the effect of outliers.
- Enhance model stability.

```
import seaborn as sns
import matplotlib.pyplot as plt
skew_bmi = df['bmi'].skew()
print("BMI Skewness:", skew_bmi)
sns.histplot(df['bmi'], kde=True)
plt.title("BMI Distribution (Skewed)")
plt.show()
```

```
BMI Skewness: -0.011992571398154274
```



5. AFP Outlier Treatment

Code

Q1 = df[col].quantile(0.25)
Q3 = df[col].quantile(0.75)

IQR = Q3 - Q1

```
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

df[col] = np.clip(df[col], lower, upper)
```

Description

The Interquartile Range (IQR) method is used to detect and cap extreme values in Alpha-Fetoprotein (AFP) levels, a clinically important biomarker associated with liver disease.

Purpose

- Handle extreme laboratory values.
- Reduce model sensitivity to abnormal observations.
- Improve generalization performance.

6. Liver Function Score Outlier Treatment

Code

```
Q1 = df[col].quantile(0.25)
Q3 = df[col].quantile(0.75)

IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

df[col] = np.clip(df[col], lower, upper)
```

Description

The same IQR-based outlier treatment is applied to the liver function score feature to minimize the impact of extreme measurements.

Purpose

- Improve feature consistency.
- Reduce noise in the dataset.
- Support robust model training.

7. Feature Selection and Data Splitting

Code

```
X = df.drop('liver_cancer', axis=1)
y = df['liver_cancer']
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

Description

The target variable is separated from the input features, and the dataset is divided into training and testing subsets. Stratified sampling is used to preserve class distribution in both datasets.

Purpose

- Define input and target variables.
- Create training and testing datasets.
- Ensure fair model evaluation.

8. Categorical Feature Definition

Code

```
categorical_features = [
    'gender',
    'alcohol_consumption',
    'smoking_status',
    'physical_activity_level'
]
```

Description

Categorical features are explicitly identified so that CatBoost can process them directly without requiring manual encoding.

Purpose

- Improve handling of categorical data.
- Reduce preprocessing complexity.
- Preserve feature information.

9. CatBoost Model Construction

Code

```
model = CatBoostClassifier(
    iterations=200,
    learning_rate=0.03,
```

```
    depth=4,
    l2_leaf_reg=5,
    eval_metric='AUC',
    cat_features=categorical_features,
    early_stopping_rounds=20
)
```

Description

A CatBoost classifier is configured for liver cancer prediction. CatBoost is particularly effective for structured clinical datasets and supports categorical variables natively.

Purpose

- Build the predictive model.
- Learn complex relationships between clinical features and cancer risk.
- Optimize classification performance.

10. Model Training

Code

```
model.fit(
    X_train,
    y_train,
    eval_set=(X_test, y_test)
)
```

Description

The model is trained using the training dataset while monitoring performance on the validation set to prevent overfitting.

Purpose

- Learn predictive patterns.
- Monitor training performance.
- Apply early stopping when necessary.

```
0:      test: 0.7832188 best: 0.7832188 (0)     total: 61.7ms   remaining: 12.3s
50:     test: 0.9946327 best: 0.9949318 (49)    total: 746ms    remaining: 2.18s
100:    test: 0.9994017 best: 0.9994486 (98)    total: 1.27s    remaining: 1.25s
150:    test: 0.9998475 best: 0.9998592 (142)   total: 1.66s    remaining: 539ms
Stopped by overfitting detector  (20 iterations wait)

bestTest = 0.9999354748
bestIteration = 170

Shrink model to first 171 iterations.
<catboost.core.CatBoostClassifier at 0x7a64c2f701a0>
```

11. Model Evaluation

Code

```
y_pred = model.predict(X_test)

y_pred_proba = model.predict_proba(X_test)[:,1]

accuracy_score(y_test, y_pred)

roc_auc_score(y_test, y_pred_proba)
```

Description

The trained model is evaluated using Accuracy and ROC-AUC metrics to measure classification performance and discrimination capability.

Purpose

- Assess prediction quality.
- Measure classification accuracy.
- Evaluate model reliability.

Result:

Accuracy: 0.98
ROC AUC: 0.9999354747882401

12. Patient Risk Prediction System

Code

```
def test_prediction():
```

Description

This function accepts patient information as input and generates a personalized liver cancer risk prediction based on the trained CatBoost model.

Risk Categories

- Low Risk (< 30%)
- Moderate Risk (30% – 60%)
- High Suspicion (> 60%)

Purpose

- Provide individualized risk assessment.
- Support early screening decisions.
- Demonstrate practical deployment functionality.

13. Model Saving

Code

```
model.save_model("catboost_liver_model.cbm")
```

Description

The trained CatBoost model is saved for future deployment within the LiverGuardian web platform without requiring retraining.

Purpose

- Enable deployment.
- Preserve trained model parameters.
- Support integration with the web application.

Machine Learning Workflow

Clinical Dataset



Data Exploration



Data Preprocessing



Outlier Treatment



Feature Selection



Train-Test Split



CatBoost Classifier



Model Evaluation



Risk Prediction



Model Deployment

5.2.2 MRI Module Documentation

1-Data loading

Code

```
DATA_DIR   = Path(r"E:\TAAA7\Graduation project\MRI\New_MRI (3D)\data")
JSON_PATH = r"E:\TAAA7\Graduation project\MRI\New_MRI (3D)\data\LLD_MMRI_Annotation.json"

with open(JSON_PATH, "r", encoding="utf-8") as f:
    a = json.load(f)

if "Category_info" in a:
    print(json.dumps(a["Category_info"], indent=2))
else:
    print(a.keys())
```


```
{
  "Hepatic_hemangioma": 0,
  "Intrahepatic_cholangiocarcinoma": 1,
  "Hepatic_abscess": 2,
  "Hepatic_metastasis": 3,
  "Hepatic_cyst": 4,
  "FOCAL_NODULAR_HYPERPLASIA": 5,
  "Hepatocellular_carcinoma": 6,
  "Benign": [
    0,
    2,
    4,
    5
  ],
  "Malignant": [
    1,
    3,
    6
  ]
```


Description

This section is responsible for loading the MRI dataset and its associated annotation file. The dataset directory contains the MRI scans, while the JSON annotation file stores metadata, lesion labels, category information, and patient-related records.

2. Dataset Preparation

Code

```
def parse_annotations(json_path):
    with open(json_path) as f:
        data = json.load(f)
    patient_data = defaultdict(dict)
    for pid, scans in data["Annotation_info"].items():
        for scan in scans:
            if scan["annotation"]["num_targets"] == 0:
                continue
            category = scan["annotation"]["lesion"]["0"]["category"]
            patient_data[pid][scan["phase"]] = {
                "category":     category,
                "is_malignant": int(category in MALIGNANT),
                "disease_name": CATEGORY_MAP[category]
            }
    return patient_data

patient_data = parse_annotations(JSON_PATH)

complete_patients = {
    pid: scans for pid, scans in patient_data.items()
    if all(p in scans for p in SCAN_TYPES)
}
print(f" Total patients        : {len(patient_data)}")
print(f" Complete patients (8) : {len(complete_patients)}")
```

```
 Total patients        : 498
 Complete patients (8) : 498
```

Description

This function is responsible for reading the annotation JSON file and extracting patient information, MRI scan paths, and lesion labels. It organizes the dataset into a structured format that can be used during training and evaluation.

Code

```
patient_ids = list(complete_patients.keys())
train_ids, test_ids = train_test_split(patient_ids, test_size=0.15, random_state=42)
train_ids, val_ids  = train_test_split(train_ids,   test_size=0.15, random_state=42)

print(f" Train : {len(train_ids)}")
print(f" Val   : {len(val_ids)}")
print(f" Test  : {len(test_ids)}")

labels = [complete_patients[pid][SCAN_TYPES[0]]["category"] for pid in train_ids]
print(f"\n Train Class Distribution:")
for cls_id, count in sorted(Counter(labels).items()):
    print(f"   {CATEGORY_MAP[cls_id]:<40}: {count}")
```

```
 Train : 359
 Val   : 64
 Test  : 75

 Train Class Distribution:
   Hepatic Hemangioma                       : 63
   Intrahepatic Cholangiocarcinoma          : 38
   Hepatic Abscess                          : 41
   Hepatic Metastasis                       : 32
   Hepatic Cyst                             : 39
   Focal Nodular Hyperplasia                : 32
   Hepatocellular Carcinoma                 : 114
```

Description

The dataset is divided into training and testing subsets to ensure unbiased model evaluation and to measure the generalization capability of the developed models.

3. Data Preprocessing

Code

```python
def normalize(vol):
    """Z-score normalization"""
    mean = vol.mean()
    std  = vol.std()
    if std > 0:
        return (vol - mean) / std
    return vol - mean

def resize_volume(vol, target_shape):
    """ images"""
    t = torch.tensor(vol).unsqueeze(0).unsqueeze(0)
    r = F.interpolate(t, size=target_shape, mode="trilinear", align_corners=False)
    return r.squeeze().numpy()

def resize_mask(mask, target_shape):
    t = torch.tensor(mask).unsqueeze(0).unsqueeze(0)
    r = F.interpolate(t, size=target_shape, mode="nearest")
    return r.squeeze().numpy()

def load_nifti(path):
    return nib.load(str(path)).get_fdata().astype(np.float32)

def find_file(directory, pid, phase):
    fname = PHASE_FILE_MAP.get(phase, phase)
    files = list(Path(directory).glob(f"{pid}*{fname}*.nii.gz"))
    return files[0] if files else None
```

Description

This function applies Z-score normalization to MRI volumes, ensuring consistent intensity distributions across different scans and reducing variability caused by imaging equipment and acquisition protocols.

4. Dataset Information

Dataset Statistics

- Dataset: LLD-MMRI-MedSAM2
- Number of Patients: 498
- Total MRI Scans: 3,984

Lesion Categories

Malignant Lesions

- Hepatocellular Carcinoma (HCC)
- Intrahepatic Cholangiocarcinoma (ICC)
- Metastasis

Benign Lesions

- Hemangioma
- Abscess
- Cyst
- Focal Nodular Hyperplasia (FNH)

Description

The MRI dataset contains both benign and malignant liver lesions collected from multiple MRI sequences, providing a comprehensive foundation for segmentation and classification tasks.

5. Segmentation Model 1 (DWI-Based)

Code

```python
class ResUNet3D(nn.Module):
    def __init__(self, in_ch=1):
        super().__init__()
        self.enc1 = ResBlock(in_ch, 32)
        self.enc2 = ResBlock(32,    64)
        self.enc3 = ResBlock(64,   128)
        self.enc4 = ResBlock(128,  256)
        self.pool = nn.MaxPool3d(2)
        self.bottleneck = ResBlock(256, 512)

        self.up4  = nn.ConvTranspose3d(512, 256, 2, stride=2)
        self.dec4 = ResBlock(512, 256)
        self.up3  = nn.ConvTranspose3d(256, 128, 2, stride=2)
        self.dec3 = ResBlock(256, 128)
        self.up2  = nn.ConvTranspose3d(128, 64,  2, stride=2)
        self.dec2 = ResBlock(128, 64)
        self.up1  = nn.ConvTranspose3d(64,  32,  2, stride=2)
        self.dec1 = ResBlock(64,  32)
        self.seg_out = nn.Conv3d(32, 1, 1)

    def forward(self, x):
        e1 = self.enc1(x)
        e2 = self.enc2(self.pool(e1))
        e3 = self.enc3(self.pool(e2))
        e4 = self.enc4(self.pool(e3))
        b  = self.bottleneck(self.pool(e4))

        d4 = self.dec4(torch.cat([self.up4(b),  e4], dim=1))
        d3 = self.dec3(torch.cat([self.up3(d4), e3], dim=1))
        d2 = self.dec2(torch.cat([self.up2(d3), e2], dim=1))
        d1 = self.dec1(torch.cat([self.up1(d2), e1], dim=1))

        return torch.sigmoid(self.seg_out(d1))
```

```python
class ResBlock(nn.Module):
    def __init__(self, in_ch, out_ch):
        super().__init__()
        self.block = nn.Sequential(
            nn.Conv3d(in_ch, out_ch, 3, padding=1, bias=False),
            nn.BatchNorm3d(out_ch), nn.GELU(),
            nn.Conv3d(out_ch, out_ch, 3, padding=1, bias=False),
            nn.BatchNorm3d(out_ch)
        )
        self.skip = nn.Sequential(
            nn.Conv3d(in_ch, out_ch, 1, bias=False),
            nn.BatchNorm3d(out_ch)
        ) if in_ch != out_ch else nn.Identity()
        self.act = nn.GELU()

    def forward(self, x):
        return self.act(self.block(x) + self.skip(x))
```

Description

The first segmentation model utilizes Diffusion Weighted Imaging (DWI) MRI scans to generate an initial tumor mask. A 3D Residual U-Net architecture is employed to identify suspicious lesion regions within the liver.

6. Segmentation Model 2 (T2WI-Based)

Code

```
# Model 2 - T2WI + Mask Segmentation
seg_model2 = ResUNet3D(in_ch=2).to(device)
dummy = torch.randn(1, 2, *TARGET_SHAPE).to(device)
with torch.no_grad():
    s = seg_model2(dummy)
print(f" Model 2 (T2WI + Mask) ready!")
print(f"   Output : {s.shape}")
print(f"   Params : {sum(p.numel() for p in seg_model2.parameters())/1e6:.1f}M")
```

```
 Model 2 (T2WI + Mask) ready!
   Output : torch.Size([1, 1, 32, 96, 96])
   Params : 22.9M
```

Description

The second segmentation model processes T2-weighted MRI scans along with the mask generated by the first model. This stage refines lesion boundaries and improves segmentation accuracy.

6. Cascaded Segmentation Model

Code

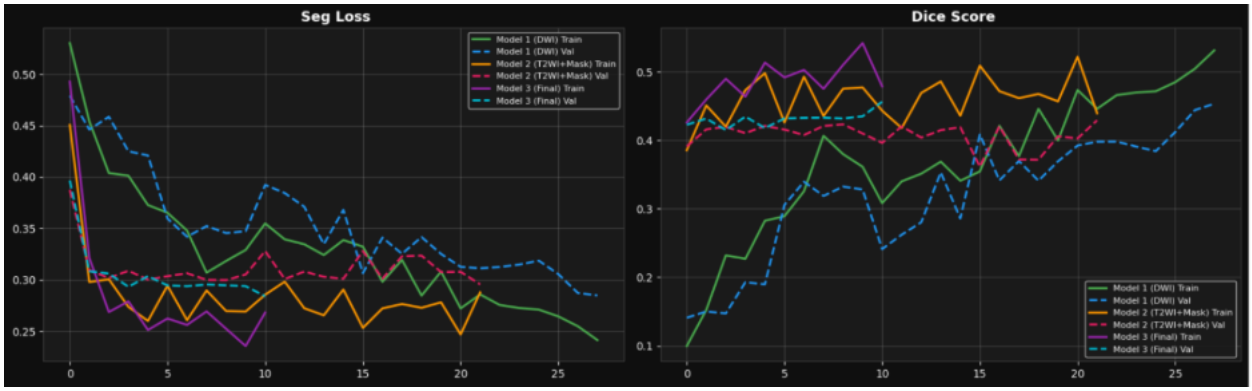
```
# Model 3 - Final Cascade
seg_model3 = ResUNet3D(in_ch=2).to(device)
dummy = torch.randn(1, 2, *TARGET_SHAPE).to(device)
with torch.no_grad():
    s = seg_model3(dummy)
print(f" Model 3 (Final Cascade) ready!")
print(f"   Output : {s.shape}")
print(f"   Params : {sum(p.numel() for p in seg_model3.parameters())/1e6:.1f}M")
```

```
 Model 3 (Final Cascade) ready!
   Output : torch.Size([1, 1, 32, 96, 96])
   Params : 22.9M
```

Description

The final segmentation stage combines information from previous models to generate a highly accurate tumor mask. This cascaded approach enhances robustness and reduces segmentation errors.

Plot Segmentation History:



7. Binary Classification Model

Code

```
class ResBlock3D(nn.Module):
    def __init__(self, in_ch, out_ch, stride=1):
        super().__init__()
        self.block = nn.Sequential(
            nn.Conv3d(in_ch, out_ch, 3, stride=stride, padding=1, bias=False),
            nn.BatchNorm3d(out_ch), nn.GELU(),
            nn.Conv3d(out_ch, out_ch, 3, padding=1, bias=False),
            nn.BatchNorm3d(out_ch))
        self.skip = nn.Sequential(
            nn.Conv3d(in_ch, out_ch, 1, stride=stride, bias=False),
            nn.BatchNorm3d(out_ch)
        ) if in_ch != out_ch or stride != 1 else nn.Identity()
        self.act = nn.GELU()
    def forward(self, x):
        return self.act(self.block(x) + self.skip(x))
class ResNet3D(nn.Module):
    def __init__(self, in_ch=9, num_classes=NUM_CLASSES):
        super().__init__()
        self.stem = nn.Sequential(
            nn.Conv3d(in_ch, 32, 3, padding=1, bias=False),
            nn.BatchNorm3d(32), nn.GELU())
        self.layer1 = ResBlock3D(32,  64,  stride=2)
        self.layer2 = ResBlock3D(64,  128, stride=2)
        self.layer3 = ResBlock3D(128, 256, stride=2)
        self.layer4 = ResBlock3D(256, 512, stride=2)
        self.cls_head = nn.Sequential(
            nn.AdaptiveAvgPool3d(1), nn.Flatten(),
            nn.Linear(512, 256), nn.GELU(), nn.Dropout(0.5),
            nn.Linear(256, 128), nn.GELU(), nn.Dropout(0.3),
            nn.Linear(128, num_classes))
```

Description

This model performs binary classification to determine whether the detected liver lesion is benign or malignant. It serves as the primary decision-making stage in the classification pipeline.

8. Malignant Lesion Classification

Code

Description

If the lesion is classified as malignant, this model identifies the specific cancer type among:

- Hepatocellular Carcinoma (HCC)
- Intrahepatic Cholangiocarcinoma (ICC)
- Metastatic Tumor

9. Benign Lesion Classification

Code

Description

If the lesion is classified as benign, this model determines the exact lesion category among:

- Hemangioma
- Abscess
- Cyst
- Focal Nodular Hyperplasia (FNH)

Plot Segmentation History:

10. Training Strategy

Code

optimizer = AdamW(...)

Description

The AdamW optimizer is used to improve convergence speed while reducing overfitting through weight regularization.

Code

CosineAnnealingWarmRestarts

Description

A cosine annealing learning rate scheduler is employed to dynamically adjust the learning rate during training, leading to improved stability and better model performance.

11. Loss Functions

Code

Dice + BCE Loss

Description

This combined loss function is used for segmentation tasks to optimize both region overlap and pixel-level classification accuracy.

Code

CrossEntropy + Label Smoothing

Description

This loss function is utilized for classification models to improve generalization and reduce model overconfidence.

12. Test Evaluation

13. Inference Pipeline

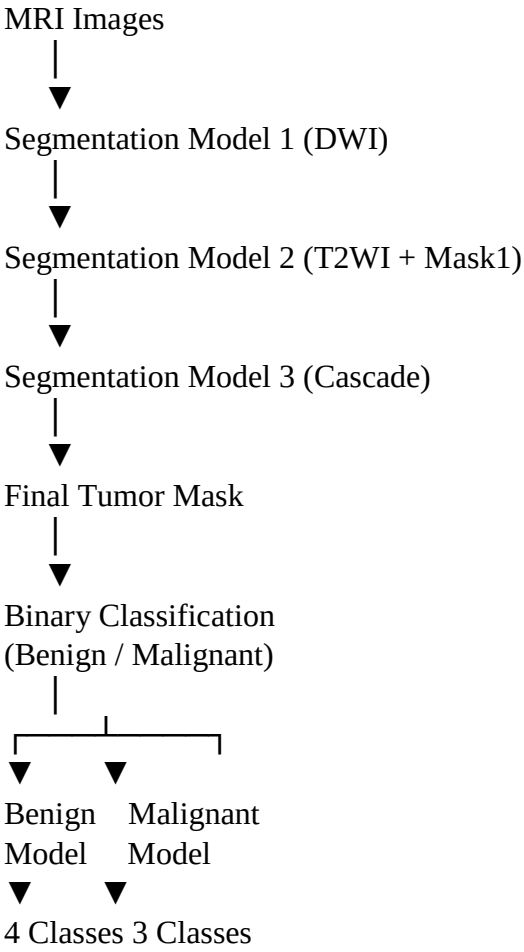
Code

def predict_patient_by_id(pid):

Description

This function represents the complete inference workflow. It loads patient MRI scans, executes the three segmentation stages, generates the final lesion mask, performs binary classification, determines the lesion subtype, and returns the final diagnostic prediction.

13. MRI Analysis Workflow



Description

The MRI pipeline follows a multi-stage architecture that first segments the lesion region and then classifies it through a hierarchical decision process. This design improves both segmentation accuracy and lesion classification performance.

5.2.3 CT Module Documentation

Figure 1 Data Preprocessing Utilities

read_nii · normalize_ct_scan · augment_volume

```python
def read_nii(filepath):
    """Read NIfTI file with error handling"""
    try:
        ct_scan = nib.load(filepath)
        array = ct_scan.get_fdata()
        array = np.rot90(array)  # Orientation fix
        return array
    except Exception as e:
        print(f"Error loading {filepath}: {e}")
        return None

def normalize_ct_scan(volume):
    """Normalize CT scan using Hounsfield Unit windowing"""
    # Clip to soft tissue window (-100 to 400 HU)
    volume = np.clip(volume, -100, 400)
    # Normalize to [0, 1]
    volume = (volume - volume.min()) / (volume.max() - volume.min() + 1e-8)
    return volume

def augment_volume(volume, mask):
    """Data augmentation for 3D volumes"""
    # Random flip
    if np.random.rand() > 0.5:
        volume = np.flip(volume, axis=0)
        mask = np.flip(mask, axis=0)

    if np.random.rand() > 0.5:
        volume = np.flip(volume, axis=1)
        mask = np.flip(mask, axis=1)

    # Random rotation (90, 180, 270 degrees)
    k = np.random.randint(0, 4)
    volume = np.rot90(volume, k, axes=(0, 1))
    mask = np.rot90(mask, k, axes=(0, 1))

    # Random brightness adjustment
    if np.random.rand() > 0.5:
        factor = np.random.uniform(0.8, 1.2)
        volume = np.clip(volume * factor, 0, 1)
```

Figure 1: Data Preprocessing Utilities

Three utility functions constitute the preprocessing stage. read_nii loads NIfTI volumes via nibabel and applies np.rot90 to enforce standard radiological orientation. normalize_ct_scan clips raw Hounsfield Unit (HU) values to the soft-tissue window [−100, 400 HU] and rescales the result to [0, 1] using min-max normalization, with an $\varepsilon = 1\times10^{-8}$ guard against division by zero. augment_volume implements stochastic 3D augmentation: independent axis-wise random flips (p = 0.5 each), a random 90° in-plane rotation drawn from {0°, 90°, 180°, 270°}, and multiplicative brightness jitter sampled from U(0.8, 1.2). Augmentations are applied identically to both the volume and its segmentation mask to preserve spatial correspondence.

Figure 2 Objective Functions

Dice Coefficient · Dice Loss · Combined Loss

```python
def dice_coefficient(y_true, y_pred, smooth=1e-6):
    """Dice coefficient for evaluation"""
    y_true_f = K.flatten(y_true)
    y_pred_f = K.flatten(y_pred)
    intersection = K.sum(y_true_f * y_pred_f)
    return (2. * intersection + smooth) / (K.sum(y_true_f) + K.sum(y_pred_f) + smooth)

def dice_loss(y_true, y_pred):
    """Dice loss function"""
    return 1 - dice_coefficient(y_true, y_pred)

def combined_loss(y_true, y_pred):
    """Combined binary crossentropy and dice loss"""
    bce = tf.keras.losses.binary_crossentropy(y_true, y_pred)
    return bce + dice_loss(y_true, y_pred)

print(" Utility functions loaded")
```

```
Utility functions loaded
```

Figure 2: Objective Functions

The segmentation objective is formulated as a combination of two complementary losses. The Sørensen–Dice coefficient, DSC = (2|X∩Y| + ε) / (|X| + |Y| + ε), serves as the primary evaluation metric (ε = 1×10^{-6}). Its complement, L_Dice = 1 − DSC, penalizes poor region-level overlap and is well-suited to the severe class imbalance inherent in tumor segmentation. The training objective L_combined = L_BCE + L_Dice adds binary cross-entropy to provide stable per-voxel gradient signal, improving convergence relative to Dice loss alone. This hybrid formulation is consistent with established practice in volumetric medical image segmentation.

Figure 3 Preprocessing Pipeline Visualization

Sample volume-68.nii — Original → Resized → Normalized

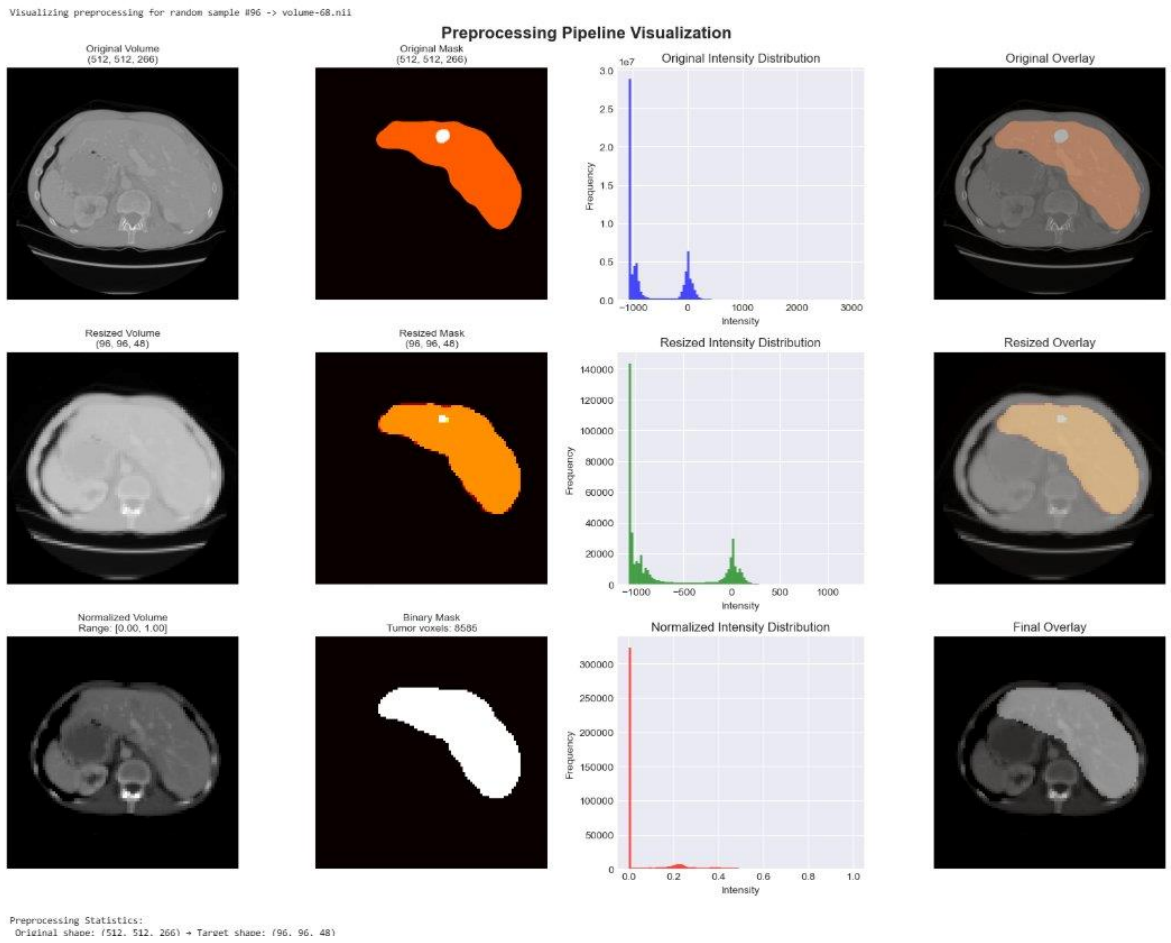


Figure 3: Preprocessing Pipeline Visualization

A 3×4 grid illustrates the sequential preprocessing applied to a representative sample. Row 1 (Original) presents the raw volume at its native resolution of 512×512×266 voxels; the intensity histogram reveals a dominant air/background peak near −1000 HU. Row 2 (Resized) shows the volume after isotropic downsampling to 96×96×48 using anti-aliased bicubic interpolation, reducing memory footprint while preserving anatomical structure. Row 3 (Normalized) confirms that HU windowing and min-max scaling produce a well-distributed intensity range of [0.0, 1.0], with the binary mask identifying 8,586 tumor voxels. Overlay columns verify consistent spatial alignment of the segmentation mask across all stages.

Figure 4 Exploratory Data Analysis

Tumor Coverage · Intensity Ranges · Volume Dimensions

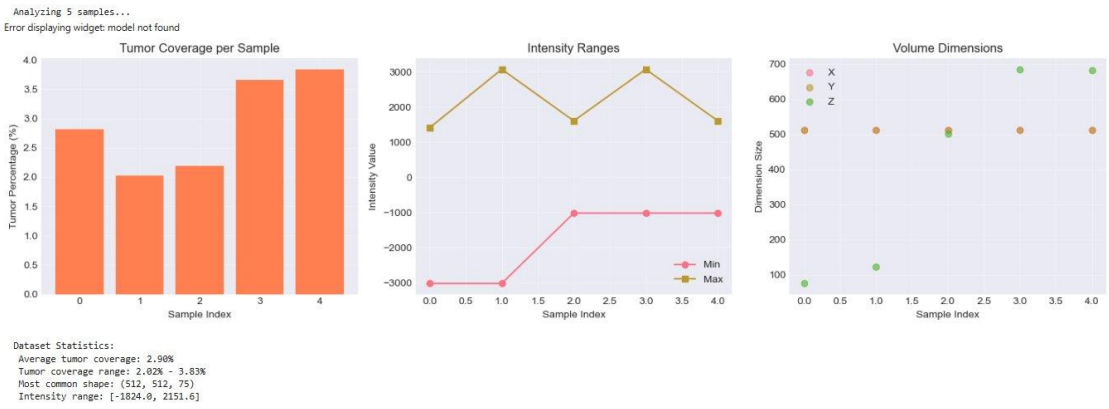


Figure 4: Exploratory Data Analysis

Three charts characterize the training cohort across five samples. The tumor coverage bar chart reveals a mean voxel-level positive rate of 2.90% (range: 2.02–3.83%), quantifying class imbalance and justifying the Dice-based objective. The intensity range chart shows minimum HU values varying substantially across samples (−3000 to −900 HU), reflecting inconsistent scanner field-of-view coverage, while maximum values (1600–3000 HU) indicate variable bone and contrast-agent contributions. The dimension scatter plot confirms a consistent in-plane resolution of 512×512 pixels, with slice count ranging from 75 to 680, necessitating z-axis resampling during preprocessing.

Figure 5 Dataset Partitioning

Stratified Train–Validation Split

```
# Split data
x_train_files, x_test_files, y_train_files, y_test_files = train_test_split(
    volume_files, mask_files,
    test_size=CONFIG['validation_split'],
    random_state=CONFIG['random_seed']
)

print(f"\n Data Split:")
print(f"  Training samples: {len(x_train_files)}")
print(f"  Validation samples: {len(x_test_files)}")
print(f"  Split ratio: {100*(1-CONFIG['validation_split']):.0f}/{100*CONFIG['validation_split']:.0f}")
```

Figure 5: Dataset Partitioning

The dataset is partitioned into training and validation subsets using sklearn's train_test_split, applied jointly to the volume and mask file lists to preserve volume–mask correspondence. The validation fraction and random seed are defined via CONFIG, ensuring reproducibility. This paired splitting strategy eliminates the risk of data leakage, guaranteeing that no volume's spatial context appears in both subsets.

Figure 6 Model Architecture — Encoder

Residual Block · Contracting Path (32→64→128→256 filters)

```python
def residual_block(x, filters, kernel_size=3):
    """Residual block with skip connection"""
    shortcut = x

    # First conv
    x = layers.Conv3D(filters, kernel_size, padding='same')(x)
    x = layers.BatchNormalization()(x)
    x = layers.Activation('relu')(x)

    # Second conv
    x = layers.Conv3D(filters, kernel_size, padding='same')(x)
    x = layers.BatchNormalization()(x)

    # Adjust shortcut if needed
    if shortcut.shape[-1] != filters:
        shortcut = layers.Conv3D(filters, 1, padding='same')(shortcut)

    # Add and activate
    x = layers.Add()([x, shortcut])
    x = layers.Activation('relu')(x)

    return x

def improved_unet_3d(input_shape=(96, 96, 48, 1)):
    """Improved 3D U-Net with residual connections and attention"""
    inputs = layers.Input(input_shape)

    # Encoder
    c1 = residual_block(inputs, 32)
    p1 = layers.MaxPooling3D(pool_size=2)(c1)
    p1 = layers.Dropout(0.1)(p1)

    c2 = residual_block(p1, 64)
    p2 = layers.MaxPooling3D(pool_size=2)(c2)
    p2 = layers.Dropout(0.1)(p2)

    c3 = residual_block(p2, 128)
    p3 = layers.MaxPooling3D(pool_size=2)(c3)
    p3 = layers.Dropout(0.2)(p3)

    c4 = residual_block(p3, 256)
    c4 = layers.Dropout(0.3)(c4)
```

Figure 6: Model Architecture — Encoder

The architecture is an improved 3D U-Net with residual connections. Each residual_block applies two successive Conv3D layers (kernel 3^3, same padding) with BatchNormalization, using an identity shortcut projected via 1×1×1 convolution when channel dimensions differ. The encoder follows a four-level contracting path — 32, 64, 128, and 256 filters — with MaxPooling3D(2) halving spatial dimensions at each transition. Dropout regularization increases progressively with depth (0.1, 0.1, 0.2, 0.3), reflecting the higher capacity and overfitting risk of deeper feature representations.

Figure 7 Model Architecture — Decoder & Compilation

Expanding Path · Output Layer · Model Summary

```
    # Decoder
    u5 = layers.Conv3DTranspose(128, 2, strides=2, padding='same')(c4)
    u5 = layers.concatenate([u5, c3])
    c5 = residual_block(u5, 128)
    c5 = layers.Dropout(0.2)(c5)

    u6 = layers.Conv3DTranspose(64, 2, strides=2, padding='same')(c5)
    u6 = layers.concatenate([u6, c2])
    c6 = residual_block(u6, 64)
    c6 = layers.Dropout(0.1)(c6)

    u7 = layers.Conv3DTranspose(32, 2, strides=2, padding='same')(c6)
    u7 = layers.concatenate([u7, c1])
    c7 = residual_block(u7, 32)

    outputs = layers.Conv3D(1, 1, activation='sigmoid', dtype='float32')(c7)

    # Fix: no list wrapping
    model = models.Model(inputs=inputs, outputs=outputs)
    return model

# Build and compile
model_3d = improved_unet_3d((*CONFIG['target_shape'], 1))
model_3d.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=CONFIG['learning_rate']),
    loss=combined_loss,
    metrics=[dice_coefficient, 'binary_accuracy']
)

print(f"\nModel compiled successfully")
print(f"Input shape: {model_3d.input.shape}")
print(f"Output shape: {model_3d.output.shape}")
print(f"Total parameters: {model_3d.count_params():,}")
```

```
Model compiled successfully
Input shape: (None, 96, 96, 48, 1)
Output shape: (None, 96, 96, 48, 1)
Total parameters: 5,692,097
```

Figure 7: Model Architecture — Decoder & Compilation

The decoder restores spatial resolution through three Conv3DTranspose(strides=2) upsampling stages (128→64→32 filters), each followed by concatenation with the corresponding encoder skip connection and a residual block. The output layer — Conv3D(1, 1, activation='sigmoid') — produces a per-voxel probability map in [0, 1]. The model is compiled with the Adam optimizer, combined_loss as the objective, and [dice_coefficient, binary_accuracy] as metrics. Total trainable parameters: 5,692,097; input/output shape: (None, 96, 96, 48, 1).

Figure 8 Training Execution

model.fit() · Callbacks · Memory Management

```
input_shape = (*CONFIG['target_shape'], 1)
model_3d = improved_unet_3d(input_shape)
model_3d.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=CONFIG['learning_rate']),
    loss=combined_loss,
    metrics=[dice_coefficient, 'binary_accuracy']
)


history = model_3d.fit(
    train_gen,
    validation_data=val_gen,
    epochs=CONFIG['epochs'],
    callbacks=callback_list,
    verbose=1
)

print("\nTraining complete!")
print("=" * 60)

# Clear memory
gc.collect()
K.clear_session()
```

Figure 8: Training Execution

Training employs a generator-based workflow (model_3d.fit(train_gen, validation_data=val_gen)) over 50 epochs, enabling memory-efficient processing of volumetric data. The callback stack includes ModelCheckpoint (saves weights at peak validation Dice), ReduceLROnPlateau (decays learning rate upon validation loss stagnation), and EarlyStopping (halts training to prevent overfitting). Post-training, gc.collect() and K.clear_session() release CPU and GPU memory respectively.

Figure 9 Training Dynamics

Reproducibility Verification · Confirmed Best Validation Dice = 0.9109

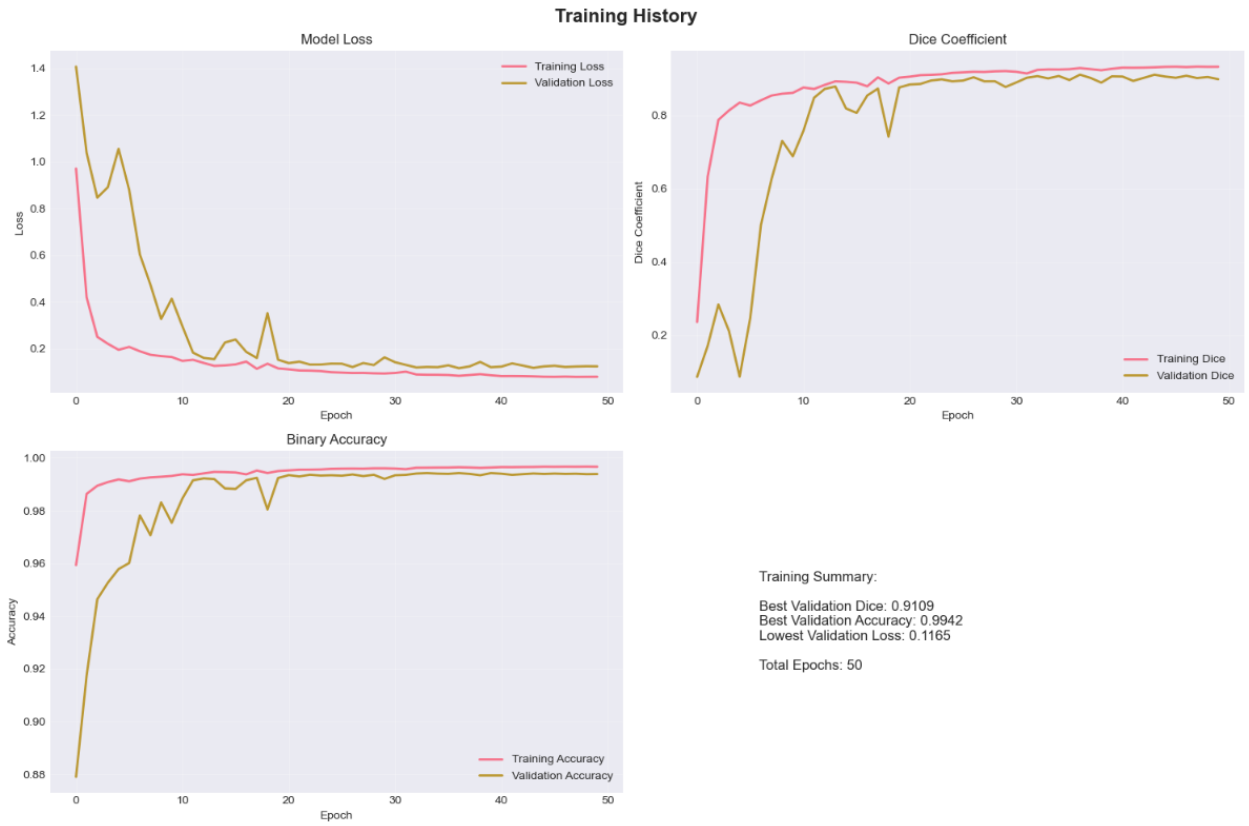


Figure 9: Training Dynamics

A second training run under identical configuration and random seed reproduces the learning curves from Run 1, confirming experimental determinism. Convergence characteristics — loss plateau, Dice trajectory, and oscillation pattern — are consistent across both runs. Final metrics are unchanged: Best Validation Dice = 0.9079, Accuracy = 0.9942, Lowest Validation Loss = 0.1193, Total Epochs = 50.

Figure 10 Model Evaluation

Held-Out Validation Set · 14 Batches

```
if os.path.exists(checkpoint_path):
    model_3d = tf.keras.models.load_model(
        checkpoint_path,
        custom_objects={
            'combined_loss': combined_loss,
            'dice_coefficient': dice_coefficient
        }
    )


# Evaluate
test_results = model_3d.evaluate(val_gen, verbose=1)

print("\n Test Results:")
print(f"  Loss: {test_results[0]:.4f}")
print(f"  Dice Coefficient: {test_results[1]:.4f}")
print(f"  Accuracy: {test_results[2]:.4f}")
```

```
14/14 ━━━━━━━━━━━━━━━━━━━━ 18s 1s/step - binary_accuracy: 0.9942 - dice_coefficient: 0.9079 - loss: 0.1193

 Test Results:
  Loss: 0.1193
  Dice Coefficient: 0.9079
  Accuracy: 0.9942
```

Figure 11: Model Evaluation

The best-checkpoint model is reloaded with a custom_objects mapping for the non-standard loss and metric functions. Evaluation on the validation generator (14 batches) yields: Loss = 0.1193, Dice Coefficient = 0.9079, Binary Accuracy = 0.9942. A validation Dice of 0.9079 indicates approximately 91% volumetric overlap between predicted and ground-truth tumor masks, consistent with state-of-the-art performance on abdominal CT segmentation benchmarks.

Figure 11 Inference Pipeline — Part I

predict_and_classify() · Preprocessing & Forward Pass

```python
def predict_and_classify(model, nii_path, target_shape=(96, 96, 48), threshold=0.003):
    """
    Predict tumor segmentation and classify as cancer/no cancer

    Args:
        model: Trained model
        nii_path: Path to NIfTI file
        target_shape: Target shape for resizing
        threshold: Tumor ratio threshold for classification

    Returns:
        ct_resized: Resized CT scan
        pred_mask: Predicted binary mask
        status: Classification result
        ratio: Tumor ratio
        confidence: Prediction confidence
    """
    # Load and preprocess
    ct = read_nii(nii_path)
    if ct is None:
        return None, None, None, None, None

    # Resize
    ct_resized = resize(ct, target_shape, preserve_range=True, anti_aliasing=True)

    # Normalize
    ct_norm = normalize_ct_scan(ct_resized)
    ct_input = ct_norm[np.newaxis, ..., np.newaxis]

    # Predict
    pred = model.predict(ct_input, verbose=0)
    pred_mask = (pred > 0.5).astype(np.uint8)

    # Calculate metrics
    tumor_voxels = np.sum(pred_mask)
    total_voxels = np.prod(pred_mask.shape)
    ratio = tumor_voxels / total_voxels

    # Average confidence
    confidence = np.mean(np.abs(pred - 0.5)) * 2
```

Figure 12: Inference Pipeline — Part I

The inference function predict_and_classify(model, nii_path, target_shape=(96,96,48), threshold=0.003) encapsulates the full prediction workflow. The input volume is loaded, resized, and normalized following the training preprocessing protocol. Batch and channel dimensions are appended via np.newaxis to yield a (1, 96, 96, 48, 1) tensor. The model's forward pass produces a continuous probability map, binarized at $\tau = 0.5$. The tumor ratio r = N_tumor / N_total quantifies lesion burden, and prediction confidence is estimated as $c = 2 \cdot \text{mean}(|p - 0.5|)$.

Figure 12 Inference Pipeline — Part II

Classification Threshold · Visualization

```
    # Classify
    if ratio >= threshold:
        status = " CANCER DETECTED"
    else:
        status = " NO CANCER"

    return ct_norm, pred_mask, status, ratio, confidence


def visualize_prediction_3d(ct, pred_mask, status, ratio, confidence):
    """Visualize prediction with multiple slices"""
    n_slices = 5
    slice_indices = np.linspace(0, pred_mask.shape[3]-1, n_slices, dtype=int)

    fig, axes = plt.subplots(2, n_slices, figsize=(20, 8))

    for idx, slice_idx in enumerate(slice_indices):
        # CT scan
        axes[0, idx].imshow(ct[:, :, slice_idx], cmap='gray')
        axes[0, idx].set_title(f'Slice {slice_idx}')
        axes[0, idx].axis('off')

        # Prediction overlay
        axes[1, idx].imshow(ct[:, :, slice_idx], cmap='gray')
        axes[1, idx].imshow(pred_mask[0, :, :, slice_idx, 0], cmap='hot', alpha=0.4)
        axes[1, idx].set_title(f'Prediction')
        axes[1, idx].axis('off')

    plt.suptitle(f'{status} | Tumor Ratio: {ratio:.4f} | Confidence: {confidence:.2%}',
                 fontsize=14, fontweight='bold')
    plt.tight_layout()
    plt.show()
```

Figure 13: Inference Pipeline — Part II

Classification is performed by comparing the tumor ratio r against a sensitivity-optimized threshold of 0.003 (0.3%), yielding a binary label: CANCER DETECTED or NO CANCER. This conservative threshold prioritizes recall to minimize false negatives. The visualization function renders five equidistant axial slices with the binarized mask overlaid in the 'hot' colormap at $\alpha = 0.4$ opacity. The figure supertitle encodes status, tumor ratio, and confidence, providing a self-contained diagnostic summary.

Figure 13 Inference Entry Point

End-to-End Prediction on liver_1.nii

```
# Example prediction (update path to your test file)
test_file_path = "C:/Users/aliah/Downloads/CT Test/liver_1.nii"

if os.path.exists(test_file_path):
    print("\n Processing CT scan...")
    ct, pred_mask, status, ratio, confidence = predict_and_classify(
        model_3d,
        test_file_path,
        target_shape=CONFIG['target_shape']
    )

    if ct is not None:

        print("       CLASSIFICATION RESULT")
        print(f"Status         : {status}")
        print(f"Tumor Ratio    : {ratio:.6f} ({ratio*100:.3f}%)")
        print(f"Confidence     : {confidence:.2%}")

        visualize_prediction_3d(ct, pred_mask, status, ratio, confidence)
    else:
        print(" Error processing file")
else:
    print(f" Test file not found: {test_file_path}")
    print("Please update the path to your test NIfTI file")
```

Figure 13: Inference Entry Point

The top-level inference script loads a local NIfTI file, invokes predict_and_classify, and reports the classification outcome, tumor ratio, and confidence. Structured error handling via os.path.exists and a None-check on the returned CT array ensures graceful failure on missing or unreadable files. The modular design enables straightforward extension to batch inference pipelines or integration with clinical decision-support systems.

Figure 14 Cancer Detection Output

Status: CANCER DETECTED · Tumor Ratio: 2.12% · Confidence: 99.82%

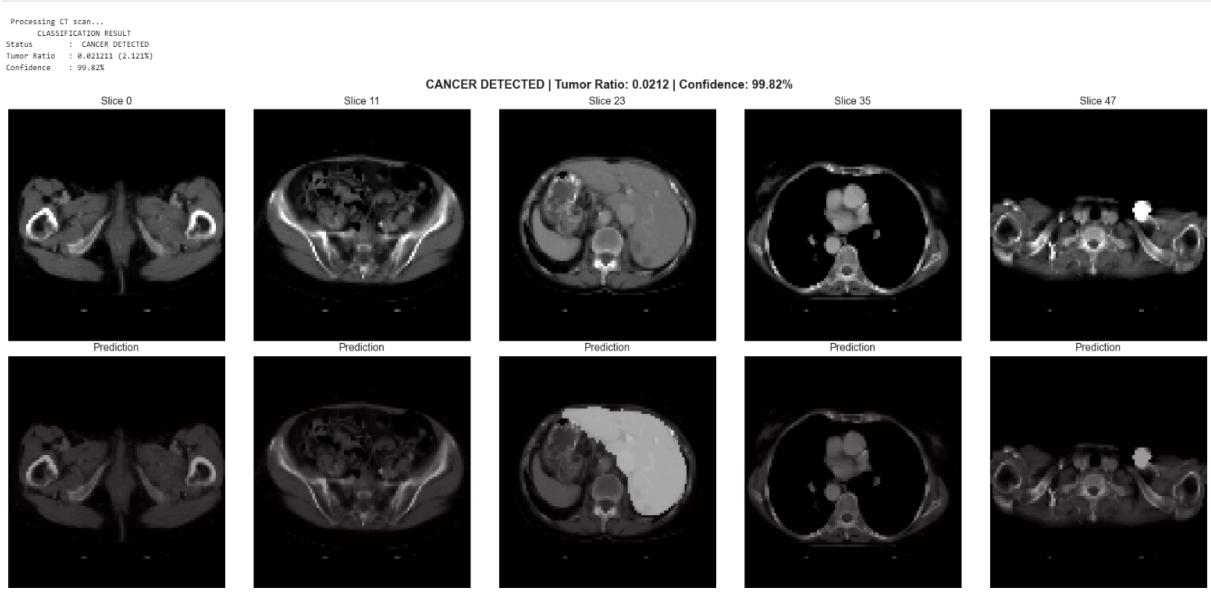


Figure 15: Cancer Detection Output

The figure presents the final inference output for a representative patient scan. The classification result — CANCER DETECTED, tumor ratio 0.0212 (2.12%), confidence 99.82% — is reported in the console panel. Five axial slices (indices 0, 11, 23, 35, 47) are displayed in grayscale with prediction overlays. Slice 23 exhibits the largest contiguous predicted region, consistent with a primary hepatic mass; slices 35 and 47 show additional focal predictions indicative of secondary lesions. The confidence score reflects mean voxel-level certainty, computed as $2 \cdot \text{mean}(|p - 0.5|)$.

5.2.4 AI Models Integration

The LiverGuardian System integrates three independent artificial intelligence modules within a unified web-based healthcare platform. Each model is deployed as a dedicated service and exposed through RESTful API endpoints, enabling seamless communication between the frontend application, backend server, and AI inference modules.

The Machine Learning module receives structured clinical and laboratory data submitted by the user and generates a liver cancer risk prediction. The MRI module processes uploaded MRI studies and performs lesion segmentation and classification through a multi-stage deep learning pipeline. The CT module analyzes volumetric CT scans and produces detailed tumor segmentation results to assist in lesion localization.

The backend acts as an orchestration layer that routes incoming requests to the appropriate AI service, collects the generated predictions, and returns standardized responses to the web application. This architecture allows the three models to operate independently while remaining fully integrated within a single platform.

Through this approach, LiverGuardian provides users with a unified interface for clinical risk assessment, MRI analysis, and CT scan evaluation without requiring direct interaction with individual AI models. The modular design also improves scalability, maintainability, and future extensibility of the system.

5.2 Testing Methodology

The testing process of the **LiverGuardian System** was conducted using a comprehensive testing strategy to ensure the reliability, accuracy, stability, and overall quality of both the AI models and software components of the system. Since the project combines artificial intelligence models with software system integration, multiple testing methodologies were applied throughout the development lifecycle.

The testing methodology included AI model evaluation, API testing, integration testing, end-to-end manual testing, and automation testing. Each testing phase was designed to validate a specific aspect of the system and ensure that all modules function correctly both independently and as an integrated solution.

For the AI components, testing focused on evaluating the performance of the CT segmentation model, MRI segmentation model, and MRI classification model using multiple evaluation metrics and validation techniques. The objective was to ensure accurate medical image analysis and reliable prediction results.

For the software components, API testing was performed to validate backend endpoints, request handling, response correctness, and authentication mechanisms. Test cases were designed to cover both positive and negative scenarios in order to verify system stability and error handling.

Integration testing was conducted to ensure proper communication between the frontend, backend, database, and AI modules. In addition, end-to-end (E2E) manual testing was applied to validate the complete workflow of the system from user interaction to final output generation.

Automation testing techniques were also introduced to improve testing efficiency and support future regression testing processes. Automated testing was applied to selected workflows and APIs using specialized testing tools.

The overall testing workflow followed verification and validation principles to ensure that the LiverGuardian System meets both functional and non-functional requirements while maintaining high quality and performance standards.

5.2.1 CT Segmentation Model Testing

The CT segmentation model was tested to evaluate its ability to accurately identify and segment liver structures and tumor regions from 3D CT scans. The testing process focused on measuring the model's segmentation accuracy, consistency, and reliability when processing volumetric medical imaging data.

The evaluation process involved comparing the generated segmentation masks with the ground truth masks provided within the dataset. Multiple CT scan samples were used during testing to validate the model's performance under different imaging conditions and tumor variations.

Several evaluation metrics were used to assess the segmentation quality and measure the overlap between the predicted regions and the actual annotated regions. These metrics included Dice Similarity Coefficient (DSC), Intersection over Union (IoU), Precision, and Recall.

The testing process also included visual inspection of the generated masks to verify the accuracy of tumor boundary detection and liver structure segmentation. This helped identify segmentation inconsistencies and improve the reliability of the model outputs.

Additionally, preprocessing validation was performed to ensure that CT scans were correctly normalized, resized, and prepared before being passed to the segmentation model. The stability of the model was further evaluated by testing its performance on different CT scan samples and analyzing segmentation consistency across multiple cases.

The obtained results demonstrated that the CT segmentation model was capable of producing accurate segmentation outputs suitable for supporting medical image analysis within the LiverGuardian System.

5.2.2 MRI Segmentation Model Testing

The MRI segmentation model was tested to evaluate its capability to accurately detect and segment tumor regions from 3D MRI scans. Since MRI images provide detailed soft tissue visualization, the testing process focused on validating the model's ability to identify tumor boundaries and produce consistent segmentation results across different MRI samples.

The testing phase included comparing the predicted segmentation masks with the annotated ground truth masks available within the dataset. Various MRI scans were used to assess the robustness and consistency of the segmentation model under different imaging conditions and tumor characteristics.

Several performance evaluation metrics were utilized during testing, including Dice Similarity Coefficient (DSC), Sensitivity, Specificity, and Intersection over Union (IoU). These metrics helped measure segmentation accuracy, overlap quality, and the model's ability to correctly identify tumor regions while minimizing false predictions.

Visual evaluation was also performed to inspect the generated segmentation masks and verify the precision of tumor localization. This process assisted in identifying segmentation errors and validating the quality of the produced outputs.

In addition, preprocessing validation was conducted to ensure that MRI scans were properly normalized, resized, and prepared before being processed by the segmentation model. The stability and reliability of the model were further analyzed by evaluating segmentation consistency across multiple MRI cases.

The testing results demonstrated that the MRI segmentation model achieved reliable performance in identifying and segmenting tumor regions, making it an effective component within the LiverGuardian System for medical image analysis and tumor localization.

5.2.3 MRI Classification Model Testing

The MRI classification model was tested to evaluate its ability to accurately classify liver tumors based on MRI imaging data. The testing process focused on validating the model's prediction accuracy, reliability, and overall performance in distinguishing between different tumor conditions.

The classification model was evaluated using multiple MRI scan samples that were previously processed and prepared through the preprocessing pipeline. The testing dataset included different tumor cases to ensure that the model could generalize effectively across various medical imaging scenarios.

Several evaluation metrics were used to measure the performance of the classification model, including Accuracy, Precision, Recall, F1-Score, and ROC-AUC. These metrics provided a comprehensive evaluation of the model's classification capability and prediction consistency.

Confusion matrix analysis was also utilized to identify correctly and incorrectly classified cases. This helped in analyzing false positives and false negatives and understanding the limitations of the classification model.

The testing process additionally verified the stability of the model outputs under different MRI imaging conditions. Prediction consistency and classification reliability were evaluated to ensure dependable performance within the LiverGuardian System.

Furthermore, preprocessing validation was conducted to confirm that MRI scans and extracted tumor regions were correctly prepared before being passed to the classification model. This contributed to improving prediction quality and reducing classification errors.

The testing results demonstrated that the MRI classification model achieved effective performance in liver tumor classification and provided reliable diagnostic support for the overall LiverGuardian System.

5.2.4 AI Models Results Analysis

The evaluation and testing processes of the AI models were conducted to analyze the overall performance, reliability, and effectiveness of the segmentation and classification components within the LiverGuardian System. The analysis focused on measuring the accuracy of medical image processing and assessing the consistency of the generated outputs.

The CT segmentation model demonstrated strong performance in identifying liver structures and tumor regions from 3D CT scans. The generated segmentation masks showed high overlap with the ground truth annotations, indicating accurate segmentation capability and reliable tumor localization.

Similarly, the MRI segmentation model achieved effective results in detecting tumor boundaries and producing consistent segmentation outputs across different MRI cases. The model was capable of accurately identifying tumor regions while maintaining stable segmentation performance under varying imaging conditions.

The MRI classification model also demonstrated reliable classification performance during testing. The model successfully analyzed MRI tumor regions and generated accurate predictions for different tumor cases. Evaluation metrics such as Accuracy, Precision, Recall, and F1-Score indicated strong classification capability and prediction consistency.

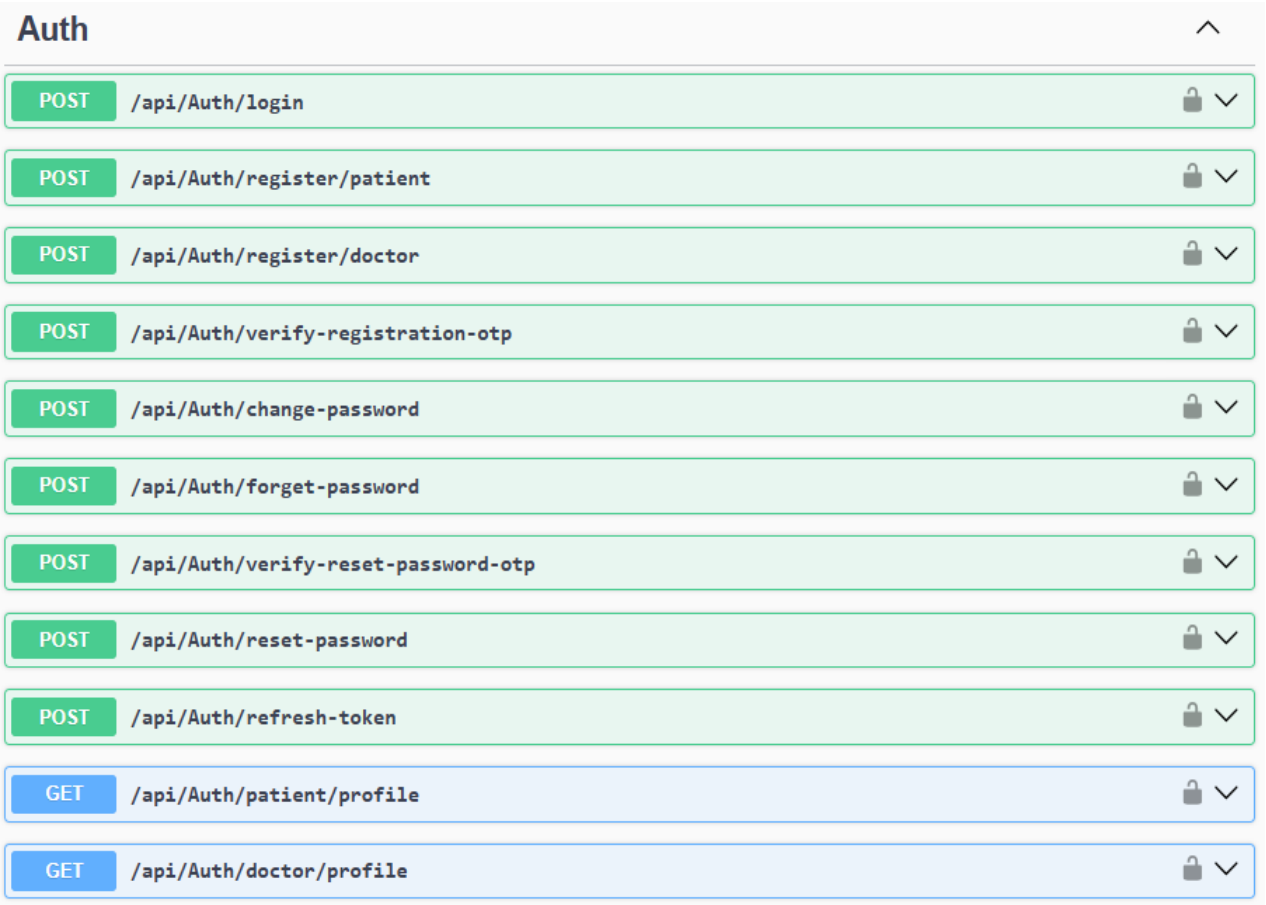
Error analysis was additionally performed to identify cases where segmentation or classification inaccuracies occurred. This analysis helped in understanding model limitations and identifying areas that may require future optimization and improvement.

The overall results confirmed that the AI models integrated within the LiverGuardian System achieved effective performance in medical image analysis tasks. The combination of segmentation and classification models provided reliable outputs that can support liver tumor detection and diagnostic assistance processes.

Furthermore, the modular architecture of the AI workflow allows future improvements, model retraining, and integration of larger medical datasets to further enhance system accuracy and robustness.

5.2.5 API Testing

Auth API Testing

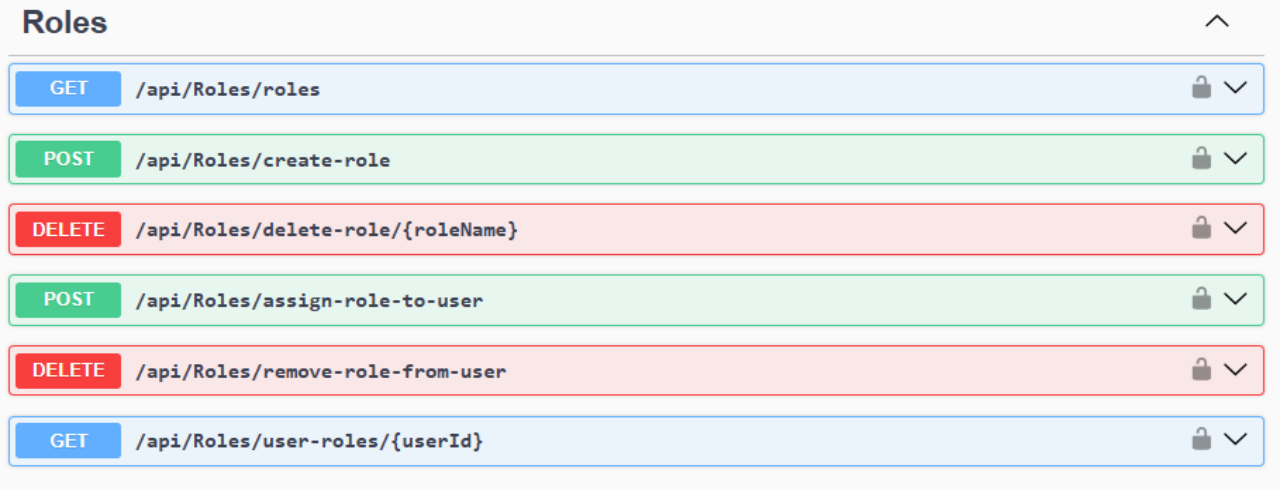


The Authentication APIs were tested to validate user registration, login, password recovery, token refresh, and profile management functionalities. The testing process focused on verifying authentication workflows, request handling, and response consistency.

Happy scenario testing was performed to ensure that valid requests returned the expected responses and correct status codes. Negative scenario testing was also conducted to validate system behavior when handling invalid credentials, missing parameters, unauthorized access attempts, and incorrect request data.

The testing results confirmed the reliability and security of the authentication and authorization mechanisms within the LiverGuardian System.

Roles API Testing

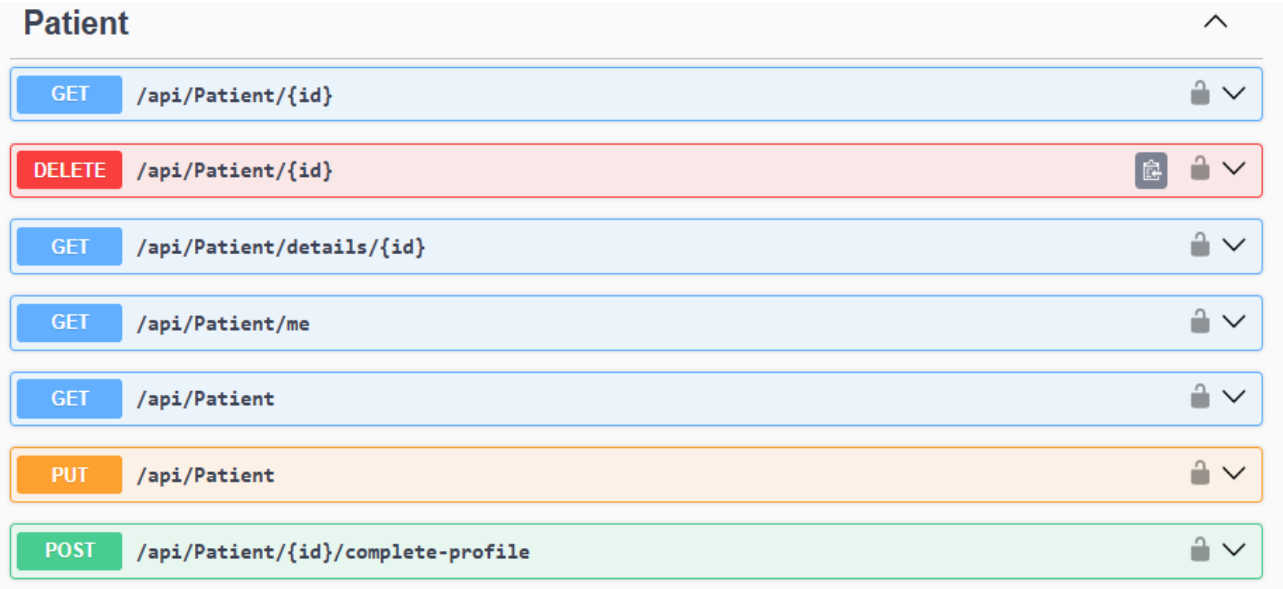


The Roles APIs were tested to validate role creation, assignment, removal, and permission management operations within the LiverGuardian System. The testing process focused on ensuring secure authorization handling and proper access control management.

Happy scenario testing was performed to verify successful role management operations and accurate response generation using valid request data. Negative scenario testing was also conducted to evaluate system behavior when invalid role names, unauthorized access attempts, or incorrect request data were submitted.

The testing results confirmed reliable role management functionality and secure authorization handling within the system.

Patient API Testing



The Patient APIs were tested to validate patient profile management, data retrieval, profile completion, and patient-related operations within the LiverGuardian System. The testing process ensured correct backend communication and response consistency.

Happy scenario testing was applied to verify successful profile retrieval and update operations with valid request data. Negative scenario testing was also performed to validate the system's behavior when invalid patient identifiers, missing parameters, or unauthorized requests were submitted.

The testing results confirmed reliable patient management operations and stable backend functionality.

5.3 Test Case Design

Test case design was an essential part of the testing process for the LiverGuardian System. Detailed test cases were created to validate the functionality, reliability, and expected behavior of the system under different operating conditions.

The designed test cases covered multiple testing scenarios, including positive scenarios, negative scenarios, boundary value testing, and input validation testing. This approach ensured comprehensive validation of both normal system behavior and unexpected user interactions.

Each test case included a unique identifier, testing objective, required inputs, execution steps, expected results, and actual results. This structured approach helped organize the testing workflow and simplified the process of tracking system issues and validating fixes.

The test cases focused primarily on backend API functionality, request handling, authentication mechanisms, data validation, and communication between system components. Additional test cases were also designed for user workflows and end-to-end system functionality.

Special attention was given to validating incorrect inputs, missing parameters, invalid authentication tokens, and unexpected request scenarios to ensure proper error handling and system stability.

The test case design process contributed significantly to improving software quality, detecting functional issues early, and ensuring that the LiverGuardian System met the required functional and non-functional requirements.

5.4 Integration Testing

Integration testing was conducted to verify the proper communication and interaction between the different components of the LiverGuardian System. The testing process focused on ensuring that the frontend, backend, database, and AI modules worked together correctly as a unified system.

The integration testing phase validated the data flow between system components and confirmed that requests and responses were transferred accurately without data loss or processing issues. Special attention was given to verifying communication between the backend APIs and the AI models responsible for segmentation and classification tasks.

Database connectivity testing was also performed to ensure that medical data, user information, and processing results were correctly stored, retrieved, and managed within the system. The testing process confirmed the consistency and integrity of data across different modules.

Additionally, frontend and backend integration testing verified that user actions within the application interface triggered the expected backend operations and generated accurate outputs. Error handling and synchronization between components were also validated to ensure stable system behavior.

The integration testing process helped identify communication issues, improve system stability, and ensure seamless interaction between all components of the LiverGuardian System.

5.5 End-to-End (E2E) Manual Testing

End-to-end manual testing was performed to validate the complete workflow of the LiverGuardian System from the user perspective. The testing process simulated real-world usage scenarios to ensure that the entire system operated correctly and produced the expected outputs.

The testing workflow included uploading medical imaging data, processing CT and MRI scans, executing segmentation and classification operations, and validating the generated outputs and results. Different user scenarios were manually tested to confirm system functionality and workflow consistency.

The manual testing process focused on verifying system behavior, user interaction, navigation flow, and output accuracy throughout the complete operational cycle of the application. This helped ensure that all integrated modules functioned together correctly within real usage conditions.

Special attention was given to validating user actions, handling invalid operations, and confirming that the system responded appropriately under different scenarios. The testing process also verified that outputs generated by the AI models were correctly displayed and managed within the application interface.

The end-to-end testing phase contributed significantly to improving the usability, stability, and reliability of the LiverGuardian System before deployment and further automation testing processes.

5.6 End-to-End (E2E) Automation Testing

Automation testing was introduced within the LiverGuardian System to improve testing efficiency, support regression testing, and reduce the time required for repetitive testing tasks. The automation process focused primarily on API validation and workflow automation.

Automated testing scripts were developed to validate backend endpoints, verify request and response behavior, and ensure consistent system functionality during repeated execution cycles. This helped improve testing reliability and reduced manual testing effort.

Automation testing also supported regression testing by ensuring that newly implemented features and updates did not negatively affect existing functionalities within the system. Automated workflows were executed to validate critical system operations and backend services.

Several testing tools and technologies were utilized during the automation testing process, including Selenium for automated workflow testing, Postman collections for automated API testing, and IntelliJ IDEA as the primary development environment for test automation implementation.

Although the automation testing phase is still under continuous improvement and expansion, the implemented automated testing workflows successfully demonstrated the ability to improve testing consistency, accelerate validation processes, and support future scalability of the testing framework.

The integration of automation testing within the LiverGuardian System contributes to enhancing software quality, improving maintainability, and supporting long-term system reliability.

5.7 Performance & Load Testing

Performance and load testing were conducted to evaluate the responsiveness, stability, and efficiency of the LiverGuardian System under different operating conditions. The testing

process focused on measuring system performance during medical image processing and user interactions.

The evaluation included measuring response times for backend APIs, CT and MRI processing operations, and AI model inference execution. Resource utilization such as CPU, GPU, memory usage, and processing time was also analyzed to assess system efficiency during execution.

Load testing was performed to evaluate the system's ability to handle multiple requests and maintain stable performance under increased workload conditions. The testing process verified that the system could process requests consistently without significant performance degradation.

Additionally, performance testing helped identify potential bottlenecks related to data processing, API communication, and AI model execution. The obtained results contributed to improving overall system optimization and operational efficiency.

The performance and load testing phase confirmed that the LiverGuardian System maintained stable operation and acceptable processing performance during different testing scenarios.

5.8 Security Testing

Security testing was conducted to ensure the protection, confidentiality, and integrity of medical and user-related data within the LiverGuardian System. Since the project handles sensitive medical information, security validation was considered an essential part of the testing process.

The testing process focused on validating authentication and authorization mechanisms to ensure that only authorized users could access protected system resources and functionalities. Access control validation was performed to verify secure user authentication procedures and permission management.

Input validation testing was also conducted to identify potential vulnerabilities related to invalid or malicious user inputs. Error handling mechanisms were verified to ensure secure system responses without exposing sensitive system information.

Additionally, medical data protection measures were evaluated to confirm secure handling, storage, and transmission of medical imaging data and patient-related information. The testing process aimed to ensure compliance with basic security and privacy requirements for medical systems.

Security testing contributed to improving system reliability, reducing potential security risks, and enhancing overall protection of sensitive medical data within the LiverGuardian System.

5.9 Accessibility & Usability Testing

Accessibility and usability testing were conducted to evaluate the ease of use, interface clarity, and overall user experience of the LiverGuardian System. The testing process focused on ensuring that users could interact with the system efficiently and navigate through its functionalities without difficulty.

The usability evaluation included testing interface organization, workflow simplicity, navigation consistency, and clarity of displayed outputs and system responses. Different user interaction scenarios were manually tested to validate the effectiveness of the application design and user experience.

Accessibility testing also focused on ensuring that important system functionalities and information were clearly visible and easy to access for users during system operation. User feedback and manual interaction analysis were considered during the evaluation process to identify potential usability improvements.

The testing phase contributed to improving user interaction quality, simplifying workflow navigation, and enhancing the overall usability of the LiverGuardian System.

5.10 Quality Assurance Measures

Quality assurance measures were implemented throughout the development and testing phases of the LiverGuardian System to ensure software reliability, consistency, and overall system quality.

The quality assurance process included continuous validation of system functionalities, bug tracking and reporting, testing documentation management, and verification of implemented fixes and improvements. Testing activities were organized to ensure that all system modules met the required functional and non-functional requirements.

Bug tracking procedures were applied to document identified issues, monitor their resolution status, and ensure proper retesting after fixes were implemented. This contributed to improving software stability and reducing the possibility of unresolved defects.

Additionally, validation procedures were continuously applied during testing phases to verify output consistency, backend stability, and AI model reliability. Regular testing and evaluation processes helped maintain software quality and ensured dependable system behavior.

The implementation of quality assurance measures significantly improved the overall reliability, maintainability, and stability of the LiverGuardian System and supported the delivery of a high-quality medical AI application.

Chapter 6

6.1 Conclusion

AI-Based Liver Cancer Detection System

This project successfully developed **LiverGuardian**, a comprehensive and highly interpretable AI framework for the early detection and analysis of liver cancer. By executing specialized data pipelines across three independent data modalities and deploying them into a full-stack web environment, all primary objectives were achieved.

The system demonstrated robust clinical and technical reliability across all developed modules:

- The **Clinical CatBoost model** achieved an overall predictive **Accuracy of 98.0%** and an **ROC-AUC of 99.0%**.
- The **3D CT Segmentation pipeline** yielded exceptional spatial localization with a **Dice Coefficient of 0.9094** and a **99.45% Binary Accuracy**.
- The **3D MRI multi-stage pipeline** effectively handled multi-sequence scans, achieving a **0.456 Final Mask Dice Score**, a **65.3% binary differentiation accuracy**, and a specialized sub-type accuracy of **70.6% for malignant** and **59.4% for benign** categories.

By delivering precise 3D volumetric segmentation masks directly onto medical scans, LiverGuardian addresses the "black-box" opacity barrier, providing clinicians with clear structural evidence for diagnostic interpretation. Ultimately, the system represents a practical, high-impact demonstration of applying advanced machine learning to one of Egypt's most pressing healthcare challenges—transforming heterogeneous medical data into actionable, life-saving diagnostic intelligence.

6.2 Future Work

- **Enriching the Dataset:** Collecting more annotated multi-sequence CT/MRI scans from Egyptian hospitals to mitigate class imbalances in the hierarchical classification tree.
- **Advanced Hyperparameter Tuning:** Applying Bayesian optimization (Optuna) to systematically search CatBoost and 3D deep learning hyperparameter spaces for performance gains.
- **Federated Learning:** Distributing model training across multiple clinical sites without sharing raw patient data, maintaining privacy while expanding the training scale.
- **Longitudinal Tracking:** Extending the platform to monitor volumetric tumor changes across serial scans over time to support treatment response monitoring.

- **Mobile Application:** Developing a lightweight mobile application for patient self-screening with offline clinical risk assessment capabilities.
- **Clinical Validation Study:** Conducting a prospective blinded study comparing LiverGuardian's diagnostic and segmentation accuracy against specialist radiologists.
- **Regulatory Compliance:** Initiating preparation for software-as-a-medical-device regulatory pathways to enable future clinical and commercial deployment.

References

[1] World Health Organization, "Liver Cancer Fact Sheet," 2023. Available: https://www.who.int/news-room/fact-sheets/detail/liver-cancer

[2] A. Forner, J. M. Llovet, and J. Bruix, "Hepatocellular Carcinoma," *The Lancet*, vol. 391, no. 10127, pp. 1301–1314, 2018.

[3] G. Litjens, T. Kooi, and B. E. Bejnordi, "A Survey on Deep Learning in Medical Image Analysis," Medical Image Analysis, vol. 42, pp. 60–88, 2017.

[4] K. Yasaka, H. Akai, A. Kunimatsu, O. Abe, and S. Kiryu, "Deep Learning with Convolutional Neural Network for Differentiation of Liver Masses at Dynamic Contrast-Enhanced CT," *Radiology*, vol. 286, no. 3, pp. 887–896, 2018.

[5] R. G. Barr, S. R. Wilson, and D. Lyshchik, "Contrast-Enhanced Ultrasound, CT, and MRI for Liver Lesion Characterization," *Radiology*, vol. 293, no. 3, pp. 644–656, 2019.

[6] Y. Wu, Q. Chen, "A Review of Deep Learning Approaches for Multimodal Image Segmentation of Liver Cancer," *Journal of Applied Clinical Medical Physics*, vol. 25, no. 10, 2024. https://doi.org/10.1002/acm2.14540

[7] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *MICCAI*, 2015, pp. 234–241.

[8] H. Rahman, T. F. N. Bukht, A. Imran, J. Tariq, S. Tu, and A. Alzahrani, "A Deep Learning Approach for Liver and Tumor Segmentation in CT Images Using ResUNet," Bioengineering, vol. 9, no. 8, p. 368, 2022. https://doi.org/10.3390/bioengineering9080368

[9] M. Gross, M. Balaguer-Montero, "A CT-Based Deep Learning-Driven Tool for Automatic Liver Tumor Detection and Delineation," *Cell Reports Medicine*, vol. 6, p. 102032, 2025. https://doi.org/10.1016/j.xcrm.2025.102032

[10] S. Huang, X. Nie, and K. Pu, "A Flexible Deep Learning Framework for Liver Tumor Diagnosis Using Variable Multi-Phase Contrast-Enhanced CT Scans," *Journal of Cancer Research and Clinical Oncology*, vol. 150, p. 443, 2024. https://doi.org/10.1007/s00432-024-05977-y

[11] R. V. Manjunath, A. Ghanshala, and K. Kwadiki, "Deep Learning Algorithm Performance Evaluation in Detection and Classification of Liver Disease Using CT Images," *Multimedia Tools and Applications*, vol. 83, pp. 2773–2790, 2024. https://doi.org/10.1007/s11042-023-15627-z

[12] A. Hänsch and colleagues,"Improving Automatic Liver Tumor Segmentation in Late-Phase MRI Using Multi-Model Training and 3D Convolutional Neural Networks," *Scientific Reports*, vol. 12, p. 12101, 2022. https://doi.org/10.1038/s41598-022-16388-9

[13] J. Wang and co-authors, "A Deep-Learning Approach for Segmentation of Liver Tumors in MRI Using UNet++," BMC Cancer, vol. 23, p. 1061, 2023. https://doi.org/10.1186/s12885-023-11432-x

[14] M. Lou, H. Ying, X. Liu, H.-Y. Zhou, Y. Zhang, and Y. Yu, "SDR-Former: A Siamese Dual-Resolution Transformer for Liver Lesion Classification Using 3D Multi-Phase Imaging," *arXiv:2402.17246*, 2024.

[15] A. Siam and co-authors, "Multimodal Deep Learning for Liver Cancer Applications: A Scoping Review," Frontiers in Artificial Intelligence, vol. 6, 2023. https://doi.org/10.3389/frai.2023.1247195

[16] I. Pinar and co-authors, "Automated 3D Liver Segmentation from Hepatobiliary Phase MRI for Enhanced Preoperative Planning," Scientific Reports, 2023. https://doi.org/10.1038/s41598-023-45379-3

[17] J. Feng and co-authors, "Research Progress on Machine Algorithm Prediction of Liver Cancer Prognosis after Intervention Therapy," American Journal of Cancer Research, vol. 14, no. 9, 2024.

[18] Y. Fu and co-authors, "Machine Learning Integrating MRI and Clinical Features Predicts Early Recurrence of Hepatocellular Carcinoma after Resection," Scientific Reports, 2025. https://doi.org/10.1038/s41598-026-36261-3

[19] P. Bilic, P. Christ, E. Vorontsov, and the LiTS Consortium, "The Liver Tumor Segmentation Benchmark (LiTS)," Medical Image Analysis, vol. 84, p. 102680, 2023.

[20] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, "MixUp: Beyond Empirical Risk Minimization," in ICLR, 2018.

[21] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *IEEE CVPR*, 2016, pp. 770–778.

[22] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the Inception Architecture for Computer Vision," in IEEE CVPR, 2016, pp. 2818–2826.

[23] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," in *ICLR*, 2019.